Cloud Computing

Exercises Part 1 (Foundation Services)

Stuttgart University
WS 2026/27
09-12-2026

Prof. Dr. Kristof Kloeckner
GM and CTO, IBM GTS (retired)
kristof.kloeckner@iaas.uni-stuttgart.de

Gerd Breiter
DE, IBM (retired)
gbreiter58@gmail.com

A Short Tour of a Commercial Cloud (AWS)



What is AWS?

- A **global** collection of ‘elastic’ **virtualized** resources
- A ‘**secure** cloud **platform**’ with more than 200 services in an expanding number of categories
- A giant ‘**toolbox** for builders’

Similar statements can be made for all significant commercial clouds.

Amazon 2Q26 Financial Report

- AWS grew 37% yty to $42.2B quarterly revenue
  - $169B FY Outlook (AWS CEO interview)
- AWS profit grew 63% to $16.6B
- AWS is app. 20% of Amazon revenue, but app. 60% of its profit
- Massive investment in AI (including Anthropic)
- New enterprise agreements (Siemens Energy, Vodafone, Danske Bank)
- Many new features announced with focus on AI

AWS Global Infrastructure in 2026

- 39 launched regions with 123 availability zones
- 2 more regions and 7 availability zones planned
- AWS European Sovereign Cloud
- 600+ CloudFront POPs
- Claude estimates 5 – 20M servers, based on extrapolation of historic data

https://aws.amazon.com/about-aws/global-infrastructure/regions_az/

Amazon's focus on 'builders' and business

- "**The broadest and deepest platform for today's builders**" (W. Vogels, CTO, AWS Summit NYC July 2019)
  - "We provide a toolbox, you pick"
  - Using most functions themselves to run and grow their own digital business
- **"Enabling digital transformation"** (A. Jassy, then AWS CEO, 2020)
- These messages were reinforced in the following years
  - **Builder Community Hub 2023**
- Agility, DevOps pipeline, operational principles
  - **'Well-architected Framework'**
- Toolkits, IDEs, Integration with e.g. Visual Studio, GitHub
- Microservices, containers, serverless across the stack
  - "You write the business logic, we do the heavy (infrastructure) lifting"
- Automation
- Security
- AI/Machine Learning (first separate keynote in 2020)
- Significant and growing investment in IoT

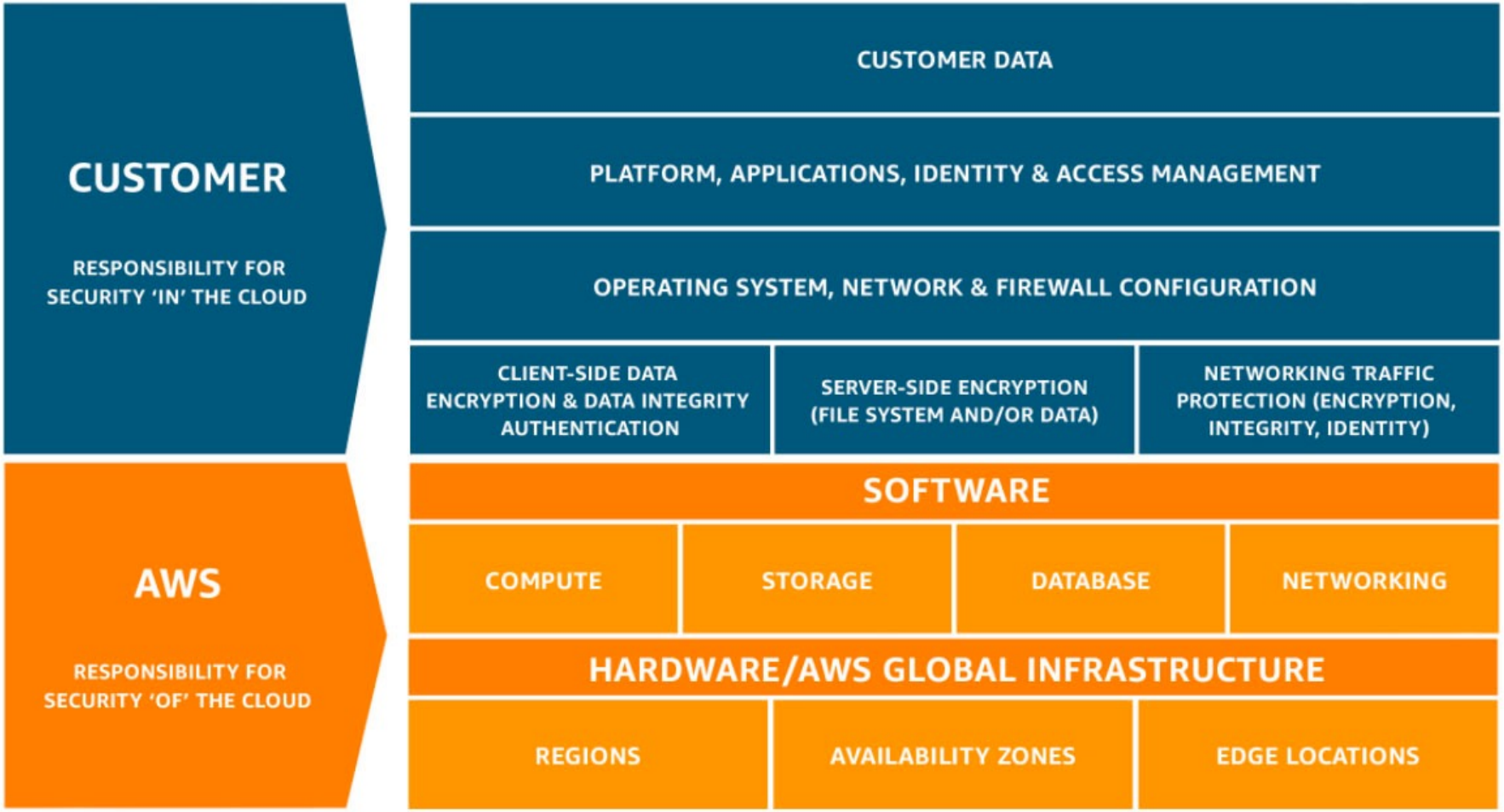
A Short Tour of AWS Services using the Learner Lab

- The AWS Console
- ‘The biggest toolbox for builders’ - AWS Services
- Some important examples of AWS services
  - Virtual Machines (EC2)
    - Your server in the cloud
  - Object Storage (S3)
    - Your photos etc. in the cloud
  - NoSQL Database (DynamoDB)
    - Very flexible and high performing – used for Amazon Shopping Cart
  - Simple Notifications (SNS)
    - News Feed etc.
  - Simple Queuing Service (SQS)
  - Serverless Functions (Lambda)
    - Handles events
- The Market Place

Other Clouds are similar; we will introduce them later!

Security and Compliance as Shared Responsibility between AWS and Customer

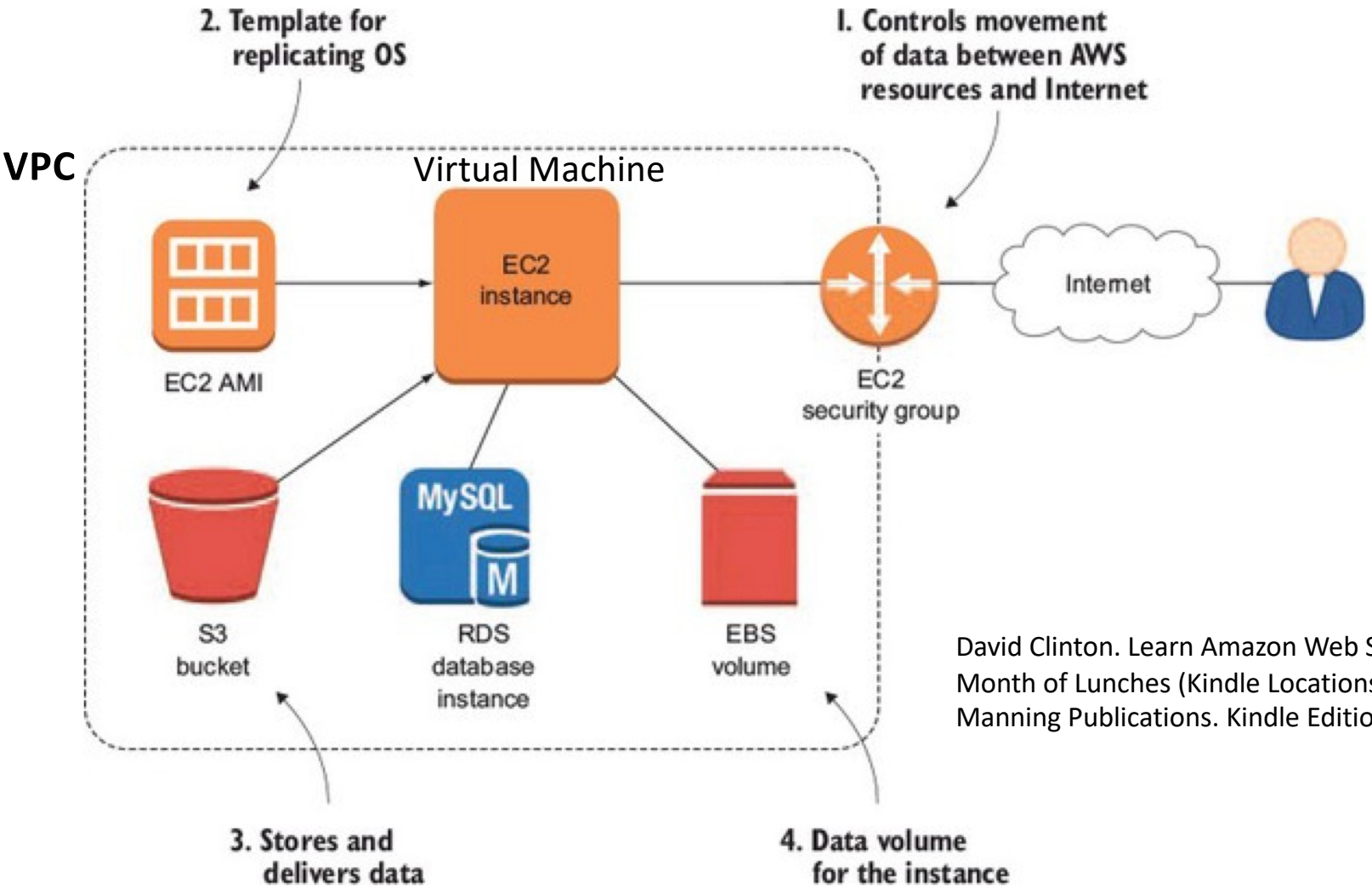
https://aws.amazon.com/compliance/shared-responsibility-model/



Some Basic Amazon (IaaS) Services

- EC2 – Elastic Compute Cloud
  - Consists of virtual machines (called EC2 instances) launched from Amazon Machine Images (AMIs)
- AMI – Amazon Machine Image
  - Templates for building EC2 instances
- VPC - Virtual Private Cloud
  - Private network that isolates your resources
- EBS – Elastic Block Storage
  - Persistent Storage Volumes that can be attached to an EC2 instance
- S3 – Simple Storage Service
  - Object Store Service
- These services plus Networking and Relational Database Services are called Foundational Services by AWS.

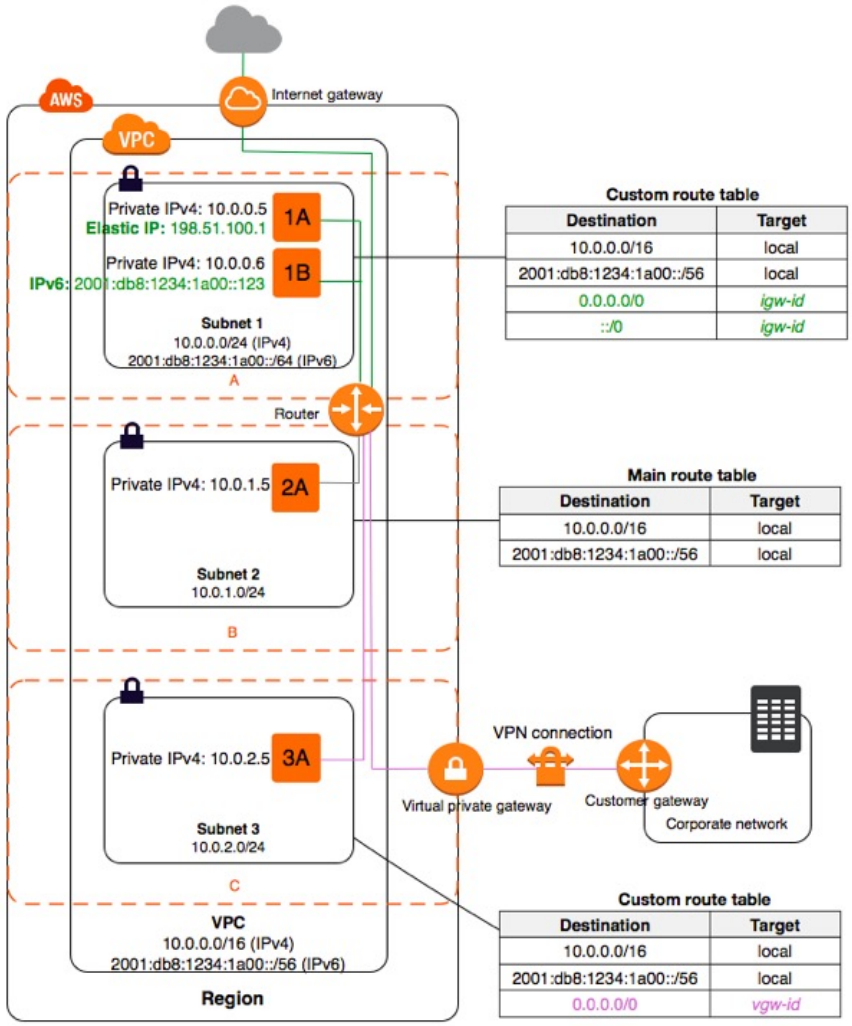
Hosting a Simple Web Application



David Clinton. Learn Amazon Web Services in a Month of Lunches (Kindle Locations 502-506). Manning Publications. Kindle Edition.

Cloud Networking

- A VPC lives in a region and can span availability zones
- A subnet is restricted to a single one
  - Some services require 2 subnets in different zones for enhanced availability
- A route table determines where traffic is routed
- Internet gateways and virtual private gateways
- Security groups control traffic (firewall rules)



Source: AWS Documentation

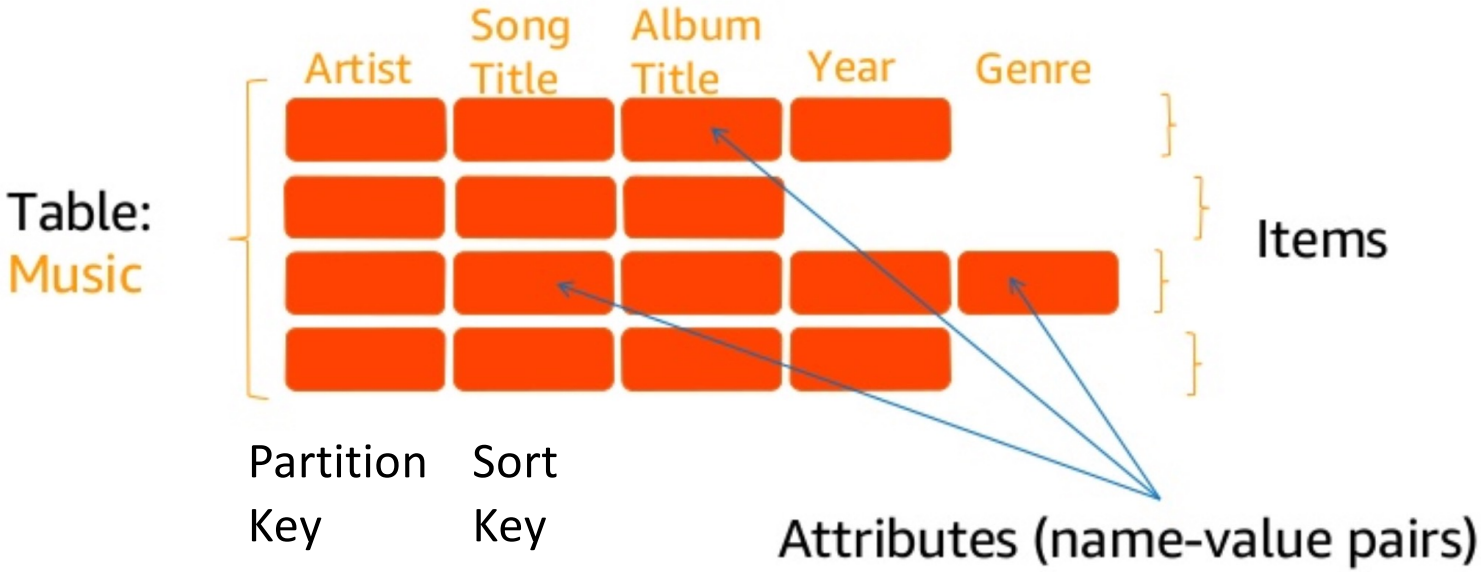
See also:

https://aws.amazon.com/vpc/?vpc-blogs.sort-by=item.additionalFields.createdDate&vpc-blogs.sort-order=desc

Storage and Databases

- Object Storage – S3
  - Objects have urls, are stored in ‘buckets’
  - Objects can be versioned
  - Objects can be arranged in ‘folders’
  - Buckets can host static websites
- Block Storage – EBS
  - Virtual Disks for Virtual Machines
- Relational Database Services (RDS)
  - Managed service with many options
- NoSQL DB (DynamoDB)
  - Managed Service
  - Key Value Store (Tables)
- Streams - Kinesis

DynamoDB Data Model

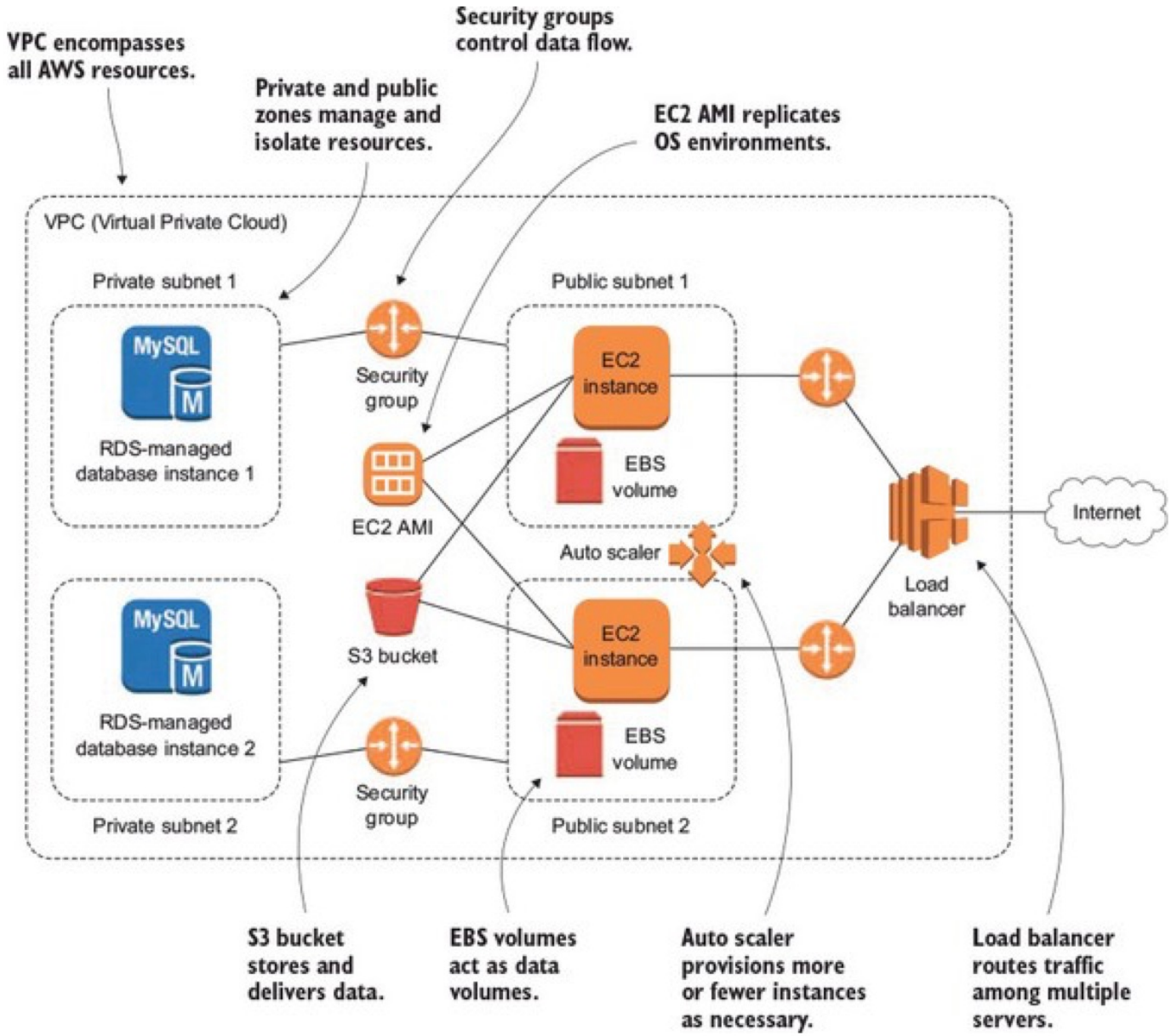


Attributes can be added after table creation

© 2019, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Source: Awsome Day 2019 Detroit Slideshare

Availability, Scaling, Security



Source: David Clinton, Learn AWS…

Integration Services

- Simple Notification Service – SNS
  - Topic based publish and subscribe service
  - Many services are able to publish or subscribe to a topic
- Simple Queuing Service – SQS
  - At-least-once delivery
- Step Functions
  - Basic Workflow
- Lambda Functions (AWS lists them under Compute)
  - Trigger-based event processing
  - Serverless computing (we will talk more about this)
  - More than 140 integrations

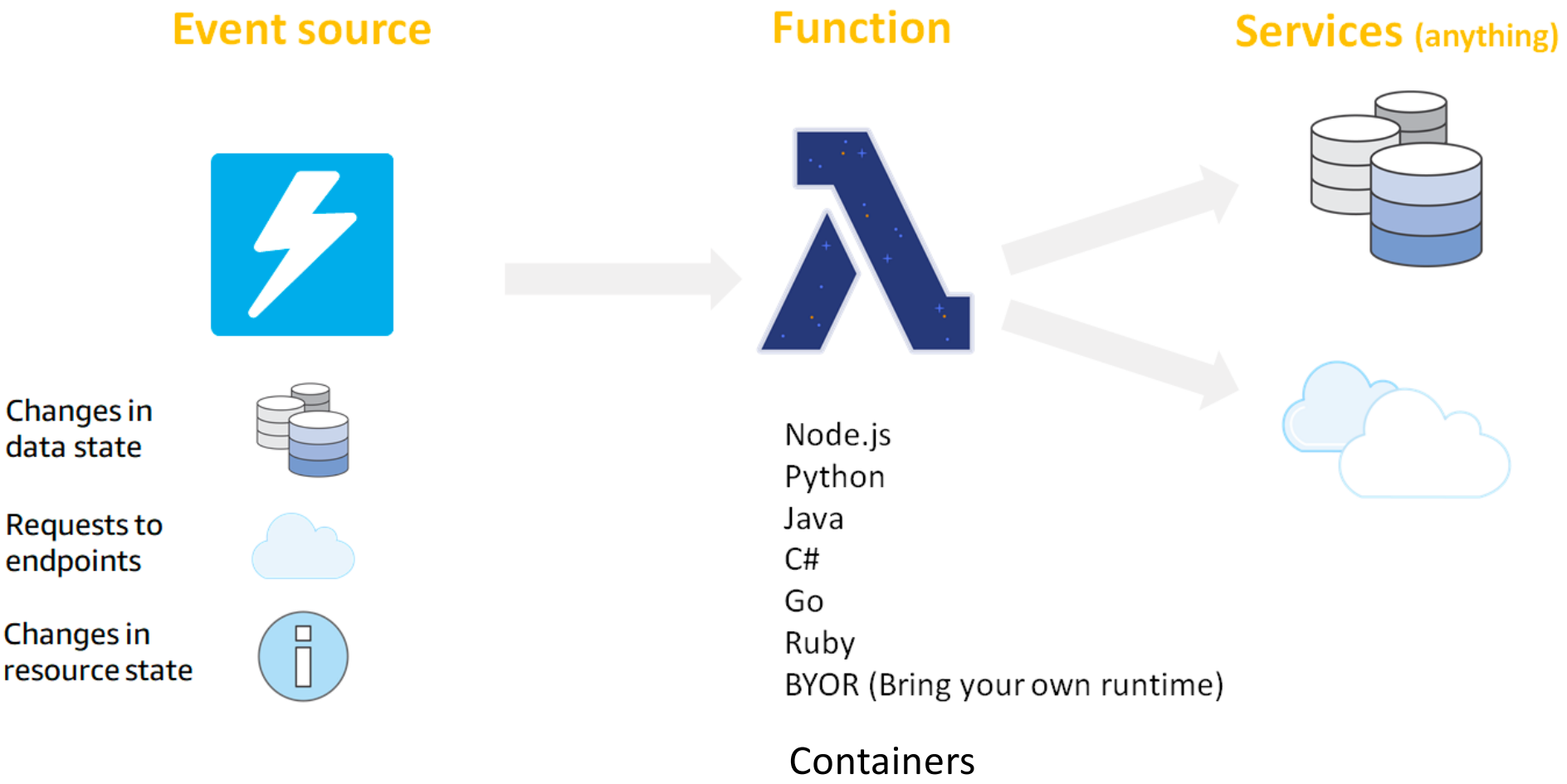
Amazon Simple Notification Service (SNS)

- (Managed) System-to-system messaging decoupling producers and consumers of messages through a publish/subscribe mechanism



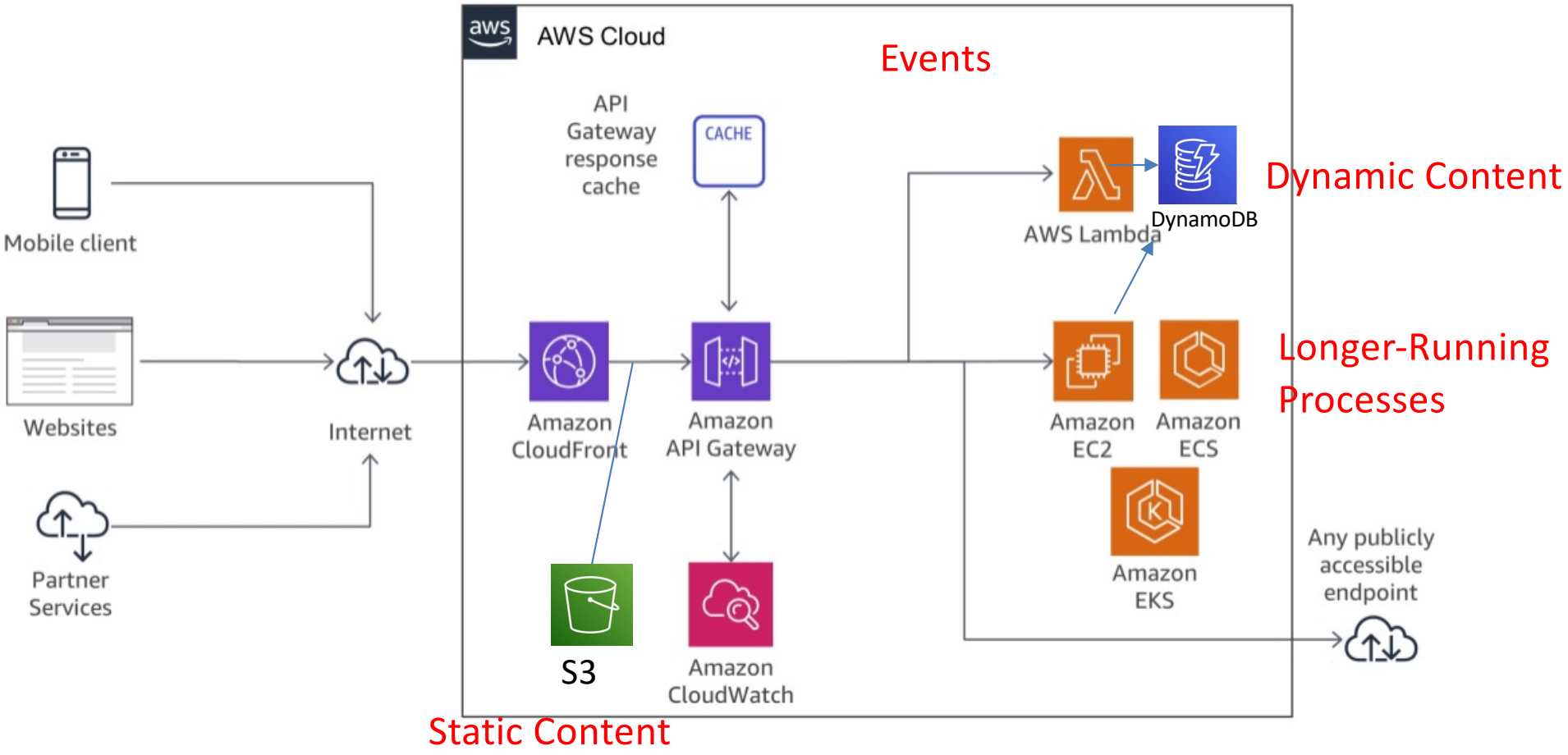
https://console.aws.amazon.com/sns/v3/home?region=us-east-1#/dashboard

Events, Functions, Services



https://aws.amazon.com/getting-started/deep-dive-serverless/?e=gs2020&p=gsrc

A Modern Application in AWS



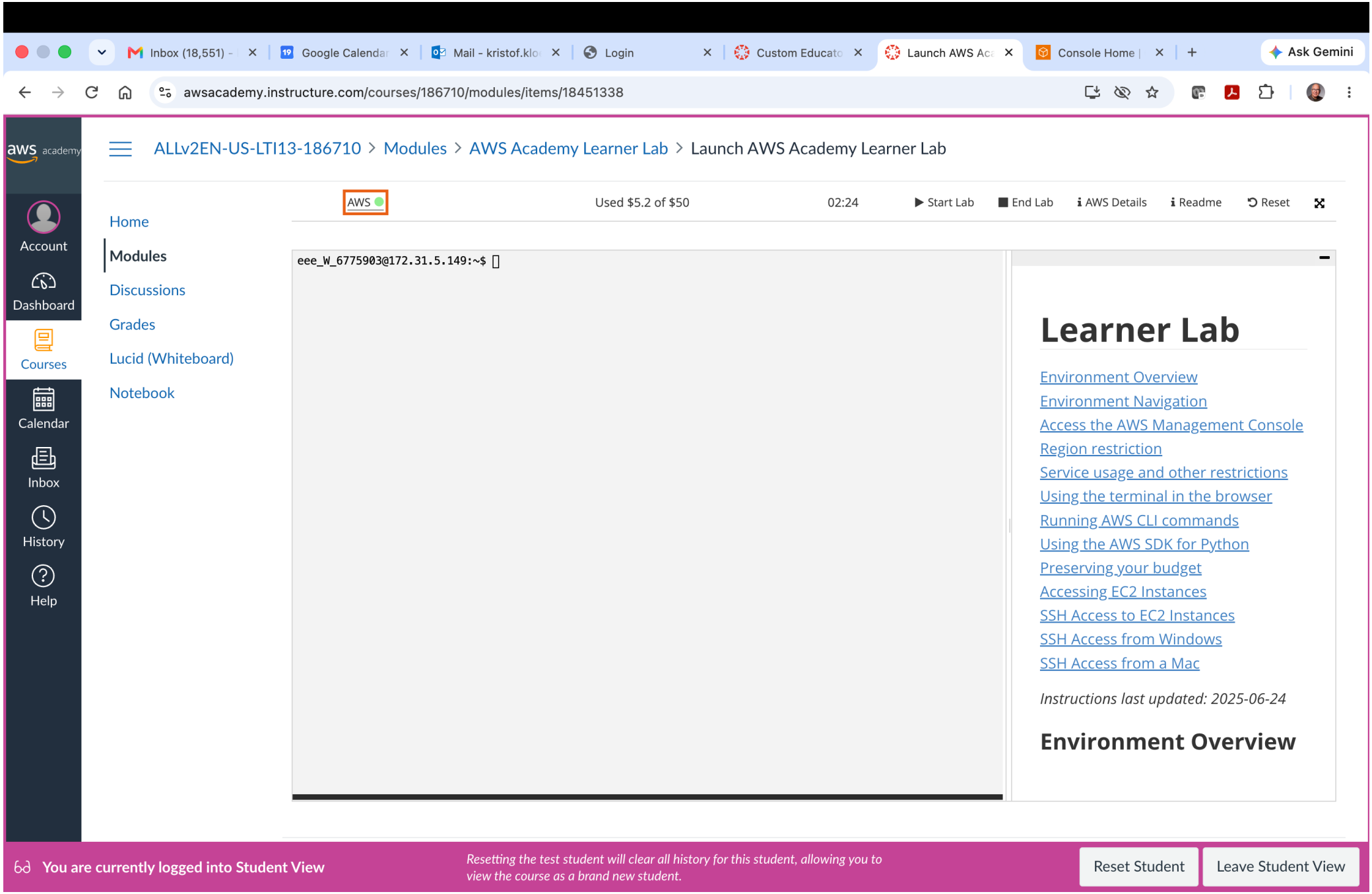
Source: Implementing Microservices in AWS

Using the AWS Learner Lab



Cloud Services in the Learner Lab

- Most AWS services are available, sometimes with capacity restrictions (see documentation)
- Limitation in using IAM, use the preconfigured role (**LabRole**) rather than creating new roles
- You can create your own key pairs, but a digital key (**vockey**) is also already available and can be used in the preconfigured terminal (~/.ssh/**labsuser**.pem).
- The Learner Lab comes with $50 credits, so stop or terminate your resources if you are not using them anymore. Some resources (like EC2 instances) are automatically restarted when starting a lab.
- Always shut down resources you don't need.
  - Relational Databases, Load Balancers, NAT Gateways continue to cost credit if they keep running


Inbox (18,551) -
Google Calendar
Mail - kristof.klo
Login
Custom Educato
Launch AWS Ac
Console Home |
Ask Gemini
awsacademy.instructure.com/courses/186710/modules/items/18451338
aws academy
ALLv2EN-US-LTI13-186710 > Modules > AWS Academy Learner Lab > Launch AWS Academy Learner Lab
Account
Dashboard
Courses
Calendar
Inbox
History
Help
Home
Modules
Discussions
Grades
Lucid (Whiteboard)
Notebook
AWS
Used $5.2 of $50
02:24
Start Lab
End Lab
AWS Details
Readme
Reset
eee_W_6775903@172.31.5.149:~$
Learner Lab
Environment Overview
Environment Navigation
Access the AWS Management Console
Region restriction
Service usage and other restrictions
Using the terminal in the browser
Running AWS CLI commands
Using the AWS SDK for Python
Preserving your budget
Accessing EC2 Instances
SSH Access to EC2 Instances
SSH Access from Windows
SSH Access from a Mac
Instructions last updated: 2025-06-24
Environment Overview
You are currently logged into Student View
Resetting the test student will clear all history for this student, allowing you to view the course as a brand new student.
Reset Student
Leave Student View

Cloud in Practice
Working with AWS Part 1



Modules

1. Foundation (ECS, ECB, S3, RDS)
2. DynamoDB, Management (CLI, CloudShell, SDK, Security, Monitoring)
3. Availability and Scaling
4. Additional Material (Networking)
5. Automation

Module 1
Foundation



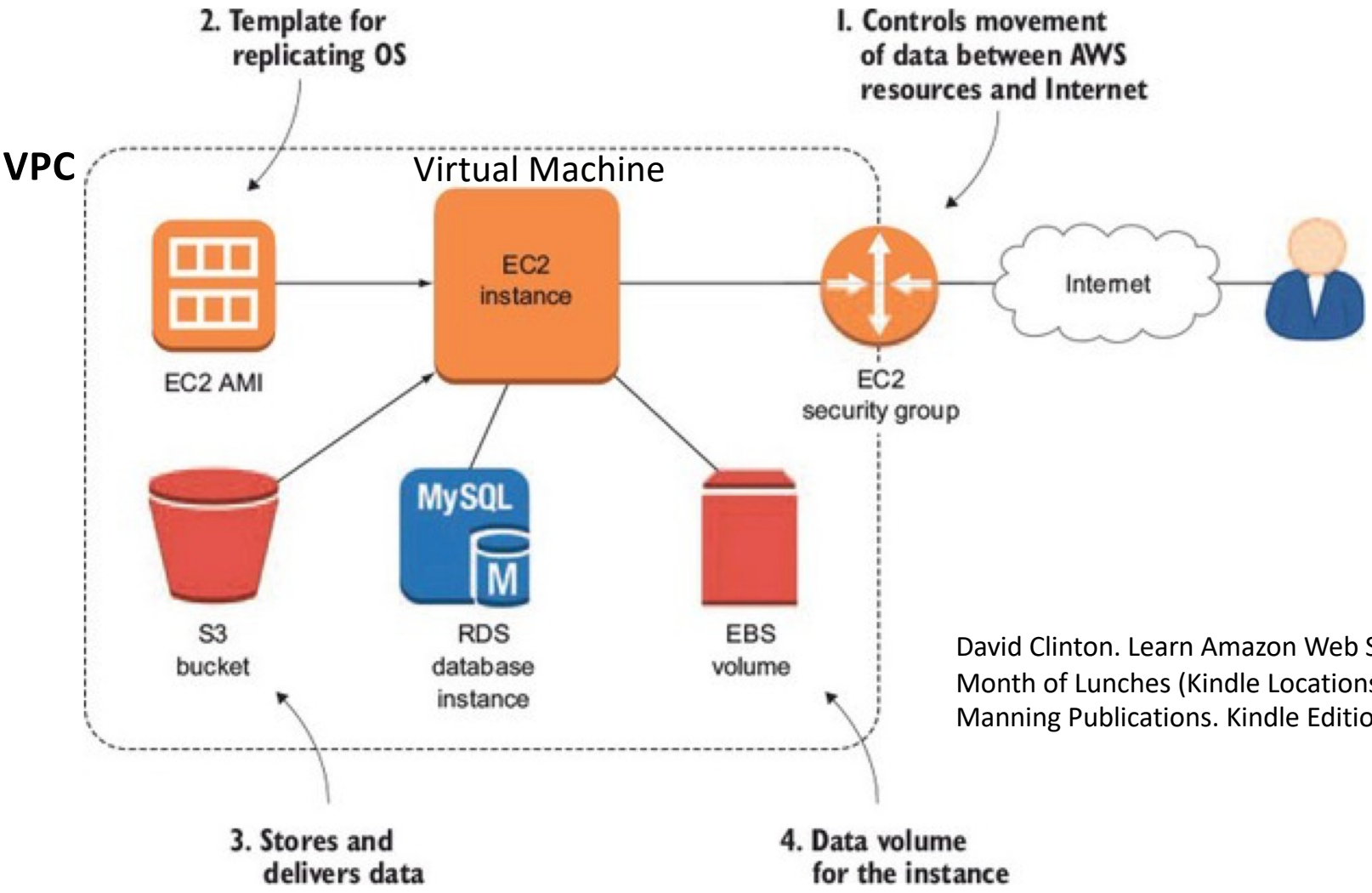
Cloud Foundational Services

- Imagine you want to host a blog on your own (virtual) server and make it visible to the Internet
  - WordPress is a popular open-source blogging tool and content management system based on LAMP, used by ~ 40% of Internet sites
  - The LAMP stack consists of Linux, Apache, MySQL, PHP
- We are going to go through the steps of acquiring and setting up the resources to do this as a cloud service with AWS.
- We are going to use compute, storage and network services, as well as managed database services to run a WordPress server
  - Starting with a **single virtual machine (an instance of EC2 or Elastic Compute Cloud)**, **in a private network (VPC or Virtual Private Cloud)**
  - Later expanding to more complex environments
  - Introducing basic management

Some Amazon (IaaS) Terminology

- EC2 – Elastic Compute Cloud,
  - Consists of virtual machines (called EC2 instances) launched from Amazon Machine Images (AMIs)
- AMI – Amazon Machine Image
  - Templates for building EC2 instances
- VPC - Virtual Private Cloud
  - Private network that isolates your resources
- EBS – Elastic Block Storage
  - Persistent Storage Volumes that can be attached to an EC2 instance
- S3 – Simple Storage Service
  - Object Store Service
- These services plus Relational Database Services are sometimes called Foundational Services.

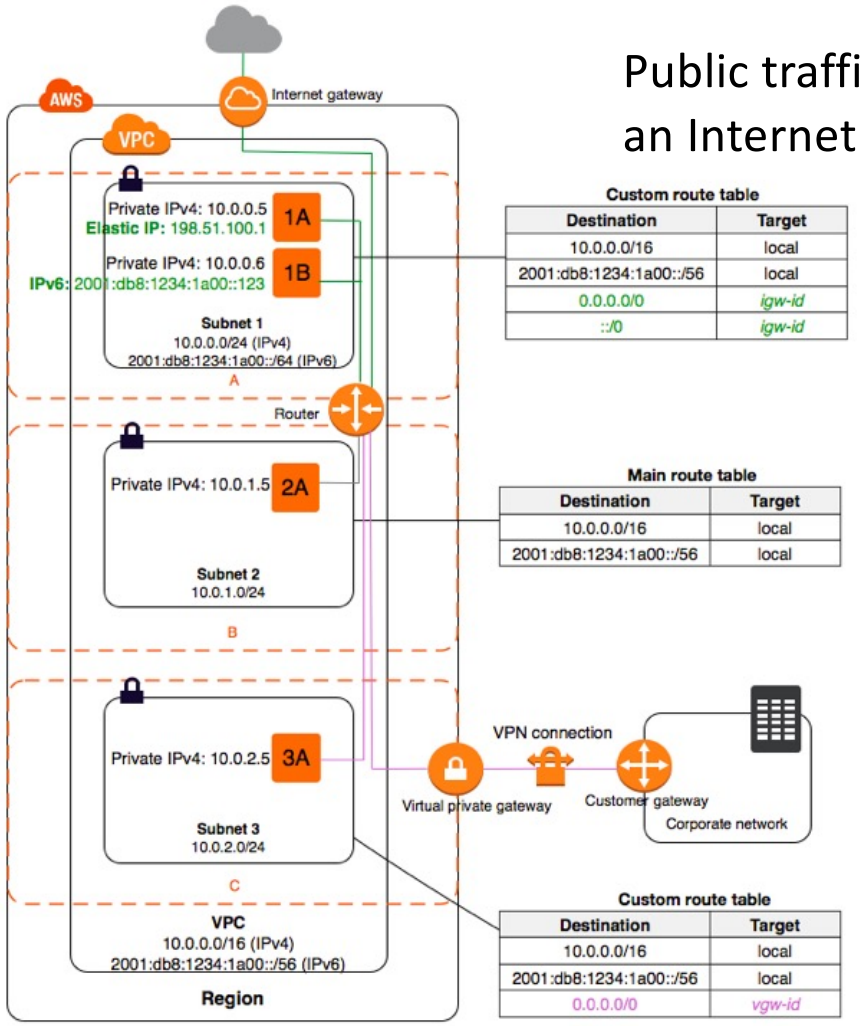
Hosting a Simple Web Application



David Clinton. Learn Amazon Web Services in a Month of Lunches (Kindle Locations 502-506). Manning Publications. Kindle Edition.

VPC with Subnets

- A VPC can span availability zones
  - Netmask 16, e.g. 10.0.0.0/16
- A subnet is restricted to a single one
  - Some services require 2 subnets in different zones
  - Netmask 24, e.g. 10.0.1.0/24
- A route table determines where traffic is routed
- Internet gateways and virtual private gateways
- Security groups control traffic (firewall rules)



Public traffic through an Internet Gateway

Local

VPN

Source: AWS Documentation

CIDR = Classless Internet Domain Routing

See also:

https://aws.amazon.com/vpc/?vpc-blogs.sort-by=item.additionalFields.createdDate&vpc-blogs.sort-order=desc

Setting up an EC2 instance using the console

- From **EC2,** select 'Launch Instance' and follow instructions – we mostly use defaults
  - Select region and availability zone
  - Choose a preconfigured Amazon Machine image (Ubuntu), and instance type (free tier)
  - Note: Amazon offers Amazon Linux 2023, also available on premise
- Set up the instance with **VPC** defaults**,** choose subnet, **enable public IP**
  - Default VPC has netmask 16 (to leave room for subnets)
  - **Static IP** will change after every restart, **elastic IP** is kept for the life of the instance, but incurs an ongoing cost
  - **Set up the security group to allow SSH (port 22) and HTTP (port 80) access**
- After the launch, use '**Connect**' to access your instance
- Alternatively, use a terminal on Linux/Mac or WLS)
  - Create or reuse a **private key, download** (from AWS details) and **reduce privileges** for private key access
    - chmod 400 keyname.pem (only on Linux/Mac or WLS)
    - Learner lab uses default key 'vockey', which downloads as labsuser.pem
  - Use SSH to access your instance from a Mac/Linux terminal (or Powershell)
    - ssh -i keyname.pem ubuntu@public-ip
    - Note: The standard user in AWS Linux is called ec2-user

More about Connecting to a Linux Instance

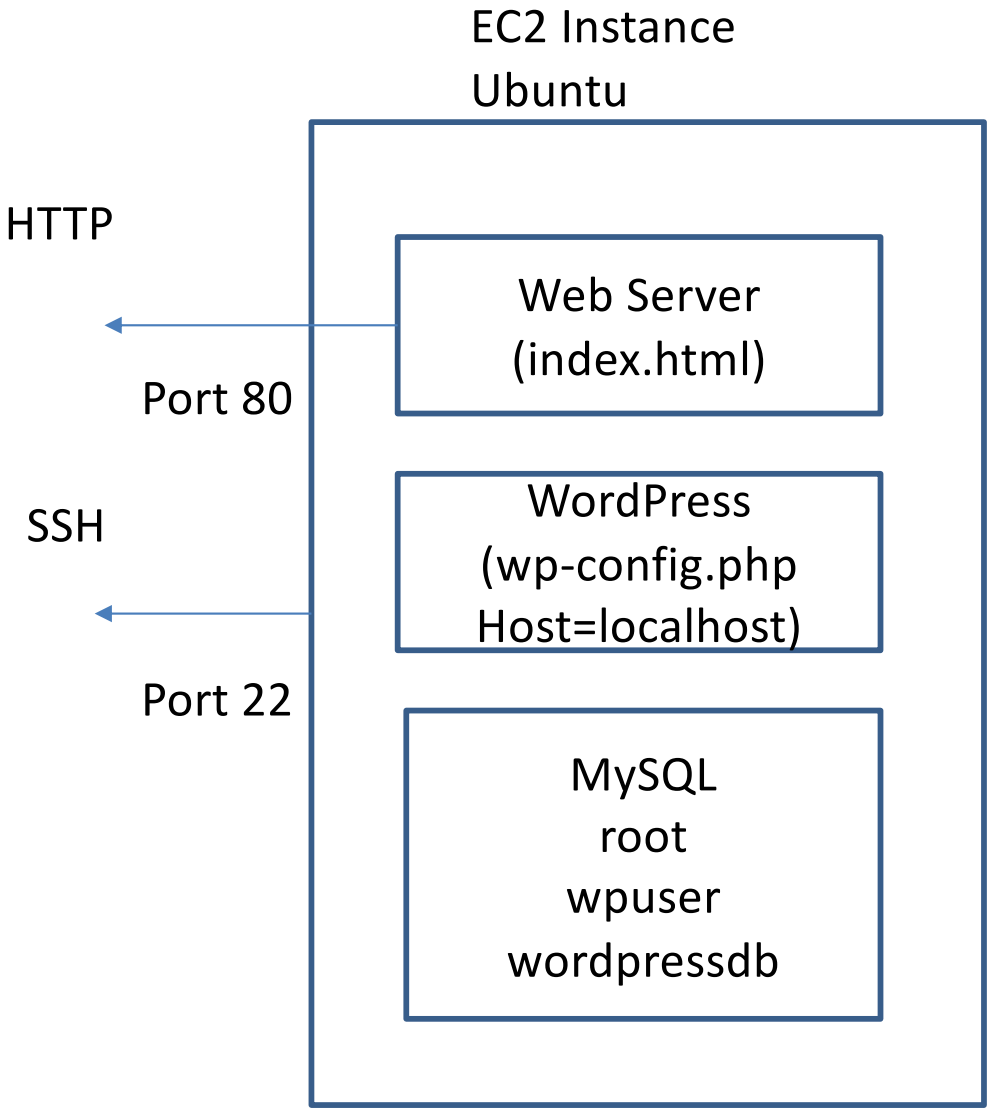
- https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AccessingInstancesLinux.html
- If you have problems with SSH on Windows, use PuTTY to connect to user_name@public_dns_name.
  - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/putty.html
- Transferring a file with scp
  - scp -i /path/my-key-pair.pem /path/SampleFile.txt ubuntu@ec2-198-51-100-1.compute-1.amazonaws.com:~
  - **For Amazon Linux replace ubuntu with ec2-user**
  - You may have to install the scp client on your Linux machine
- **I recommend using the Connect function in the EC2 dashboard**

Some useful Linux commands for managing your instance

- For a **very basic tutorial** see e.g.

  https://www.digitalocean.com/community/tutorials/an-introduction-to-linux-basics
- Command line (shell) is similar to Powershell
  - command –option1 –option2
- sudo command1 – assume superuser execution privileges for command1
- pwd – gives present working directory
- ps – lists active processes for user, ps –e lists all processes
- ls – lists a directory
- cd directory1 – changes directory to directory1
- cp file1 file2 – copies file1 into file2
- mv file1 file2 – renames file1 to file2
- nano – simple editor
- cat file – print content of file to terminal
- man command1 – gives manual page for command1

WordPress Configuration with ‘localhost’



Configuring your (Ubuntu) Instance

LAMP

- Install a **LAMP** Server (Linux, Apache, MySQL, PHP)
  - sudo apt update *apt is the Debian Advanced Package Tool*
  - sudo apt upgrade
  - sudo apt install lamp-server^
  - sudo mysql

    >ALTER USER 'root'@'localhost' IDENTIFIED WITH caching_sha2_password BY 'YourNewPassword'; (MySQL Version 8.4 +)

    >FLUSH PRIVILEGES;

    >exit
  - sudo mysql_secure_installation
- Modify the starter web page (index.html)
  - cd /var/www/html
  - sudo mv index.html index.html.backup
  - Edit the starter page (with the 'nano' editor)
- From browser, go to http://your-public-ip
  - You should see the modified index page

Configuring your instance

WordPress and Database

- Log in (see prior page) and add wpuser and wordpressdb to mysql service
  - sudo mysql –u root –p (login with prompt for password)
  - mysql> CREATE DATABASE wordpressdb;

    Query OK, 1 row affected (0.00 sec)
  - mysql>CREATE USER 'wpuser'@'localhost' IDENTIFIED BY 'AWStest2026';

    Query OK, 0 rows affected (0.00 sec)
  - mysql> GRANT ALL PRIVILEGES ON wordpressdb.* To 'wpuser'@'localhost';

    Query OK, 0 rows affected (0.00 sec)
  - mysql> FLUSH PRIVILEGES;

    Query OK, 0 rows affected (0.00 sec)
  - mysql> exit
- Download and configure WordPress (always use sudo after wget)
  - wget https://wordpress.org/latest.tar.gz *downloads a 'tar' archive*
  - sudo tar xzf latest.tar.gz (unpack), go to wordpress directory
  - sudo cp wp-config-sample.php wp-config.php
  - Modify this file to point to' wordpressdb', 'wpuser' and 'AWStest20256
  - Copy wordpress directory: sudo cp –r * /var/www/html
- Before completing the setup, consider assigning an elastic IP address and creating an AMI
  - Complete the setup in your browser with public-ip/wp-admin/install.php

Database Commands and Tips

- See also dev.mysql.com for further documentation
- For instance:
  - show databases;
  - use [dbname]; *select a specific db to apply subsequent commands to*
  - show tables;
  - create database {dbname]; *create a database*
  - drop database [dbname]; *delete a database*
  - create table[table name]; create a table (similar for drop)
  - SELECT * FROM [table name] WHERE [field name] = "value"; *show rows with a particular value*
  - UPDATE [table name] SET field_name1 = 'fieldvalue1' WHERE [field _name2] = 'field_value2'; *replace a field value for all rows with a specific value in (another) field*

Database Commands and Tips…

- Further commands
  - select user from mysql.user; *list the users*
  - drop user *username*; *remove user*

Comments on WordPress

- During installation, Wordpress puts the public IP address into a wp_options table in wordpressdb under option_name ‘siteurl’ and ‘home’
- If your IP address changes after a restart, Wordpress will not work anymore.
- You can reinstall Wordpress by dropping and then recreating wordpressdb.
- You can also modify the table in the following way with your new IP address
- In MySQL:
  - use wordpressdb;
  - select * from wp_options where option_name IN('siteurl','home’);
  - UPDATE wp_options SET option_value='http://your-IP’ WHERE option_id=‘2’; *or altneratively use option_name = 'siteurl’*
  - UPDATE wp_options SET option_value='http://your-IP’ WHERE option_id=’1’; *or alternatively option_name = ‘home’*
- See also https://wordpress.org/support/article/changing-the-site-url/ for additional approaches
- **To avoid this, allocate an elastic IP address to your instance BEFORE you execute wp-install.php**

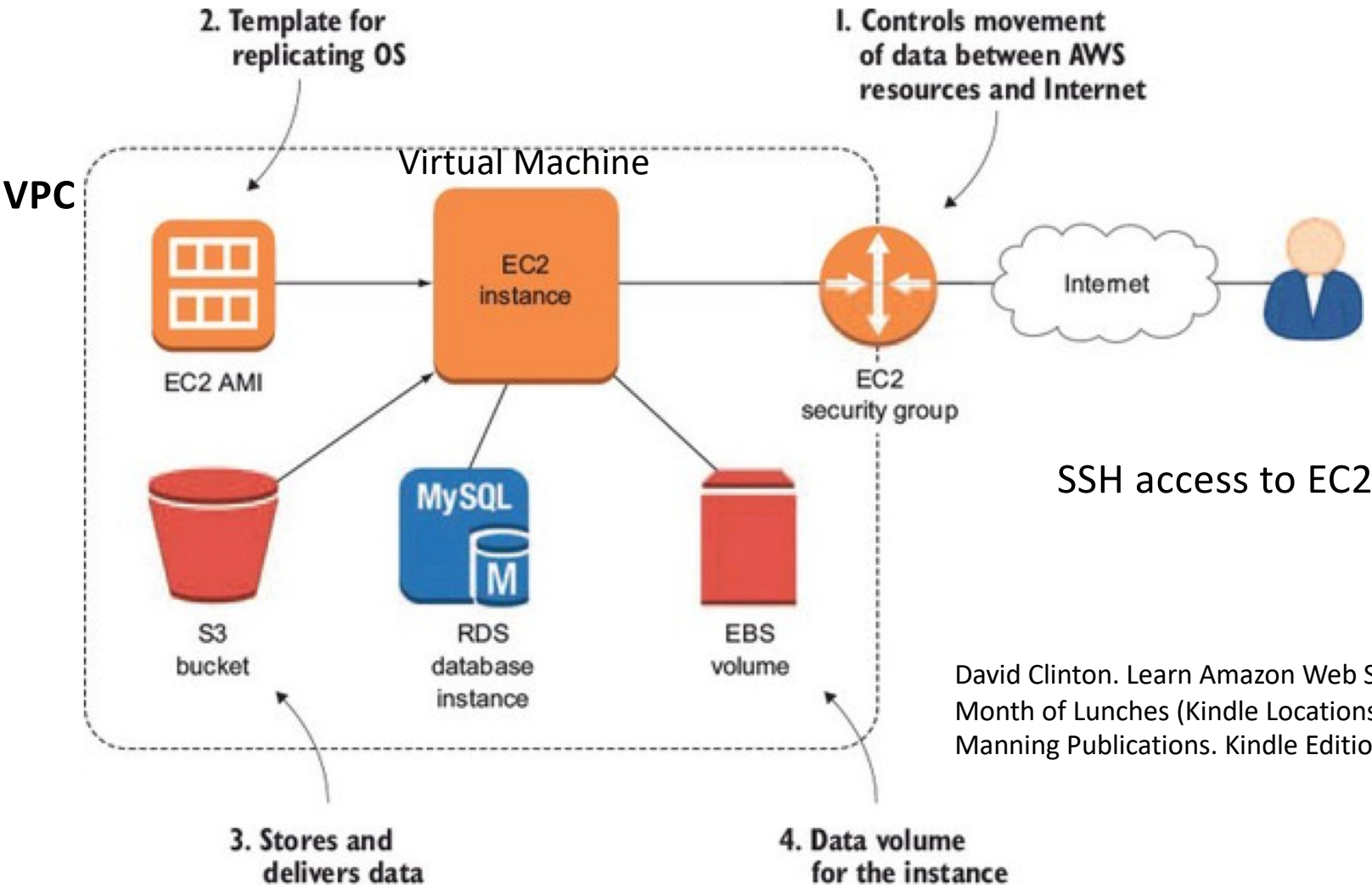
Configuring an Amazon Linux Image with LAMP

- https://docs.aws.amazon.com/AWSEC2/latest/UserGuide.html
- Amazon Linux 2023 is derived from Fedora and CentOS
- **Amazon uses its own extension manager and also uses yum**
- $ sudo yum update -y
- $ sudo amazon-linux-extras install -y lamp-mariadb10.2-php7.2 php7.2
- $ sudo yum install -y httpd mariadb-server
- For other configuration, see documentation link

Module 1 – ctd.
EC2, EBS, S3



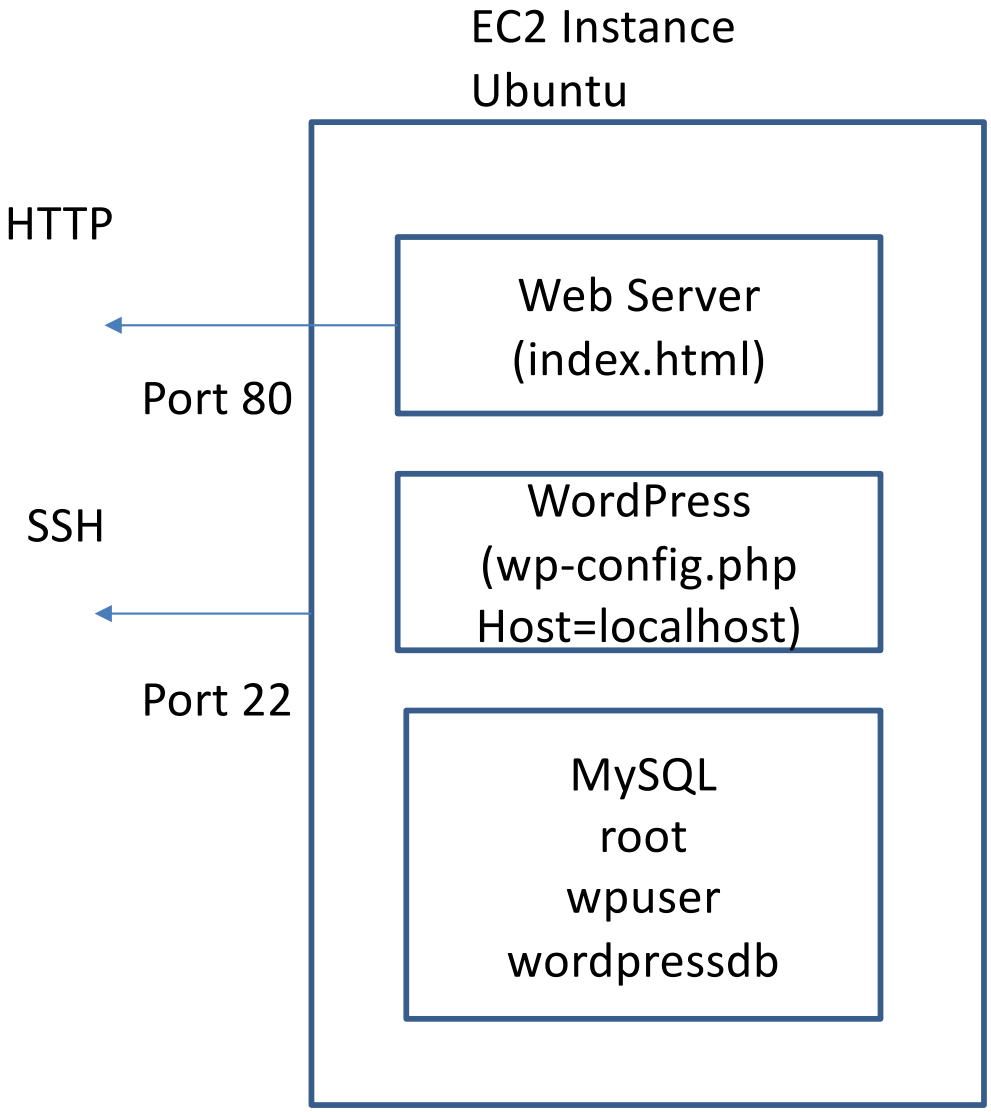
What we are building



SSH access to EC2

David Clinton. Learn Amazon Web Services in a Month of Lunches (Kindle Locations 502-506). Manning Publications. Kindle Edition.

Our WordPress Configuration



We created an AIM before completing the installation.

We are going to use it to set up a clone.

Cloning an Instance

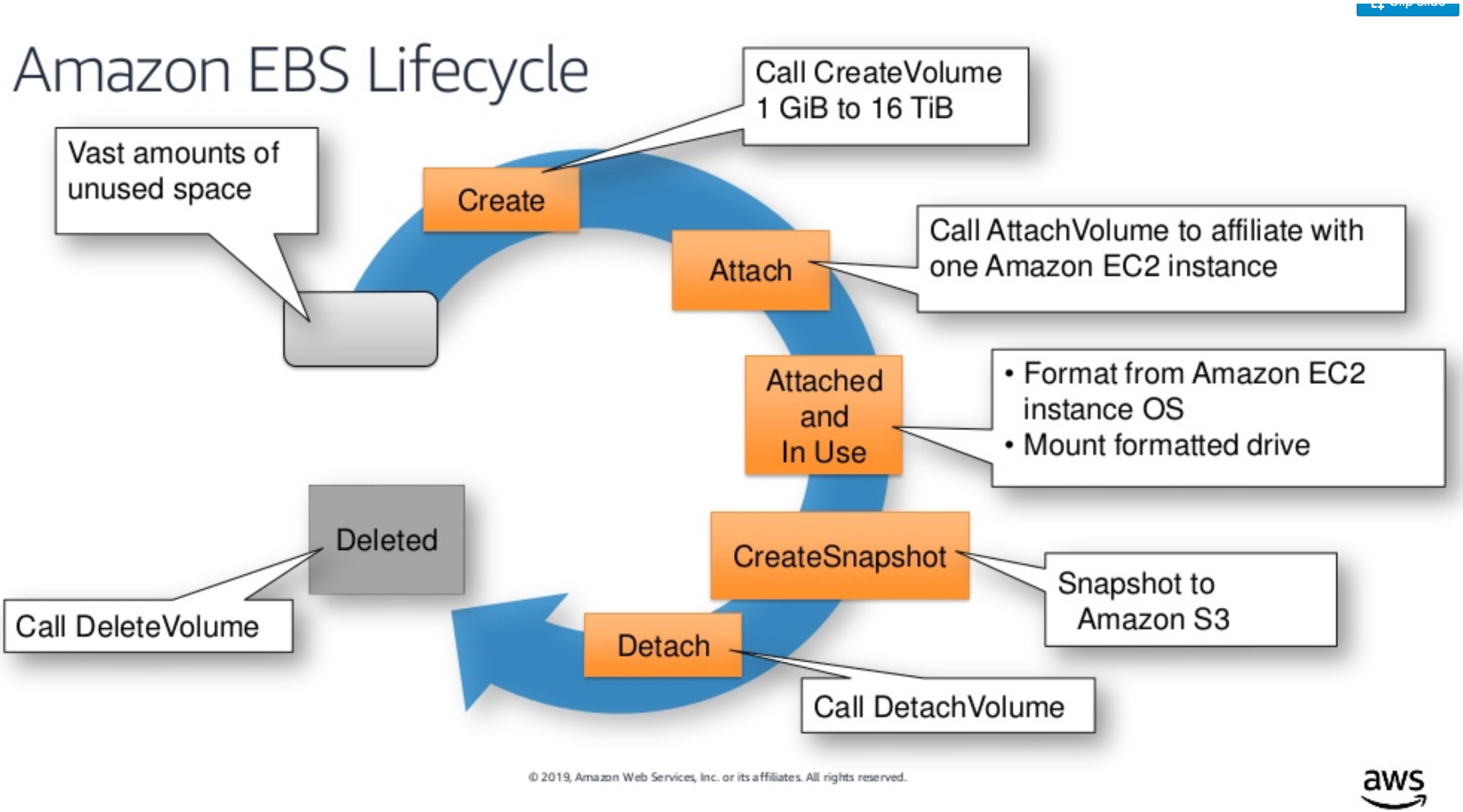
- Select an instance, click on ‘Actions’, select ‘Images and templates’
- Choose one of three options
  - **Create an Image**
    - EC2 creates a snapshot of the instance (all changes since launch)
    - At launch, you can choose all parameters
    - You can find it under ‘AMI’
    - AMIs can be public or private, you can give explicit permissions to other accounts
  - **Create a template from an instance**
    - Creates a blueprint (configuration, but no snapshot of changes)
  - **Launch more like this**
    - Launches an exact clone, but allows editing of configuration

Adding Block Storage

- With the console, create an EBS (Elastic Block Storage) volume and attach it as /dev/sdf
  - Note: **A volume is only accessible within its availability zone**
  - Use **lsblk** to determine names of block devices
  - Could be called /dev/xvdf or /dev/nvme1n1 in Ubuntu
- Format volume, create mount point, mount with the link below
  - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-using-volumes.html
- EBS persists beyond the life of an instance, while EC2 instance storage is ephemeral
- The default type is SSD (optimized for IOPS), but HDD is available (for massive throughput)
  - $ 0.1 per GB-month (4x the price of S3)
- EBS is automatically replicated within its availability zone

A few useful commands

- lsblk — *List block devices*
- sudo file -s /dev/xvdf — *Check if a file system exists*
- sudo mkfs -t xfs /dev/xvdf — *Make a file system*
- sudo mkdir /data — *Make a mount point directory*
- sudo mount /dev/xvdf /data — *Mount*
- sudo umount /dev/xvdf — *Unmount*



Source: Awsome Day 2019 Detroit Slideshare

Elastic File System

- An NFS (Network File System) managed distributed file system
- Can be attached to multiple EC2 instances and used by any Linux-based service
- On-premise can be included
- Use if sharing is needed or capacity demand unclear, otherwise EBS is cheaper and faster
- Only I/O and used storage capacity is charged for
- It is not available with AWS Academy Learner Labs

Object Storage: Simple Storage Service (S3)

- Object storage (in ‘buckets’)
  - Objects have urls
  - Objects can be versioned
  - Objects can be arranged in ‘folders’
- Significantly cheaper than EBS file storage
  - 2.3$ vs. 10$/100GB/month
  - Used internally by AWS to store snapshots, configuration templates etc.
  - Over 100 Trillion objects stored worldwide
- Designed for 11 nines durability
- Wizard-driven creation and management
- Different redundancy and encryption properties available
- Since 2021, S3 has read-after-write consistency
- Lifecycle policies to offload or delete buckets
- S3 can be used to host static websites
- You can offload very infrequently used data to Amazon S3 Glacier low cost storage ($1/month for 1TB)

A Static Website: Displaying an image from an S3 bucket

- From S3 Dashboard, select a bucket
- Go to ’Management’ in pull-down
- Go to ‘Properties’, enable static web hosting
- Create an index.html file and upload it to the bucket. Make public.
- A sample index.html showing one image from an S3 bucket

```
<div align="center">
<br><br>
<hr>
<b>
<img style="-webkit-user-select: none;margin: auto;cursor: zoom-in;" src="https://krk-2bucket.s3.amazonaws.com/PEI.jpeg" width="558" height="745">
<hr>
```

- Invoke with bucket endpoint

  http://krk-bucket1.s3-website-us-east-1.amazonaws.com

Amazon Simple Notification Service (SNS)

- (Managed) System-to-system messaging decoupling producers and consumers of messages through a publish/subscribe mechanism
- Works with many AWS services as publisher or subscriber



https://console.aws.amazon.com/sns/v3/home?region=us-east-1#/dashboard

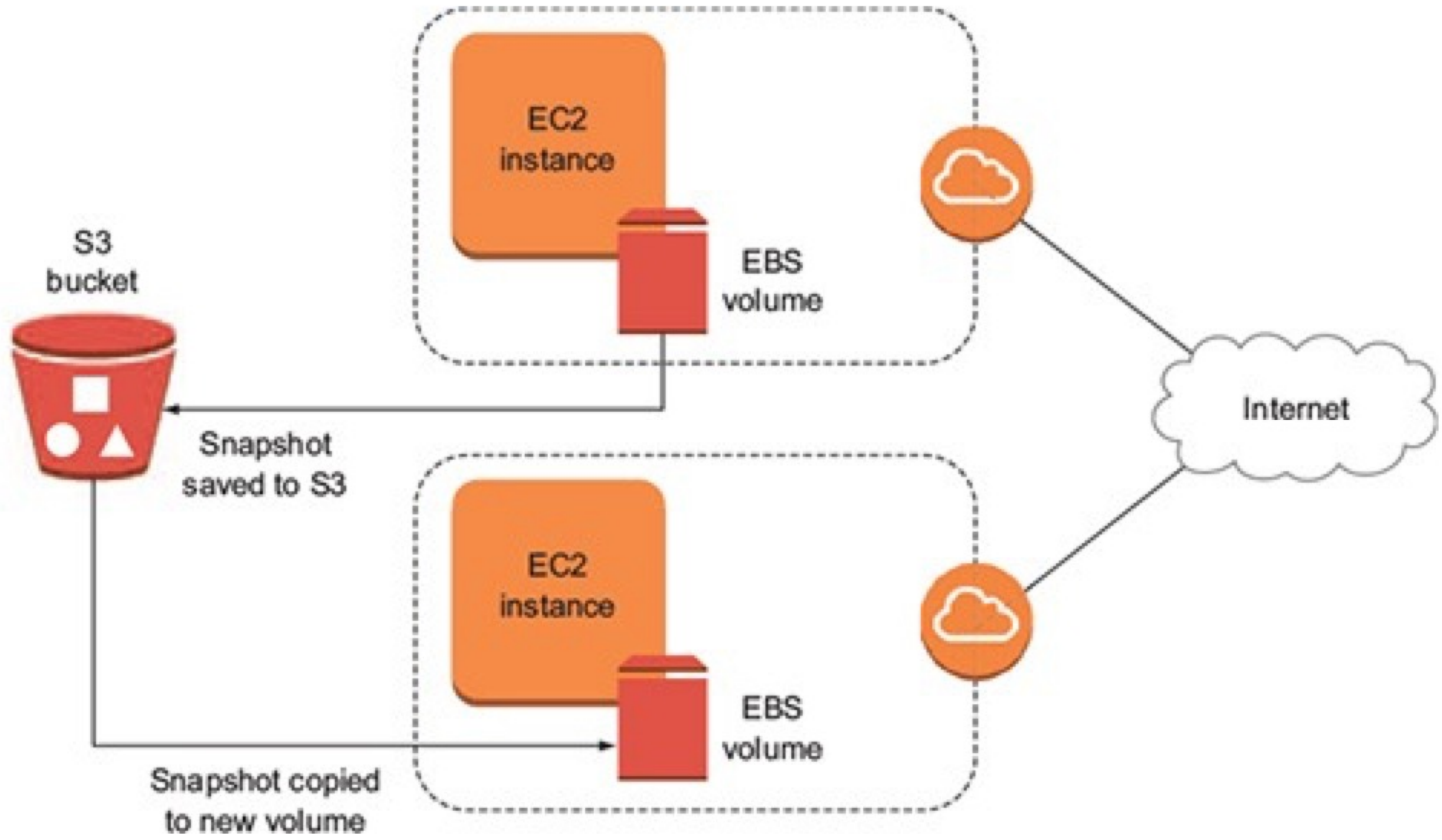
Creating Topics and Subscriptions

- Create a Topic
  - Select name
  - Select access policy (basic/advanced)
    - Default access is owner only
    - You can enable specific accounts, services or everyone
  - Several advanced options available
- Create Subscription
  - Select protocol
  - Select endpoint
  - Subscription is pending until confirmed by endpoint (subscriber)
- Publish a message to the topic and check subscriber
- **Allowing an S3-bucket to publish to a topic:**

  https://docs.aws.amazon.com/sns/latest/dg/sns-access-policy-use-cases.html#sns-allow-s3-bucket-to-publish-to-topic

S3 for Backups: Snapshots of EC2 instances or Volumes

AWS stores snapshots in AWS administrated S3, not with your own S3 buckets.

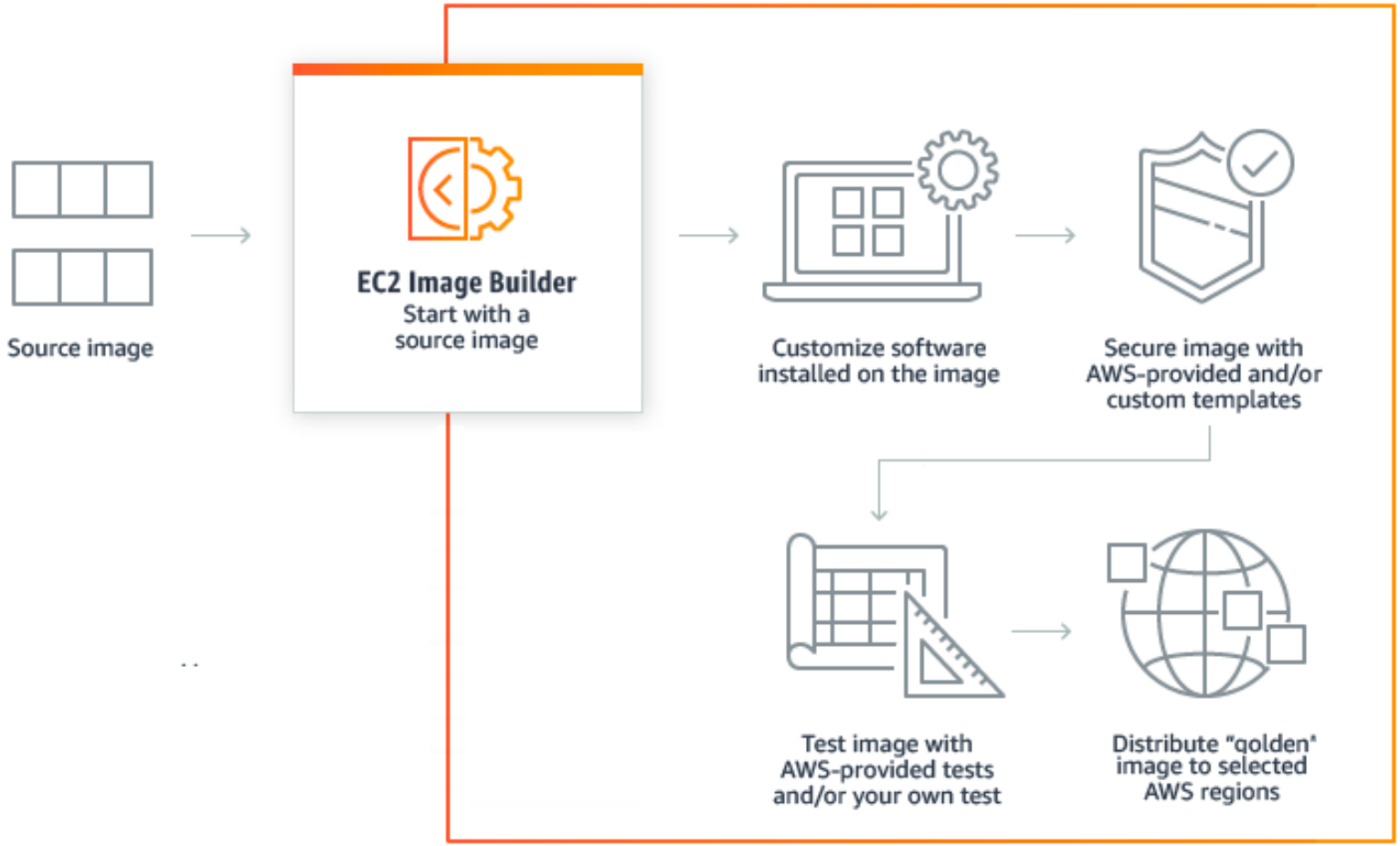


Source: David Clinton, Learn AWS…

Creating and Testing Backups

- Click '**Volumes**' on EC2 Dashboard sidebar
  - –Select one volume, explore the details
  - –Under 'Actions' choose 'Create snapshot'
- Select '**Snapshots**' from sidebar
  - –Create image (**AMI – Amazon Machine Image**)
- Launch the image to test if it works
  - –Will have different IP address (unless you re-assign elastic IP address of original instance)
- Note:
  - –You can create lifecycle policy for snapshots (frequency, retention etc.)
  - –You can build 'launch templates' from AMIs

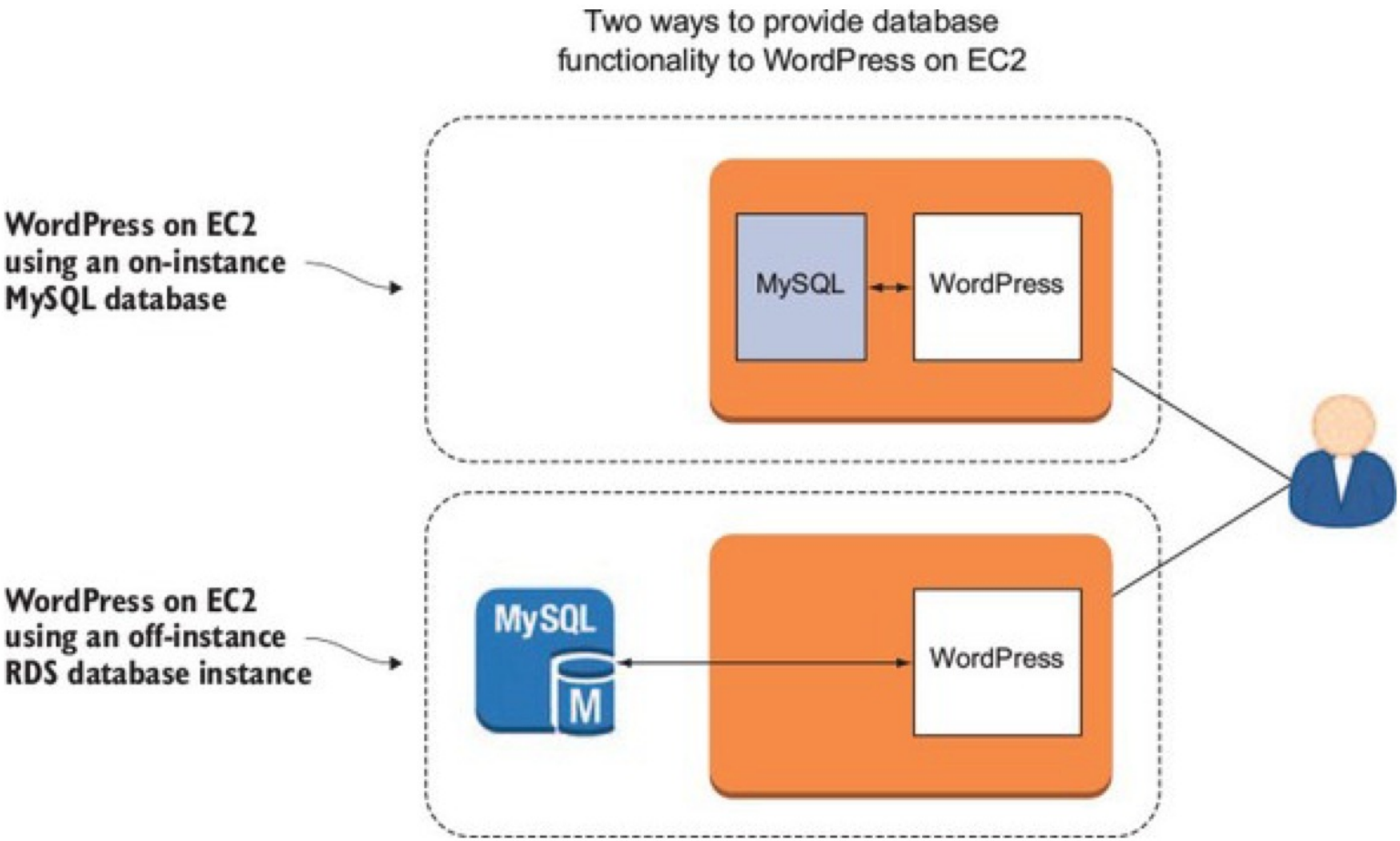
Secure Images – EC2 Image Builder



Database Services

- Relational Databases – AWS RDS
  - SQL
  - Structured data, transactional (ACID)
- NoSQL – AWS DynamoDB
  - Unstructured, not transactional (mostly)
  - http://aws.amazon.com/nosql

Database Options

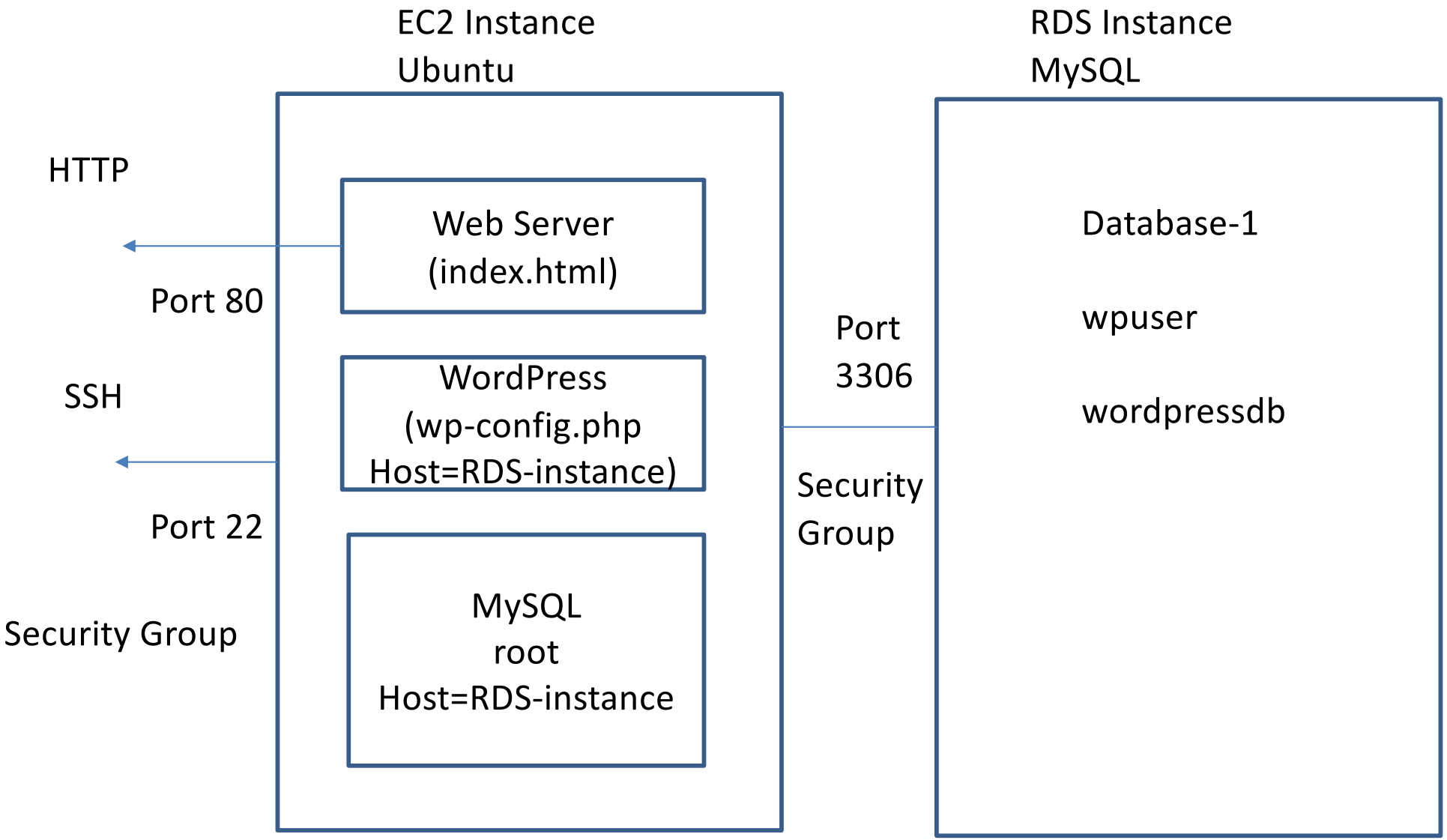


Source: David Clinton, Learn AWS…

Reasons for DB in a Separate Instance

- Improved Security
  - Limit outside access to data
- Improved Performance & Scalability
- Managed Services to reduce OPEX
  - As compared to just another AMI with a database
- Caution: Cost!
  - (http://calculator.s3.amazonaws.com/index.html)
  - Or: https://calculator.aws/#/
  - Burst capable class type (db.t2) are the cheapest, but with lowest performance
  - The smallest instance used in the exercises costs about 15$/month if running continuously

WordPress Configuration with RDS



Note: Use an elastic IP address if you want the configuration to survive a shutdown

Amazon Relational Database Services (RDS)

- https://console.aws.amazon.com/rds/home?region=us-east-1#
- Options
  - Aurora (MySQL or PostgreSQL compatible)
  - MariaDB
  - MySQL
  - PostgreSQL
  - SQL Server
  - Oracle
- Choose 'sandbox' (otherwise you potentially get authorization problems)
- Size can scale elastically
- **Master user (wpuser) and password need to match wp-config.php**
- **Port 3306** will be enabled by default (to your system), but add the security group of your EC2 instances as 'custom' source (and choose your Wordpress EC2 instance to connect to)
- Other services access an RDS database through its **endpoint** (cp 12 Factor)
- AWS Learning Path Project with WordPress
- **Note: RDS is restarted automatically after 7 days, so be wary of the costs**

Using an RDS Service with WordPress

- **Make a usable** copy of your existing database, (a database dump).

  mysqldump -u root -p wordpressdb > /home/ubuntu/mybackup.sql

- Use the AWS Console to **create an RDS instance**.
  - Make wpuser the main user, use the password you set for wpuser
- Make sure a **secure connection** between this instance and your EC2 instance is possible
  - Enable access to port 3306 (this is the default)
- You can **access the RDS instance from your EC2 instance** by using a 'host' option with your database **endpoint** (on details page for your database), e.g.

  sudo mysql -u wpuser -p --host=*your-endpoint.rds.amazonaws.com*

  Create database wordpressdb;

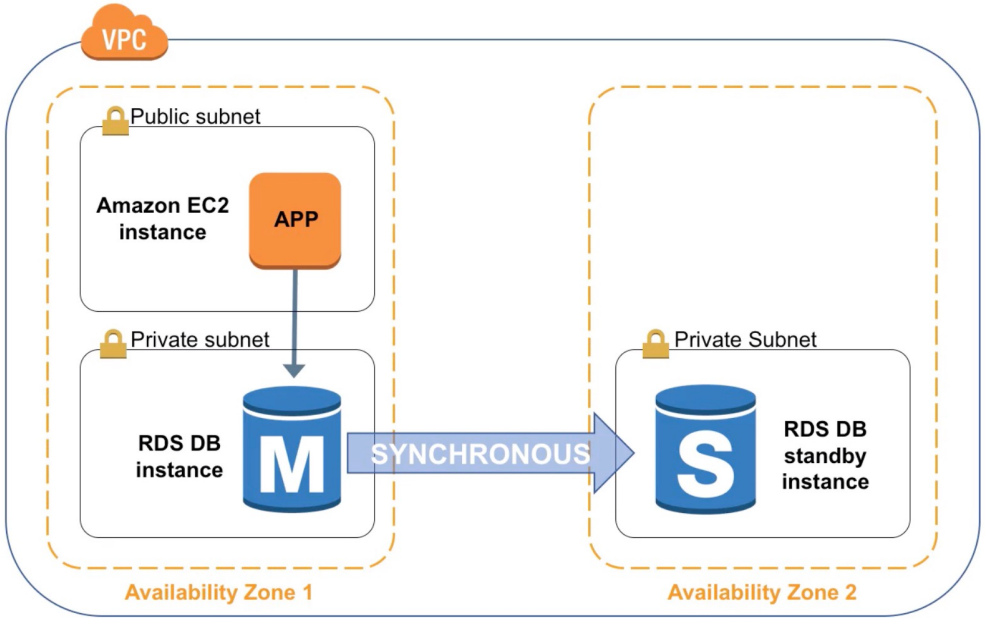
- **Upload your saved database dump** to the RDS database server, e.g.

  sudo mysql -u wpuser -p --database=wordpressdb --host=*your-endpoint.rds.amazonaws.com* < /home/ubuntu/mybackup.sql

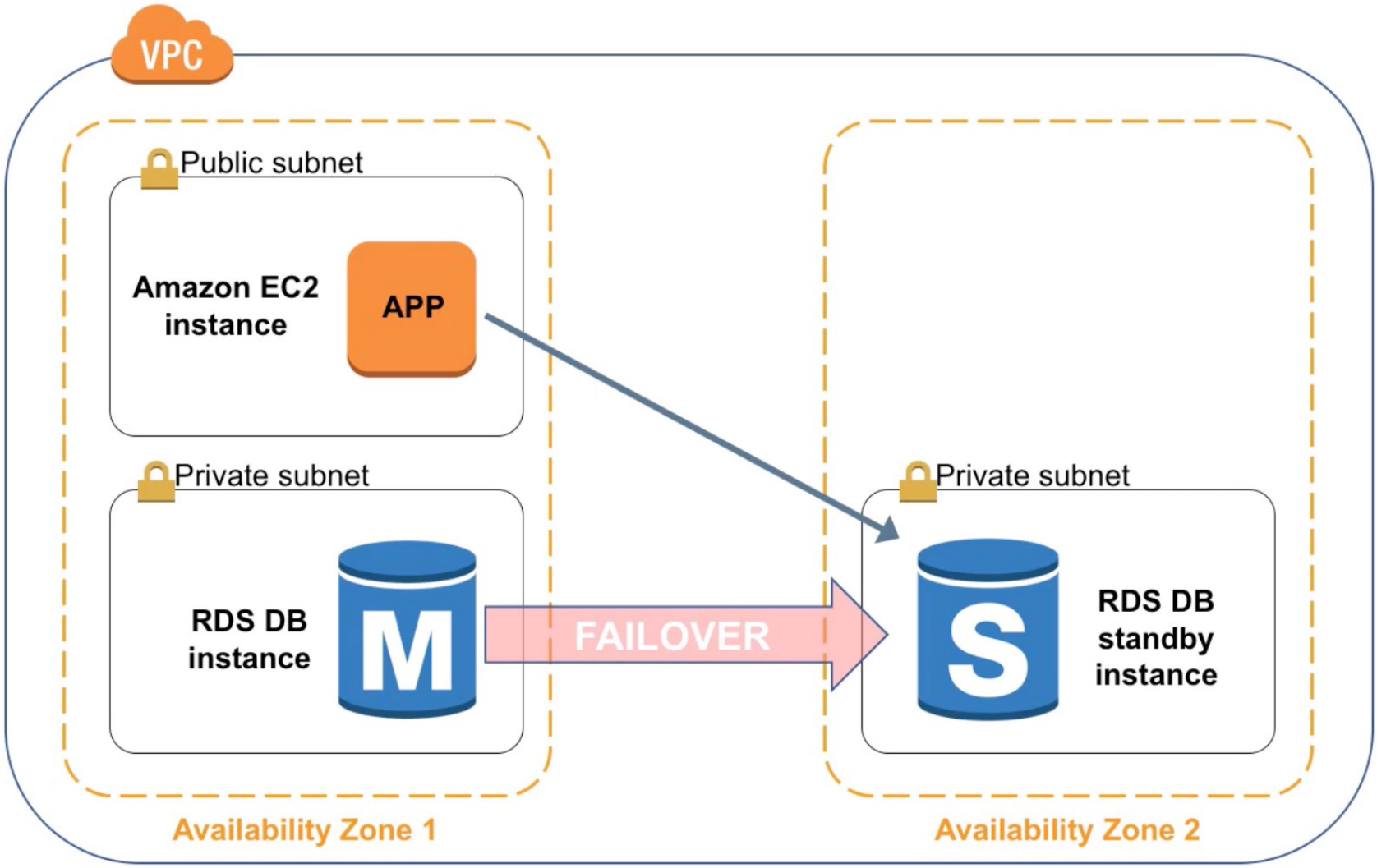
- **Update wp-config.php** with the DB_HOST value equal to the RDS endpoint
- Log in to Wordpress and verify access. If you stop the RDS instance and clear the browser cache, you should not be able to connect.
- Note: In real life, consider the AWS Database Migration Service

Standby Replicas

- When creating a database, you can select deployment to multiple availability zones
- This creates a **standby** replica that is updated synchronously
  - Standby replicas can also be created for an existing database, but this incurs an initial performance hit.
- In case of failure of the database, RDS automatically switches to the standby copy



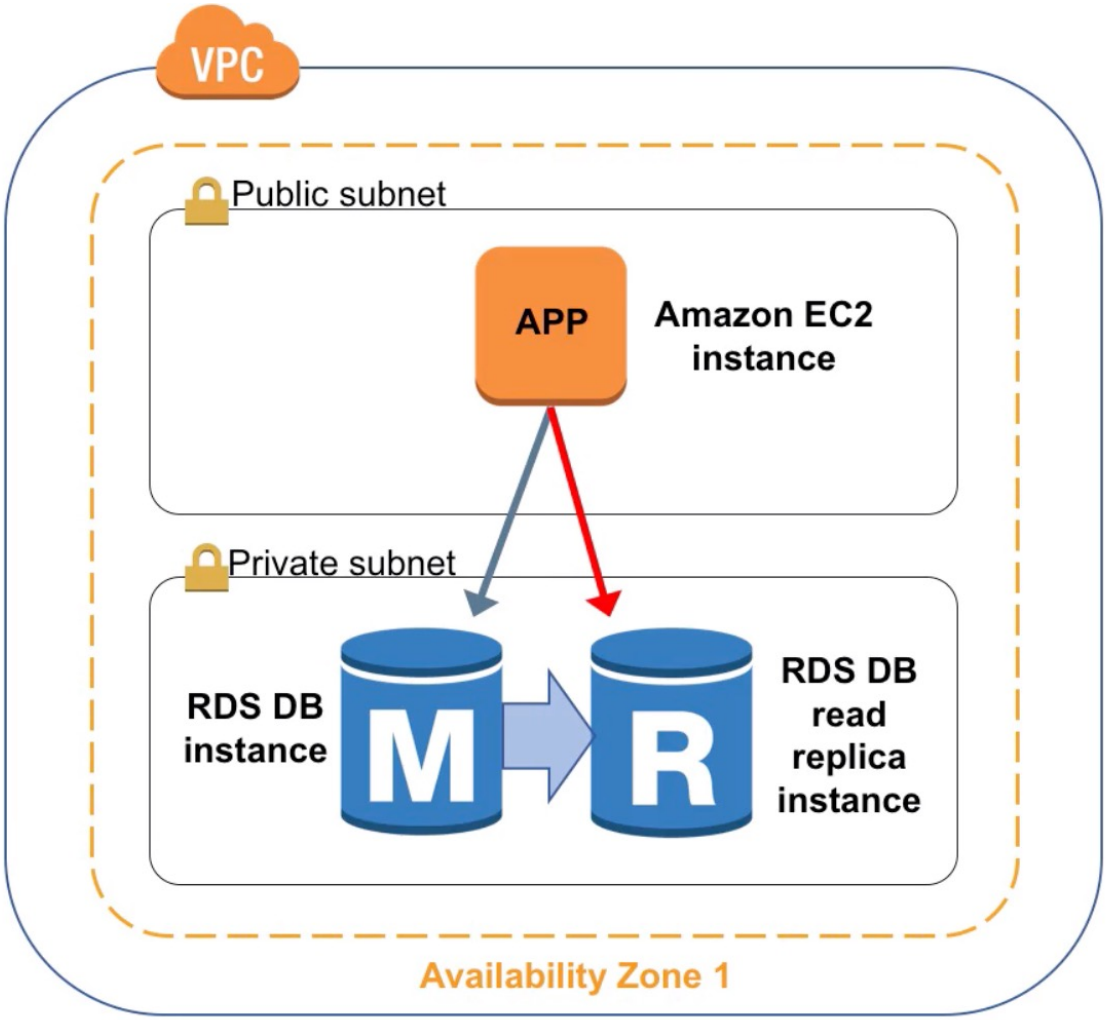
Failover to a standby instance



Read Replicas

For read-heavy databases, you can also create **read replicas.**

- Replication to read replicas is done asynchronously
- Read replicas can be promoted to master, for instance in case the source instance fails



Homework

EC2: Bring up a Wordpress instance (pre-installation) from an AMI and check it out

S3: Create an S3 bucket "mybucket-*yourinitials"*, enable public access

- Upload an Image, make the object public
- Access the Image URL
- Enable web hosting. Create an index file displaying the text "This is *YourInitial's* S3-based web site!", make it public and test that your website works.

- Access an image from your S3 bucket for your WordPress installation or your static website

AMI: Create an AMI from your instance, launch another instance from it (if you haven't done so already)

RDS: Create a database instance and direct WordPress to it.

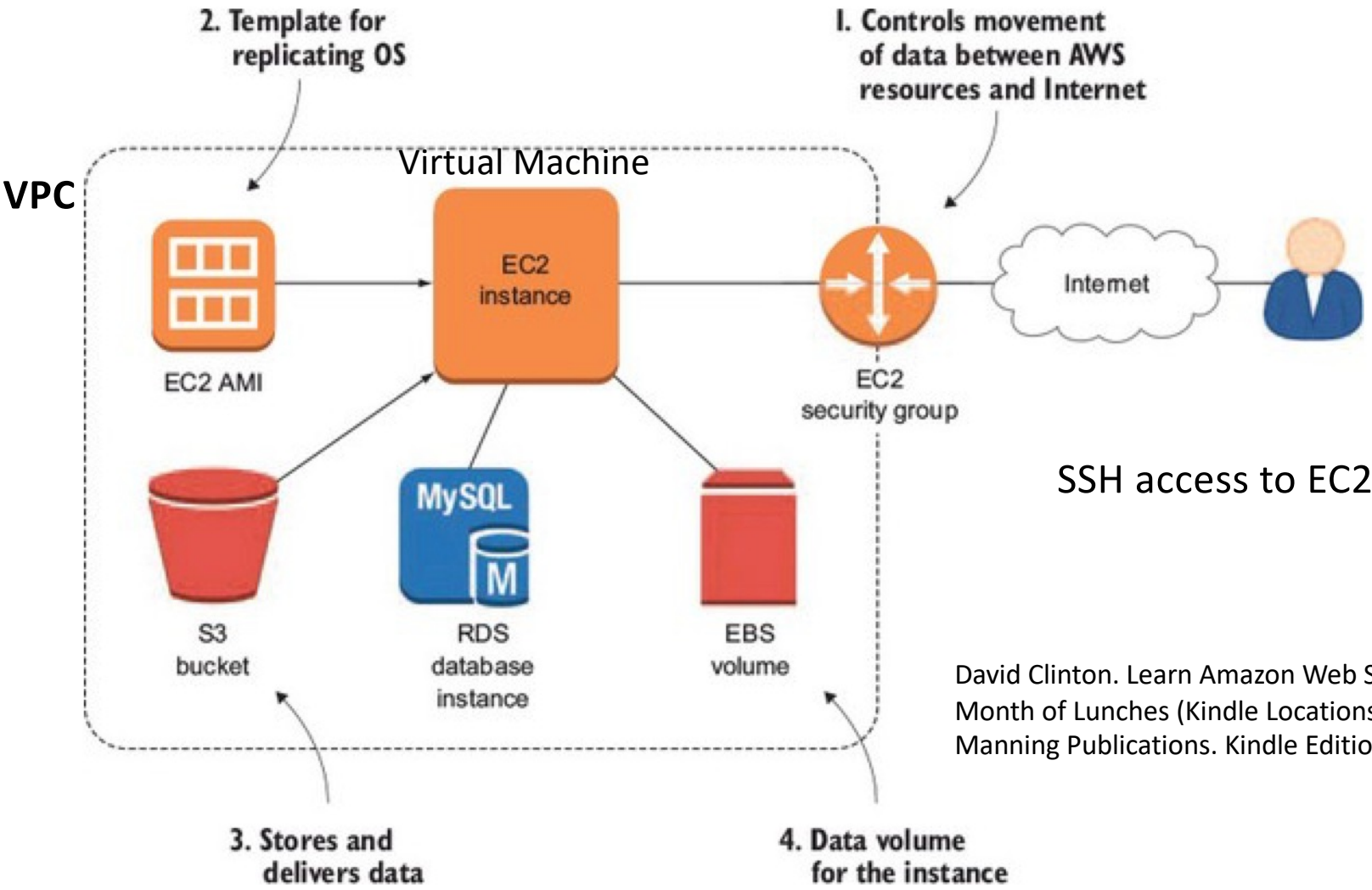
Module 2
DynamoDB, CLI, AWS CloudShell, SDK, Security, Monitoring



Some blogs

- AWS Developer Center https://aws.amazon.com/developer/
- AWS Blogs https://aws.com/blogs
- Azure Blogs https://azure.microsoft.com/en-us/blog/
- Google Blogs https://cloud.google.com/blog/
- IBM Blogs https://developer.ibm.com/depmodels/cloud/blogs/
- Cloud Native Computing Foundation https://www.cncf.io/blog/
- allthingsdistributed.com (Werner Vogels)

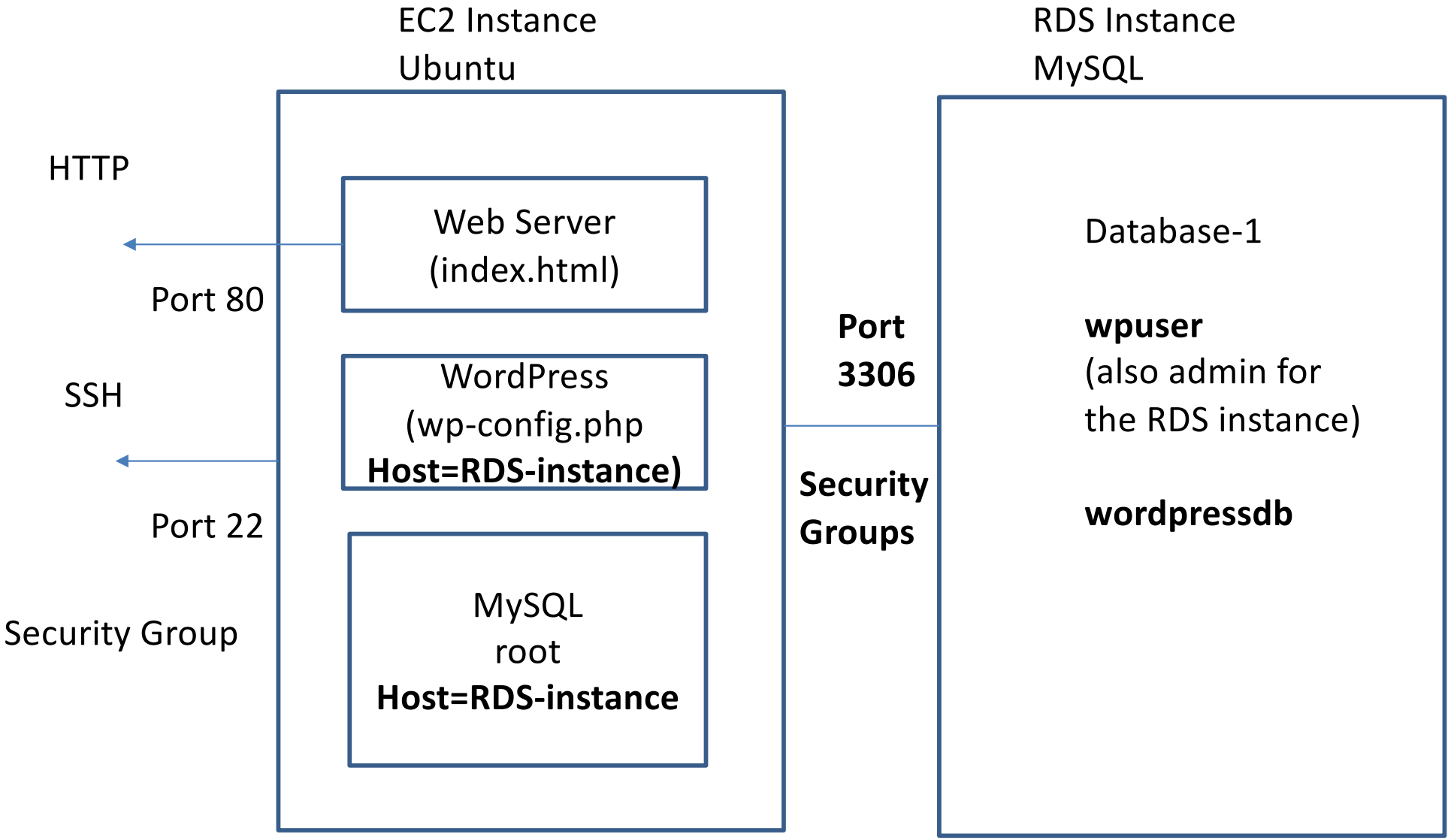
Recap: Where do we stand?



SSH access to EC2

David Clinton. Learn Amazon Web Services in a Month of Lunches (Kindle Locations 502-506). Manning Publications. Kindle Edition.

WordPress Configuration with RDS



Note: Use an elastic IP address if you want the configuration to survive a shutdown

Using Aurora with an EC2 Instance

- Aurora is Amazon's cloud-optimized relational database system
  - MSQL and PostGreSQL compatible versions
  - Serverless or provisioned capacity (only provisioned works with Learner Lab)
- Automatically creates a database cluster with a write instance and optionally multiple read instances
  - Only Dev/Test configuration works with Learner Lab
- Can be attached to an EC2 instance during or after creation
  - Make sure the necessary security groups have been attached
- Use the writer endpoint for the cluster for mysql remote access and Wordpress configuration

  sudo mysql -u wpuser -p --host=*your-cluster-writer-endpoint.rds.amazonaws.com*
- Don't forget to stop your Aurora cluster

Database Services

- Relational Databases – AWS RDS
  - SQL
  - Structured data, transactional (ACID)
- **NoSQL – AWS DynamoDB**
  - **Unstructured, not transactional (mostly)**
  - **http://aws.amazon.com/nosql**

Amazon DynamoDB: A Cloud Native Database Service

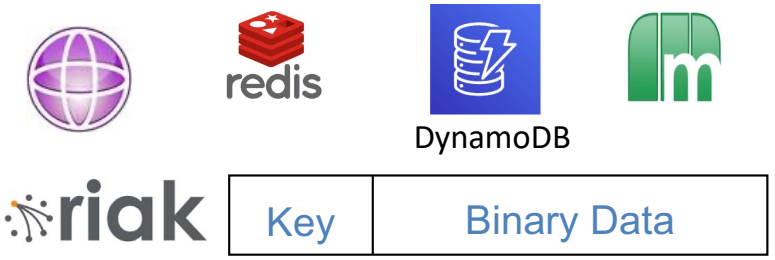
- A NoSQL database service supporting key-value and document data models
  - Highly flexible, no need to declare full schema at creation
- Fully managed, built for performance
  - Handles 20 million requests/second
  - Highly available, replicated storage, eventually consistent
  - Now also supports ACID as an option
- Underpins Amazon Shopping Cart and other core services
- For an interesting discussion of design considerations, see

  DeCandia et al., Dynamo, Amazon's highly available key-value store, SOSP 2007

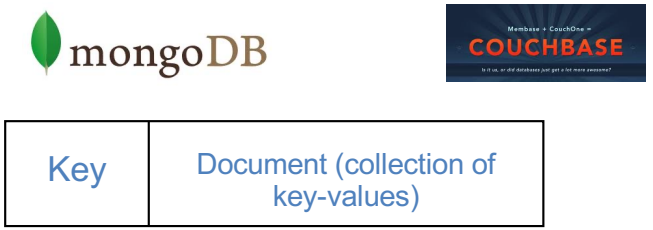
  https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
- Major applications in retail, finance, gaming, IoT etc.
- Can be used with the NoSQL Workbench

The NoSQL Landscape

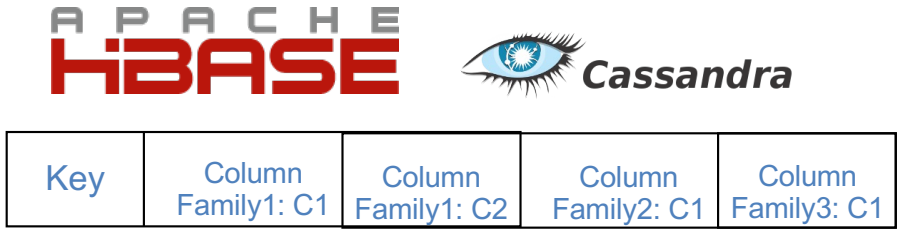
- Simple Key Value Stores
  - Data is stored in a hash-table of keys. Values are opaque binary objects.



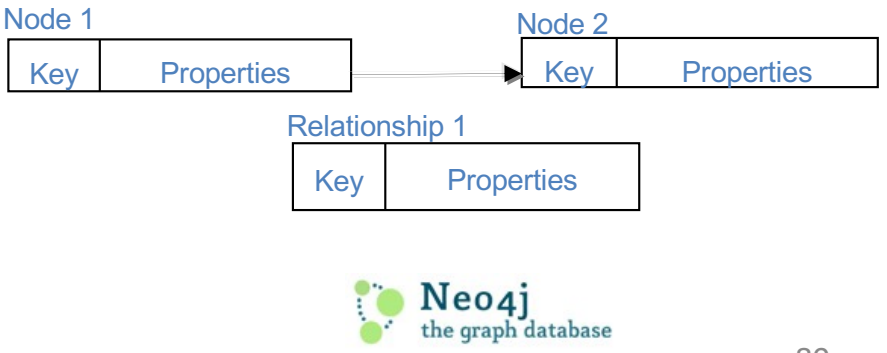
- Document Stores
  - Data is stored as documents with tagged elements



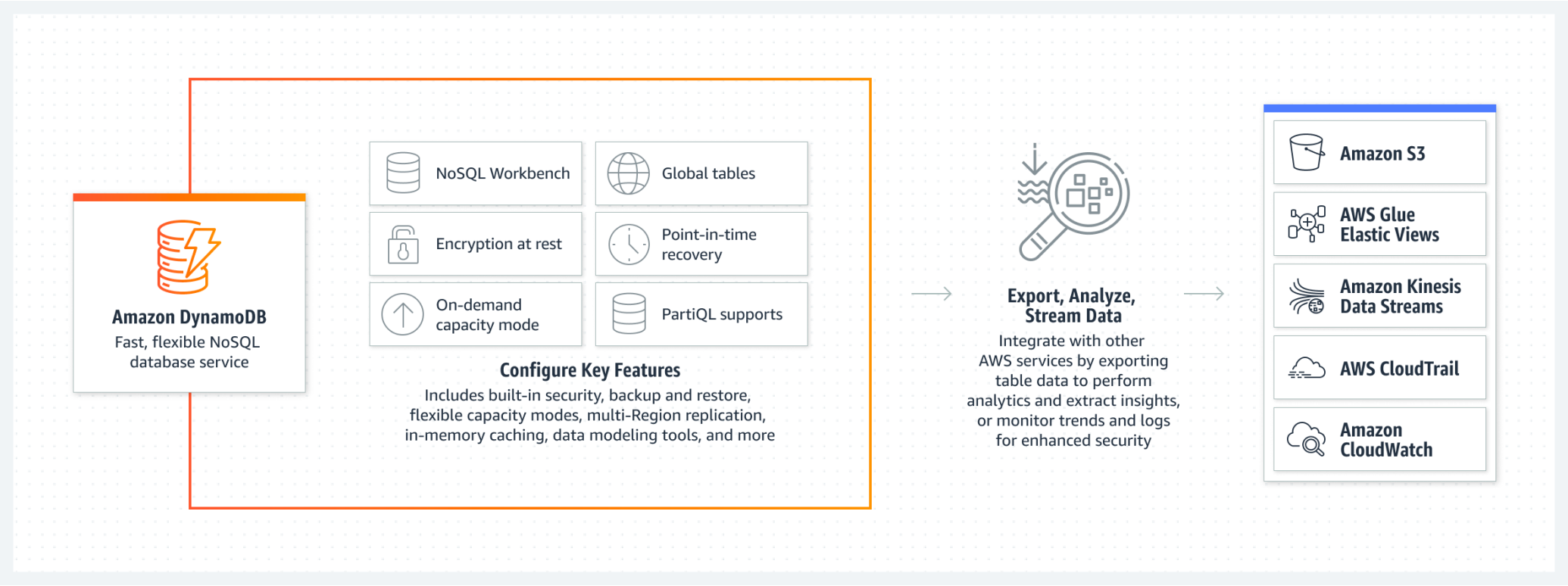
- Column Family
  - Data attributes are grouped into column sets. Each storage block contains data from only one column set.



- Graph Store
  - Data is stored in nodes and edges of a graph and accessed using graph traversal

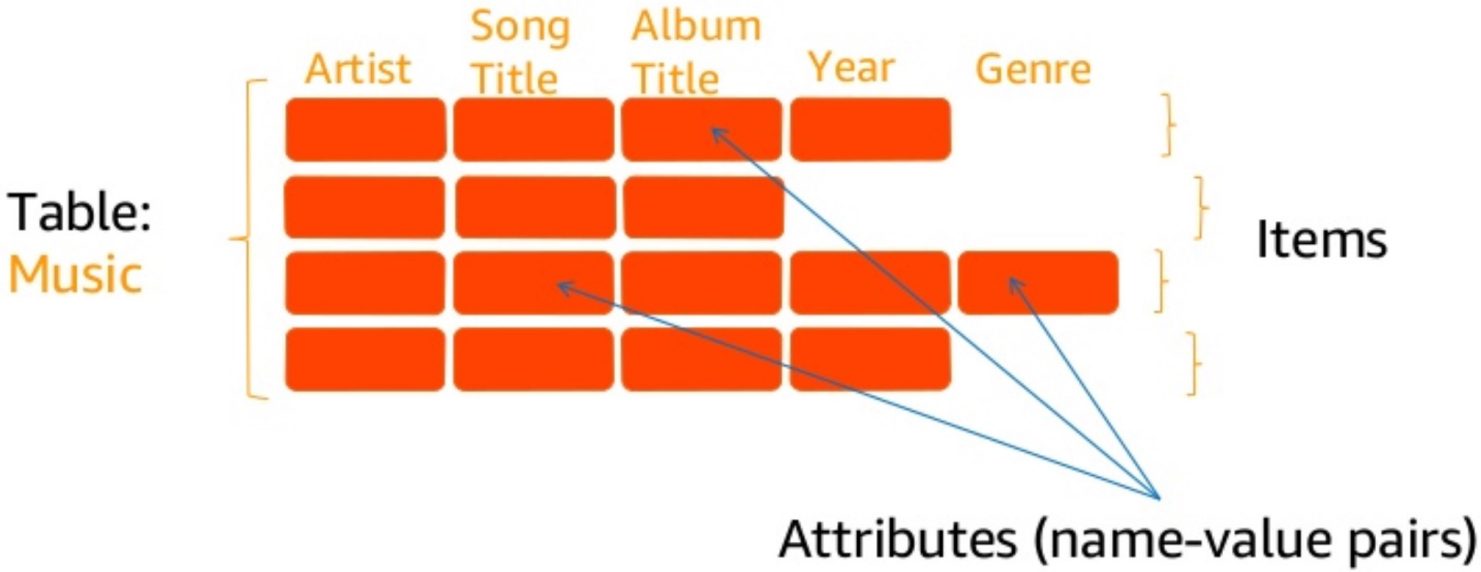


DynamoDB



Source: AWS

DynamoDB Data Model



Attributes can be added after table creation

© 2019, Amazon Web Services, Inc. or its affiliates. All rights reserved.

aws

Source: Awsome Day 2019 Detroit Slideshare

Primary Keys

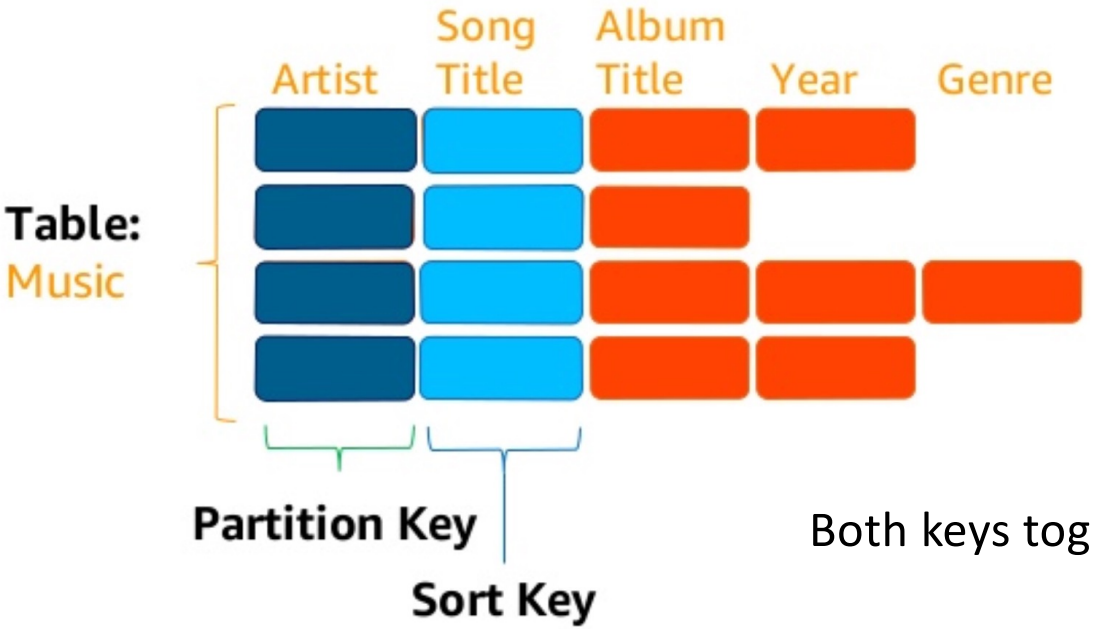


Table: Music
Partition Key: Artist
Sort Key: Song Title

Both keys together form the primary key

(DynamoDB maintains a sorted index for both keys)

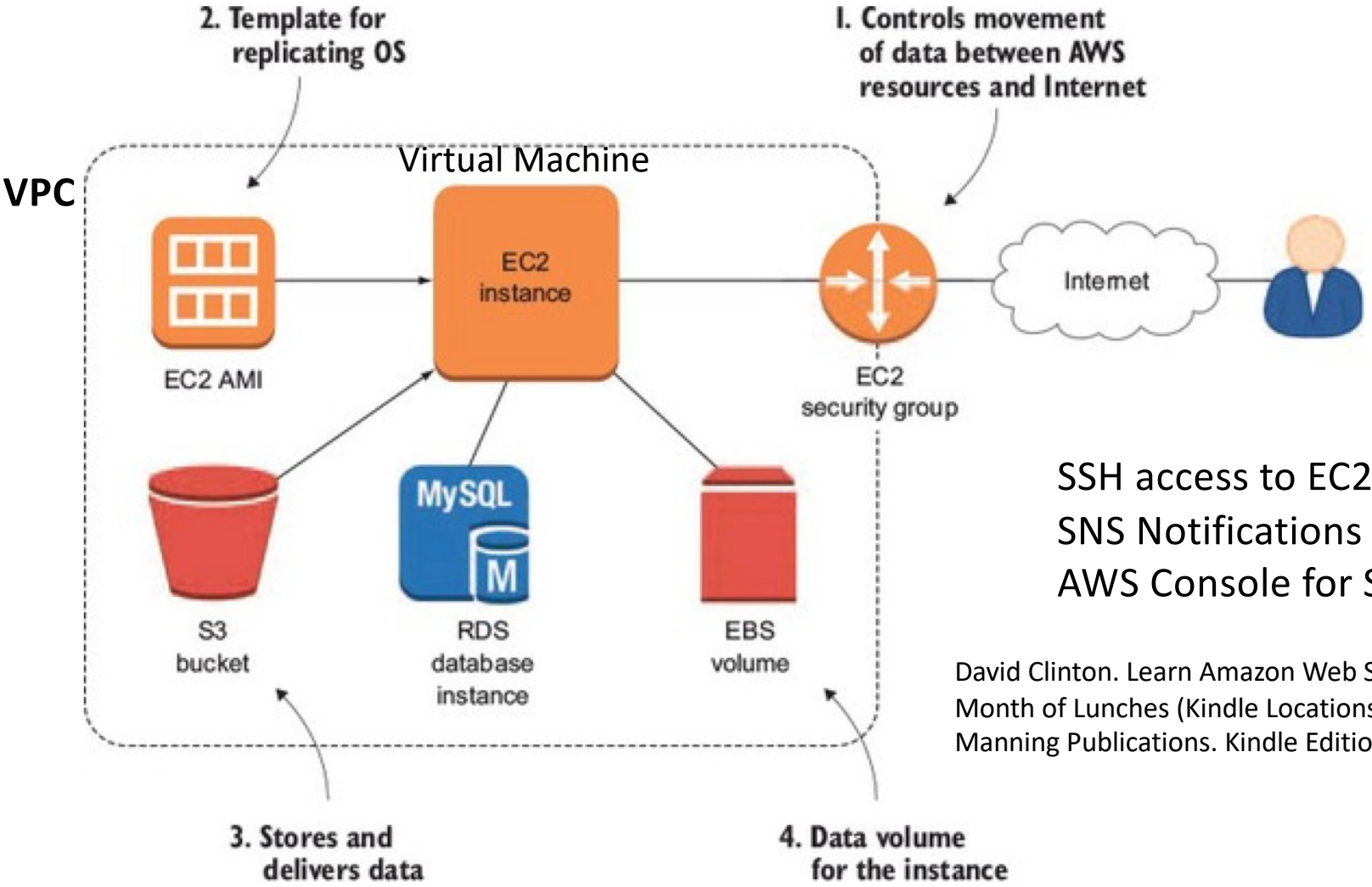
© 2019, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Source: Awsome Day 2019 Detroit Slideshare

Getting started with DynamoDB

- https://console.aws.amazon.com/dynamodb/home?region=us-east-1#gettingStarted:
- Click blue ‘Create table’ button
- Choose table name, e.g. Books
- The **partition key** is used to spread items across partitions, use one with a wide distribution, e.g. Author
  - Optionally choose a **sort key**, e.g. Title
- You have to change the default options to enable auto-scaling
- ‘Create table’ brings up a dashboard for your table
- Create some items (items tab)
  - Query searches for a set of items, scan traverses the entire table
  - You can add additional attributes after/during creation of an item
- Delete table!
- See also Tutorials: https://aws.amazon.com/dynamodb/getting-started/

We built it, now we need to manage it!



SSH access to EC2 (Connect)
SNS Notifications
AWS Console for Services

David Clinton. Learn Amazon Web Services in a Month of Lunches (Kindle Locations 502-506). Manning Publications. Kindle Edition.

Managing and Accessing Services

- Command Line Interface (CLI)
- AWS SDK (boto3 for Python)
- CloudWatch (Monitoring)

All three capabilities are important for automation!

AWS CloudShell

https://aws.amazon.com/cloudshell

- A browser-based Linux shell
- Fully managed Amazon Linux 2 environment
- Configurable with 1 GB storage per region
- Many tools are pre-installed & pre-configured, like the AWS Command Line Interface
- Easy file upload and download
- Suggested as an alternative to the deprecated Cloud9 environment

AWS Command Line Interface (CLI)

- https://aws.amazon.com/cli/
- Requires Python
- If a command line text starts with aws, it will be parsed by the CLI shell, e.g.

  $ aws resource command --option, like

  $ aws ec2 start-instances --instance-ids i-0a5d814e9e853f721
- Can be embedded in automation scripts
- **Pre-installed and configured in CloudShell**
- To install and configure on another system, see
  - https://docs.aws.amazon.com/cli/latest/userguide/cli-chap-install.html
- On your Ubuntu Linux instance

```
$curl "https://s3.amazonaws.com/aws-cli/awscli-bundle.zip" -o “awscli-bundle.zip”
unzip awscli-bundle.zip
sudo ./awscli-bundle/install -i /usr/local/aws -b /usr/local/bin/aws
$ aws configure
```

- Configuration requires **access keys and region**
  - With a standard account, you can get keys form the ‘My Security Credentials’ pull-down under your name on the console page
  - With AWS Academy, look at AWS Details in the Learner Lab Starting Page

A few useful CLI commands

- Find help:
  - $ aws help (exit with ‘q’)
  - $ aws <resource> <command> help (specific operations)
- S3 commands
  - aws s3 ls s3://mybucket (list objects in mybucket)
  - aws s3 rm s3://mybucket/myobject (removes object)
  - aws s3 cp myfolder s3://mybucket/myfolder –recursive (folder level upload)
  - aws s3 sync myfolder s3://mybucket/myfolder --exclude *.tmp (sync local folder and S3)
- EC2 commands
  - $ aws ec2 start-instances --instance-ids i-0a5d814e9e853f721
  - $ aws ec2 run-instances –image-id *ami-yyy* –count 1 \

  --instance-type t2.micro –key-name kk-educate –security-group-ids *sg-xxx*
- Dynamodb
  - $ aws dynamodb scan –table-name Music
  - $ aws dynamodb list-tables

Managing and Accessing Services

- Command Line Interface (CLI)
- **AWS SDK (boto3 for Python)**
- CloudWatch (Monitoring)

All three capabilities are important for automation!

boto3: Using Python with AWS

- https://boto3.amazonaws.com/v1/documentation/api/latest/guide/quickstart.html
- To use AWS commands in Python, install boto3

  $sudo pip install boto3 (or pip3…. For Python3)

- **To use boto3, you need authentication credentials. You can get these by configuring CLI.**
- Features:
  - Low-level APIs mapping HTTP calls to APIs
  - Resource APIs with resource objects
  - Dynamically generated classes driven by JSON models describing AWS APIs
  - 'Waiters' polling for predefined status changes
- Code for Python

  https://github.com/awsdocs/aws-doc-sdk-examples/tree/main/python

boto3 Examples

- EC2

```
import boto3

ec2 = boto3.client('ec2')
response = ec2.describe_instances()
print(response)
```

- S3

```
import boto3
# Retrieve the list of existing buckets
s3 = boto3.client('s3')
response = s3.list_buckets()
# Output the bucket names
print('Existing buckets:')
for bucket in response['Buckets']:
        print(bucket["Name"])
```

- DynamoDB Examples (Movie Database)

  https://docs.aws.amazon.com/code-library/latest/ug/python_3_dynamodb_code_examples.html

Module 2
SDK Recap, Monitoring, Security



Managing and Accessing Services

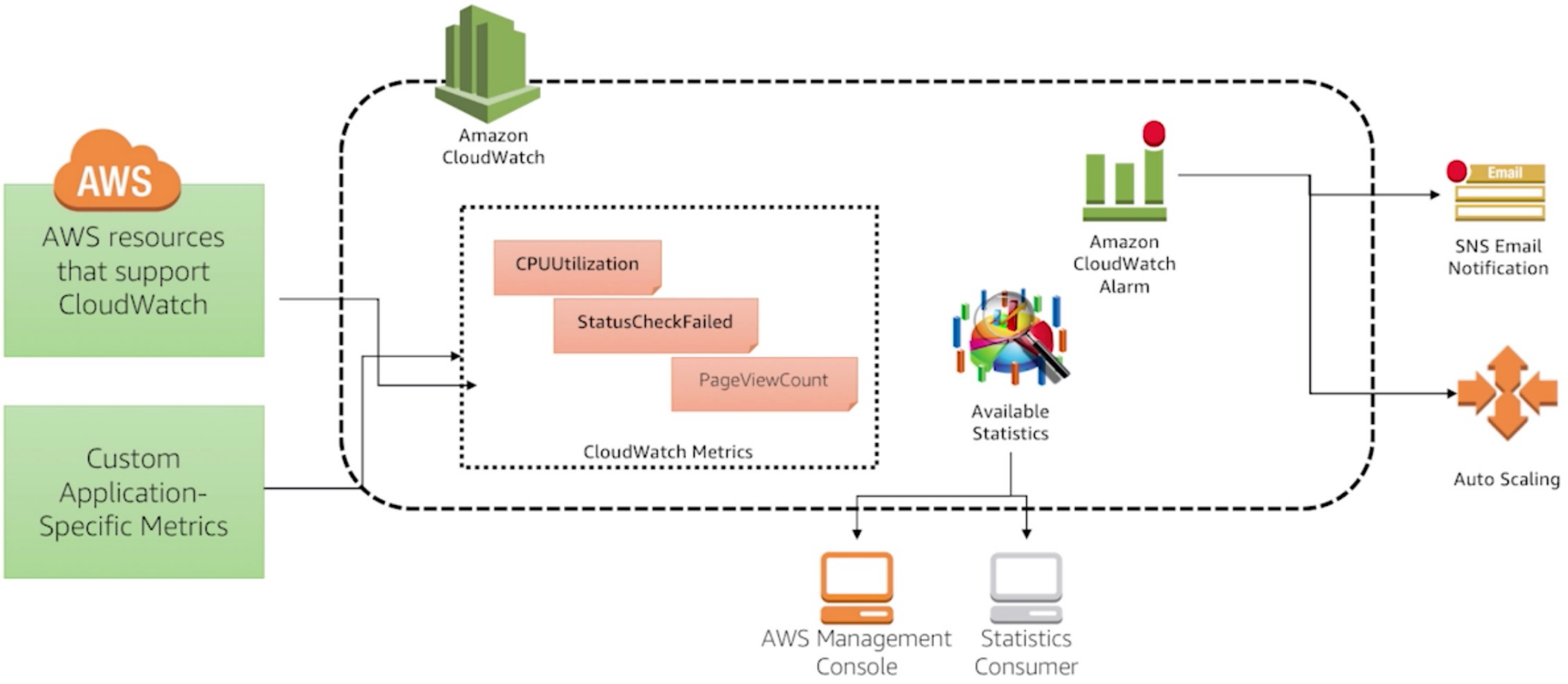
- Command Line Interface (CLI)
- AWS SDK (boto3 for Python)
- **CloudWatch (Monitoring)**

All three capabilities are important for automation!

Amazon CloudWatch

- Monitoring for AWS
  - Metrics like CPU Utilization
  - Problem Situations, Threshold Breaches
- Alarms
  - Notifications through Amazon SNS (more later)
  - Actions for EC2 or Auto Scaling (more later)
- Events
  - Rules and Routing
  - Anomaly Detection
- Logs

CloudWatch Structure



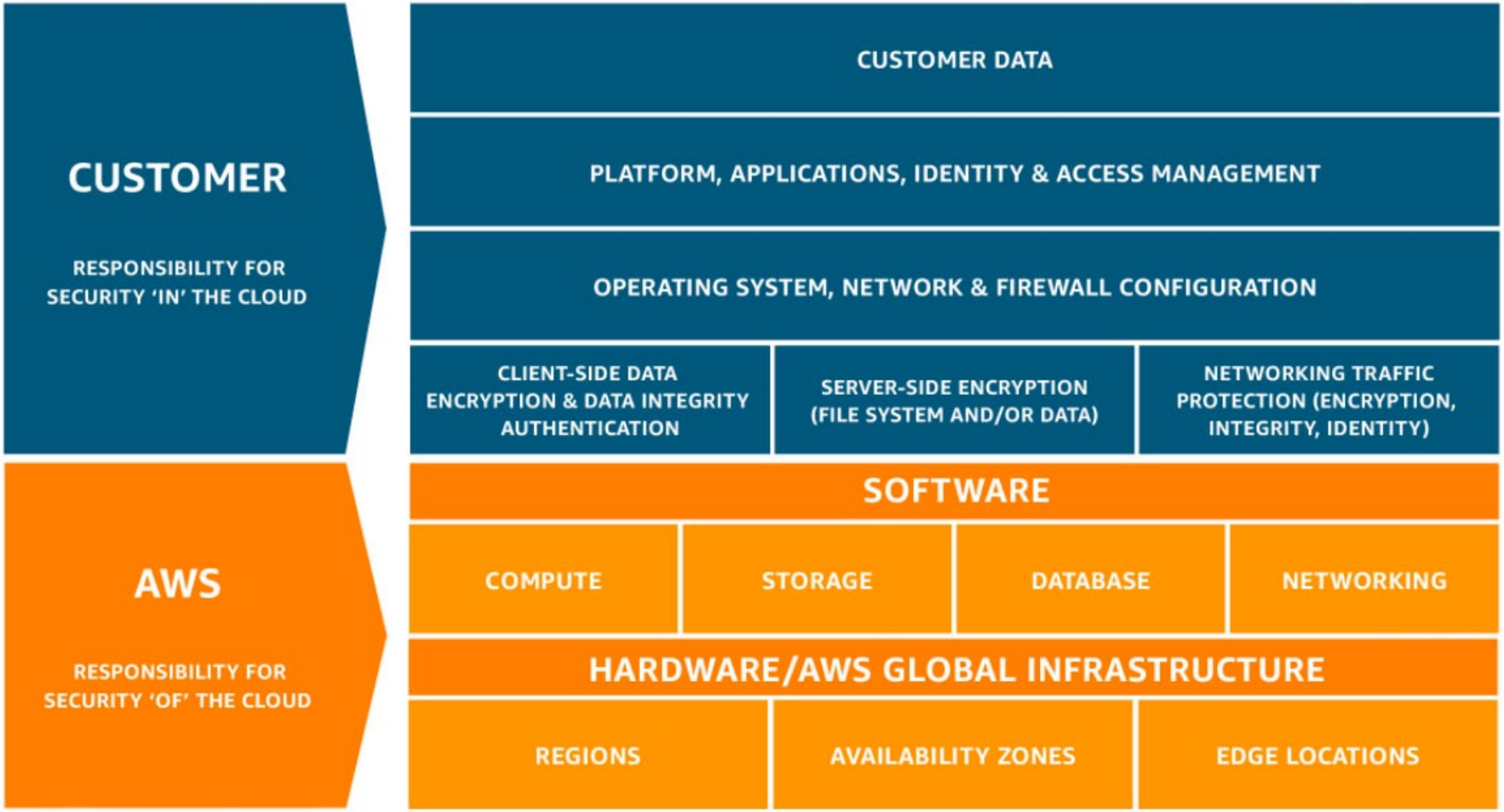
https://console.aws.amazon.com/cloudwatch/home?region=us-east-1#cw:dashboard=Home

Creating an Alarm

- From the CloudWatch console sidebar, choose ‘Alarms’
- Click blue ‘Create Alarm’ button
- Select Metric
  - Many metrics are offered for standard resource types like EC2 instances or databases
  - Suggestion: Search for id of one of your instances
- Specify details
  - Name and description
  - Threshold
  - Notification or other action

Security and Compliance as Shared Responsibility between AWS and Customer

https://aws.amazon.com/compliance/shared-responsibility-model/



Customer compliance

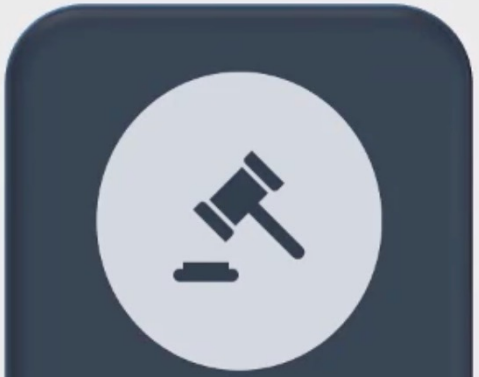
aws training and certification



Review trusted information and document compliance requirements.



Design and implement control objectives that meet compliance requirements.



Identify and document controls owned by outside parties.



Verify all control objectives are met and all key controls are designed and operating effectively.

© 2018, Amazon Web Services, Inc. or its Affiliates. All rights reserved.

https://aws.amazon.com/compliance/

Well Architected Framework - Design Principles for Security

Risk Assessment and Mitigation

- Implement a strong identity foundation (least privilege, separation of duties, centralized privilege management, reduce long-term credentials)
- Enable traceability
- Apply security to all layers (**defense in depth**)
- Automate security best practices
- Protect data in transit and at rest (encryption)
- Keep people away from data (eliminate manual processing)
- Prepare for security events

Best Practices:

Identity and Access Management, Detective Controls, Infrastructure Protection (Separation/Segmentation), Data Protection, Incident Response

Source: AWS 'Well-Architected Framework'

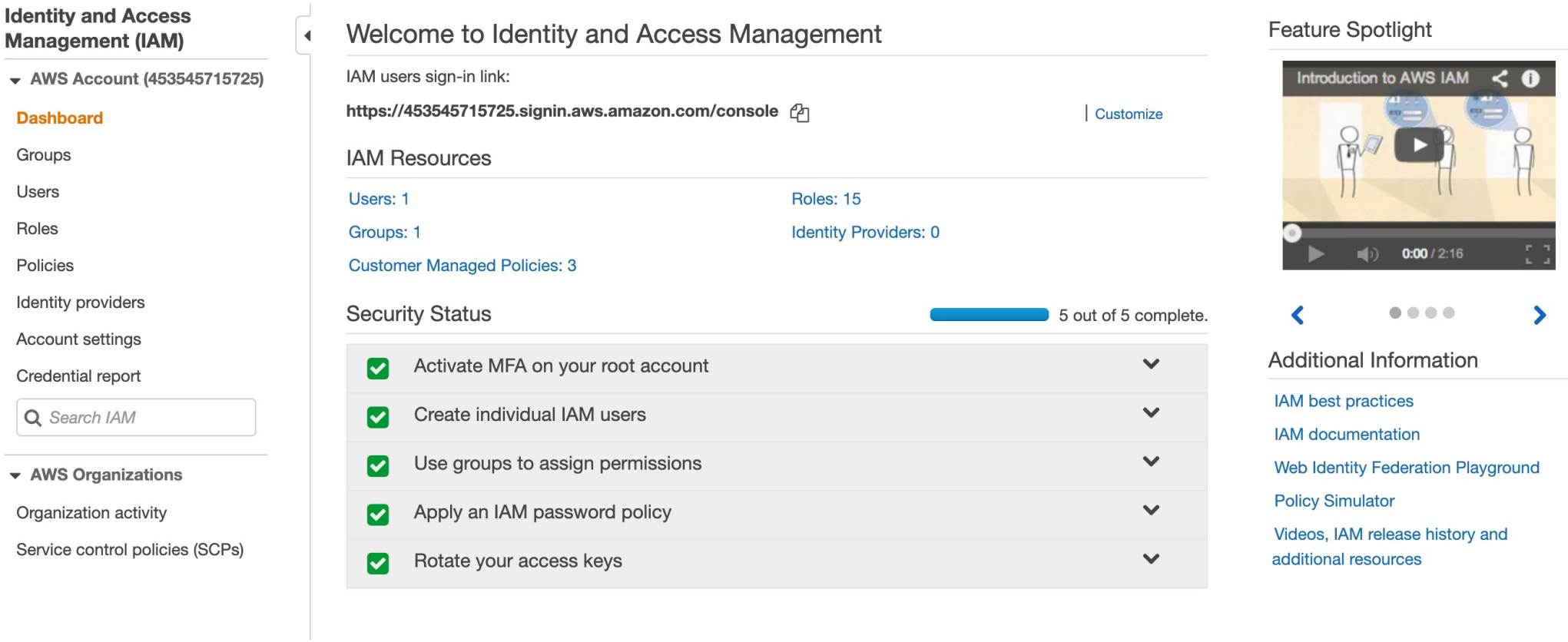
AWS Security Capabilities

- Network Security
  - Built-in Firewalls
  - Encryption in transit
  - Private/dedicated connections
  - Distributed Denial of Service (DDoS) mitigation (Amazon Shield)
- Inventory and Configuration Management
  - Control and track changes
- Data Encryption
  - Encryption capabilities
  - Key management services
  - Hardware-based cryptographic key storage
- Access and Control Management
  - **Identity and Access Management**
  - Multi-factor authentication
  - Directory integration and Federation
  - **Amazon Cognito User Management** and Secure Sign-On (SSO)
- **Monitoring and Logging**

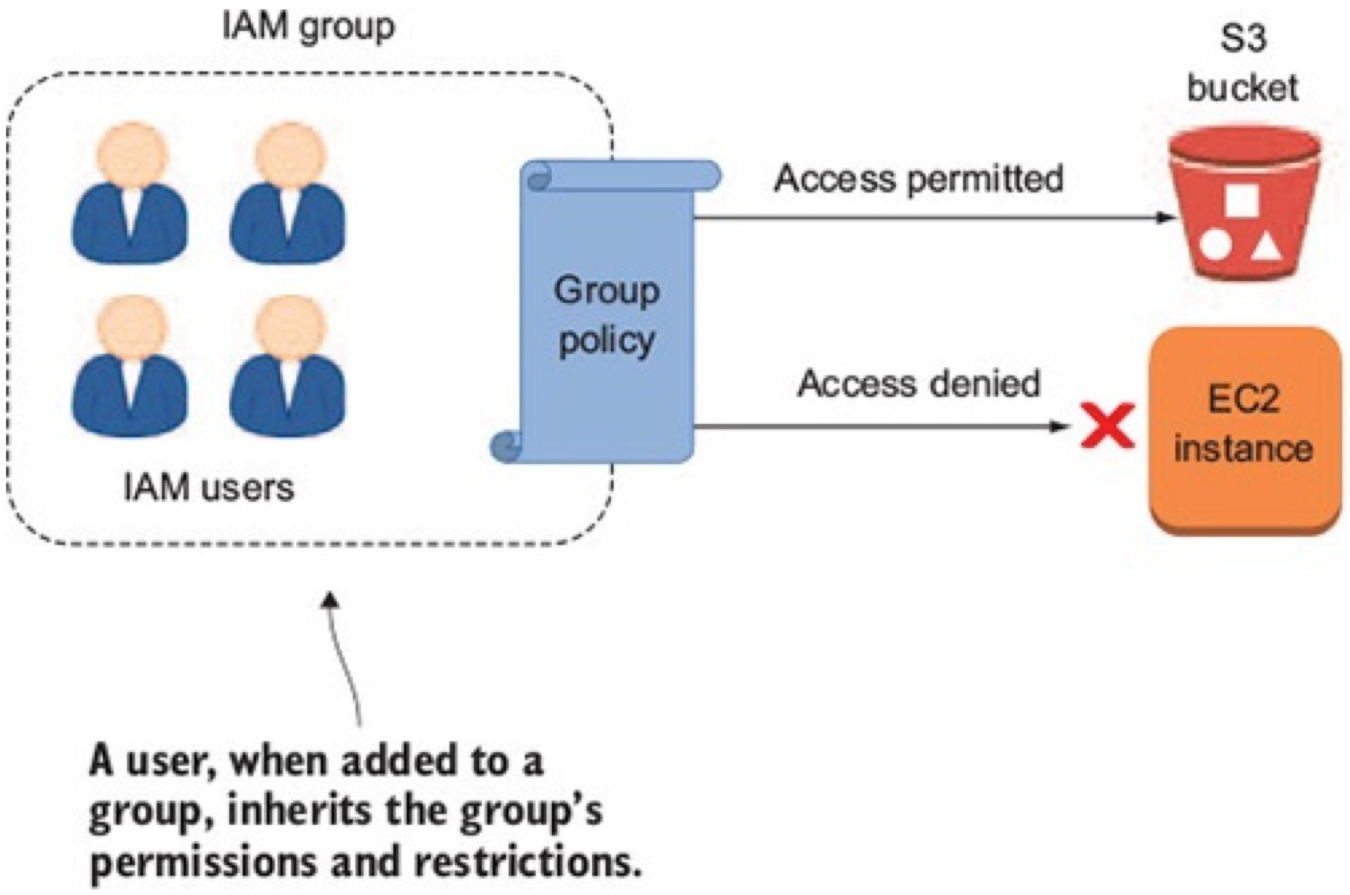
IAM - Configuring Access Rights

- IAM = Identity and Access Management
  - https://aws.amazon.com/iam/
- The IAM console promotes best practices
  - Select IAM from services panel
- Create and manage users, roles and groups
  - Users are named operators with permanent credentials
  - Roles have temporary credentials (for specific execution)
- Policies define permissions and are attached to users or roles
- AWS recommends to lock down root
  - Eliminate root access keys
  - Note: Only root user can control budget

IAM Management Console



IAM User Groups



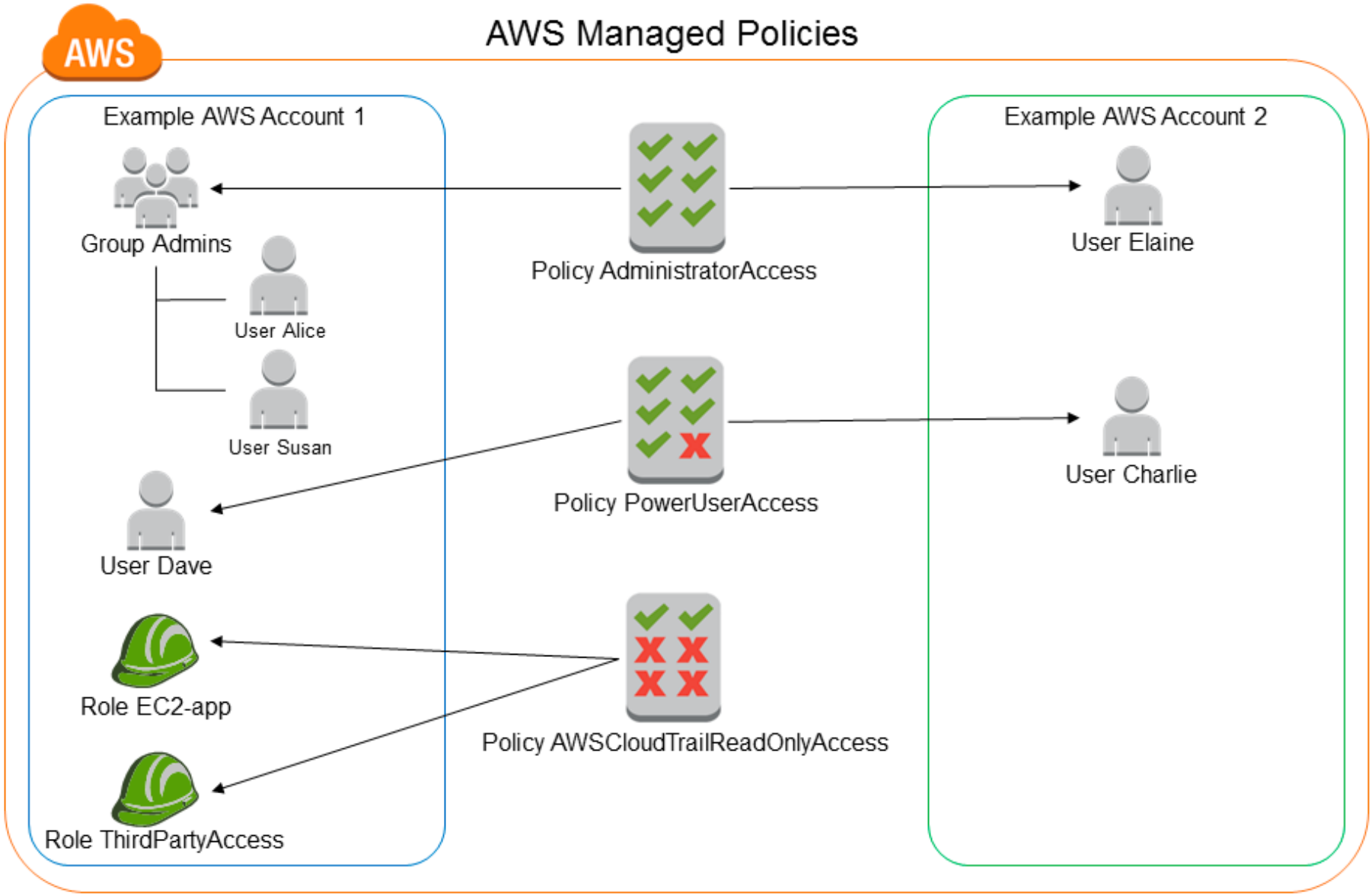
Source: David Clinton, Learn AWS…

Policy Types

- Identity-based policies
- Resource-based policies
- Permissions boundaries
- Organization SCPs (Service Control Policies)
- Access control lists (ACLs)
- Session Policies

https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

Stand-alone Policies managed by AWS



JSON

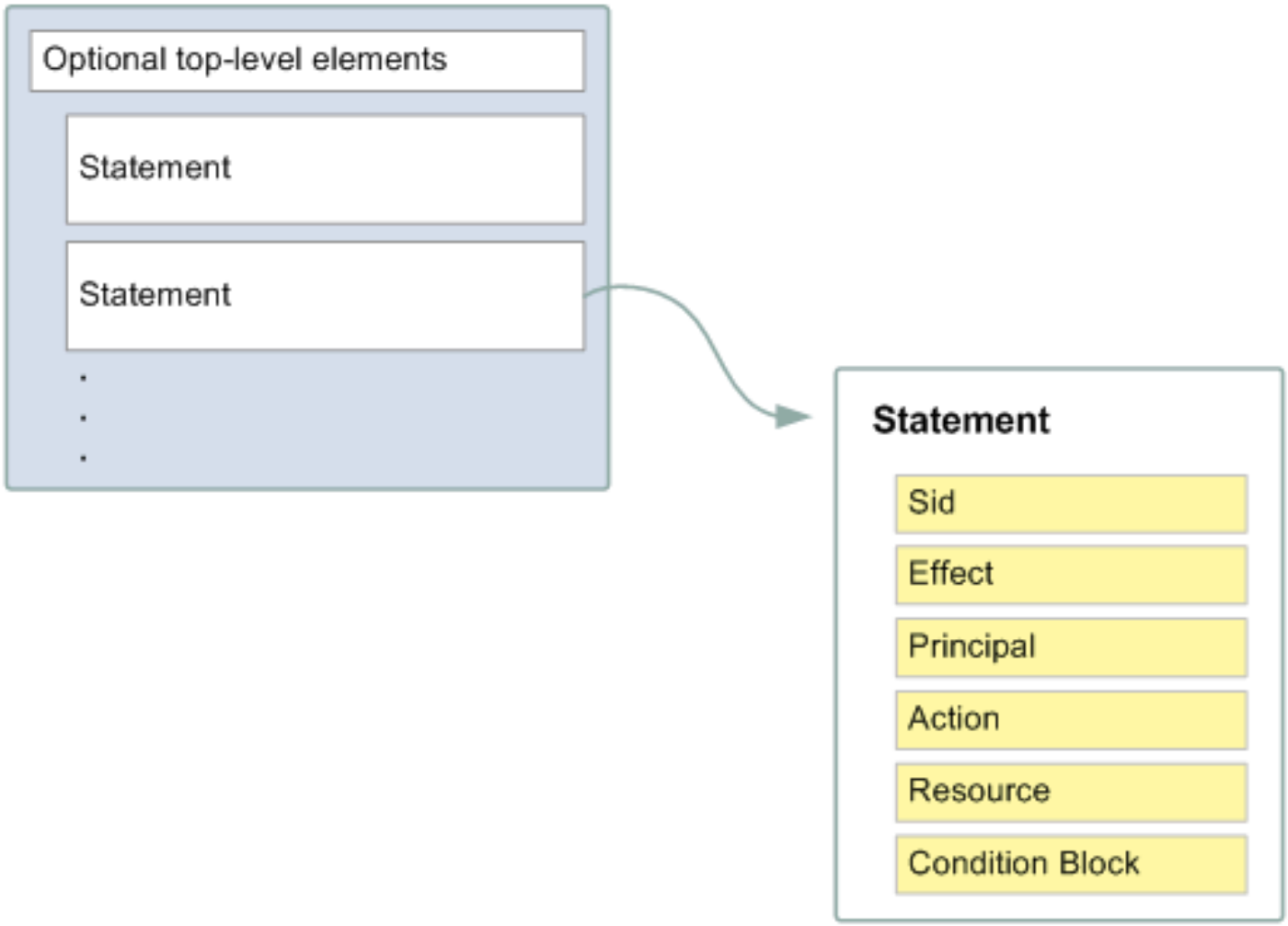
- Java Script Object Notation
- Data Exchange Format (like XML), works with many languages
- **Key-value pairs**, can be arranged in objects and arrays and be nested
- **JSON Schema** is a vocabulary that allows you to annotate and validate JSON documents

```
var jane = {
    "firstName" : ”Jane",
    "lastName" : ”Smith",
    "age" :  "25"
};
```

```
{"students":[
        {"name":"John", "age":"23", "city":”New York"},
        {"name":"Steve", "age":"28", "city":”London"},
        {"name":"Peter", "age":"32", "city":”Fairfield"},
        {"name":”Jane", "age":”25", "city":”Frankfurt"}
]}
```

- Access e.g. with jane.firstName or students[0].age

JSON Policy Document

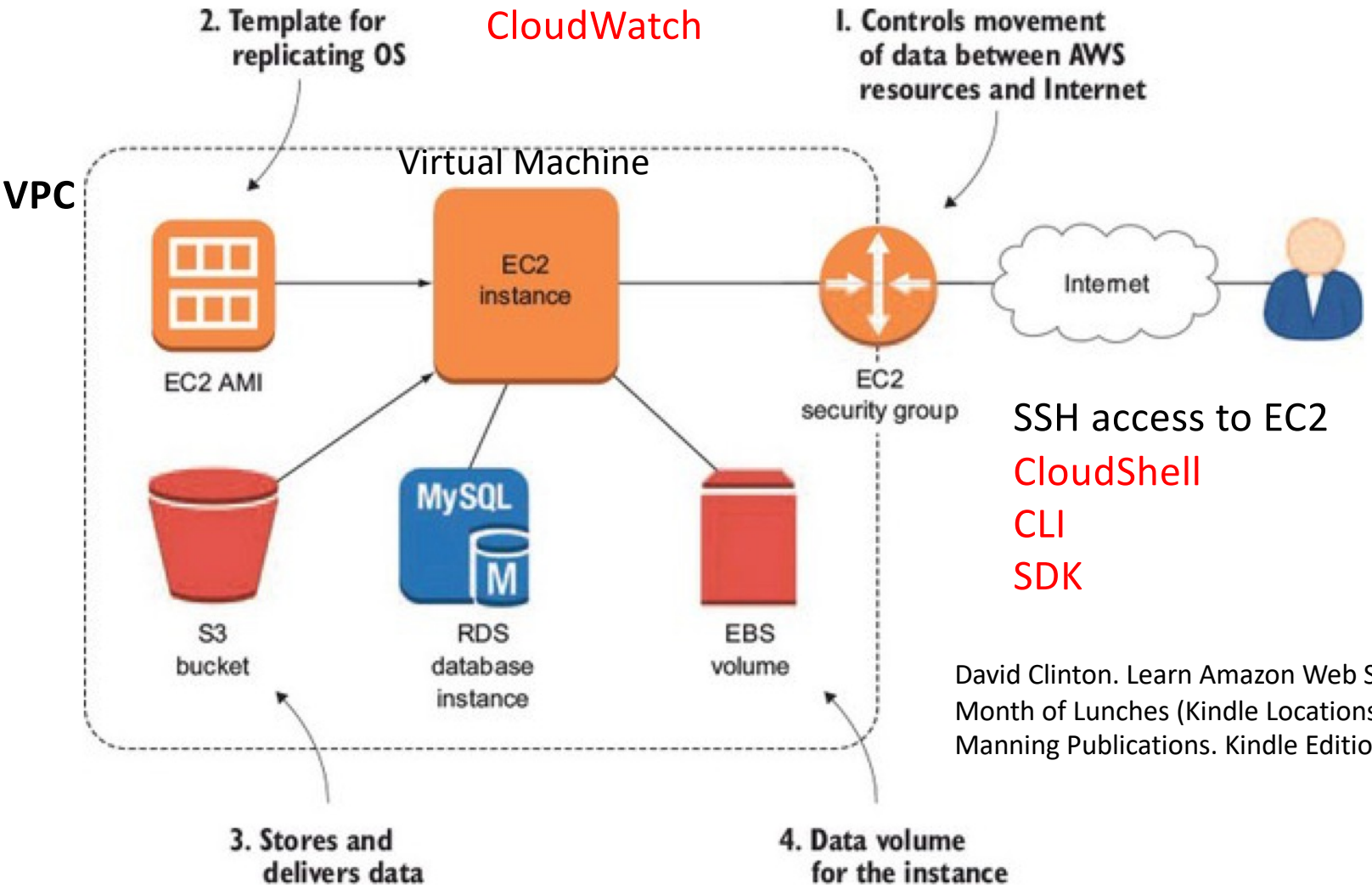


https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies.html

Example Policy with 3 Statements

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "FirstStatement",
   "Effect": "Allow",
   "Action": ["iam:ChangePassword"],
   "Resource": "*"
  },
  {
   "Sid": "SecondStatement",
   "Effect": "Allow",
   "Action": "s3:ListAllMyBuckets",
   "Resource": "*"
  },
  {
   "Sid": "ThirdStatement",
   "Effect": "Allow",
   "Action": [
    "s3:List*",
    "s3:Get*"
   ],
   "Resource": [
    "arn:aws:s3:::confidential-data",
    "arn:aws:s3:::confidential-data/*"
   ],
   "Condition": {"Bool": {"aws:MultiFactorAuthPresent": "true"}}
  }
 ]
}
```

Where do we stand?



SSH access to EC2

CloudShell

CLI

SDK

David Clinton. Learn Amazon Web Services in a Month of Lunches (Kindle Locations 502-506). Manning Publications. Kindle Edition.

What have we learned so far?

- We have set up our first service in the Cloud
  - Virtual Machines (EC2)
  - Virtual Private Cloud with Security Groups
  - A Relational Database Service managed by AWS
  - Storage – EBS and S3
- We have learned about some additional services
  - DynamoDB, SNS
- We can manage our service through the AWS Management Console or the CLI
- We can access services programmatically with the SDK
- We can use AMIs and snapshots
- We can create alarms in CloudWatch and monitor resources

What’s next?

- We need to ensure our service can cope with variable demand
  - Surges, low demand periods
  - Elasticity and scalability
- We need to be highly available to be able to serve our customers ”all the time”
  - Geographic redundancy, availability zones
  - Avoid single points of failure

Homework

- Create a DynamoDB table, insert and update items, select items
- Use CloudShell, CLI and SDK
- DynamoDB (use CloudShell instead of Cloud9)
  - https://aws.amazon.com/getting-started/projects/create-manage-nonrelational-database-dynamodb/?trk=gs_card
- RDS Recap (optional)
  - https://aws.amazon.com/getting-started/hands-on/create-mysql-db/

Module 3
Availability and Scaling



Hardening a Cloud Service

Availability and Scaling

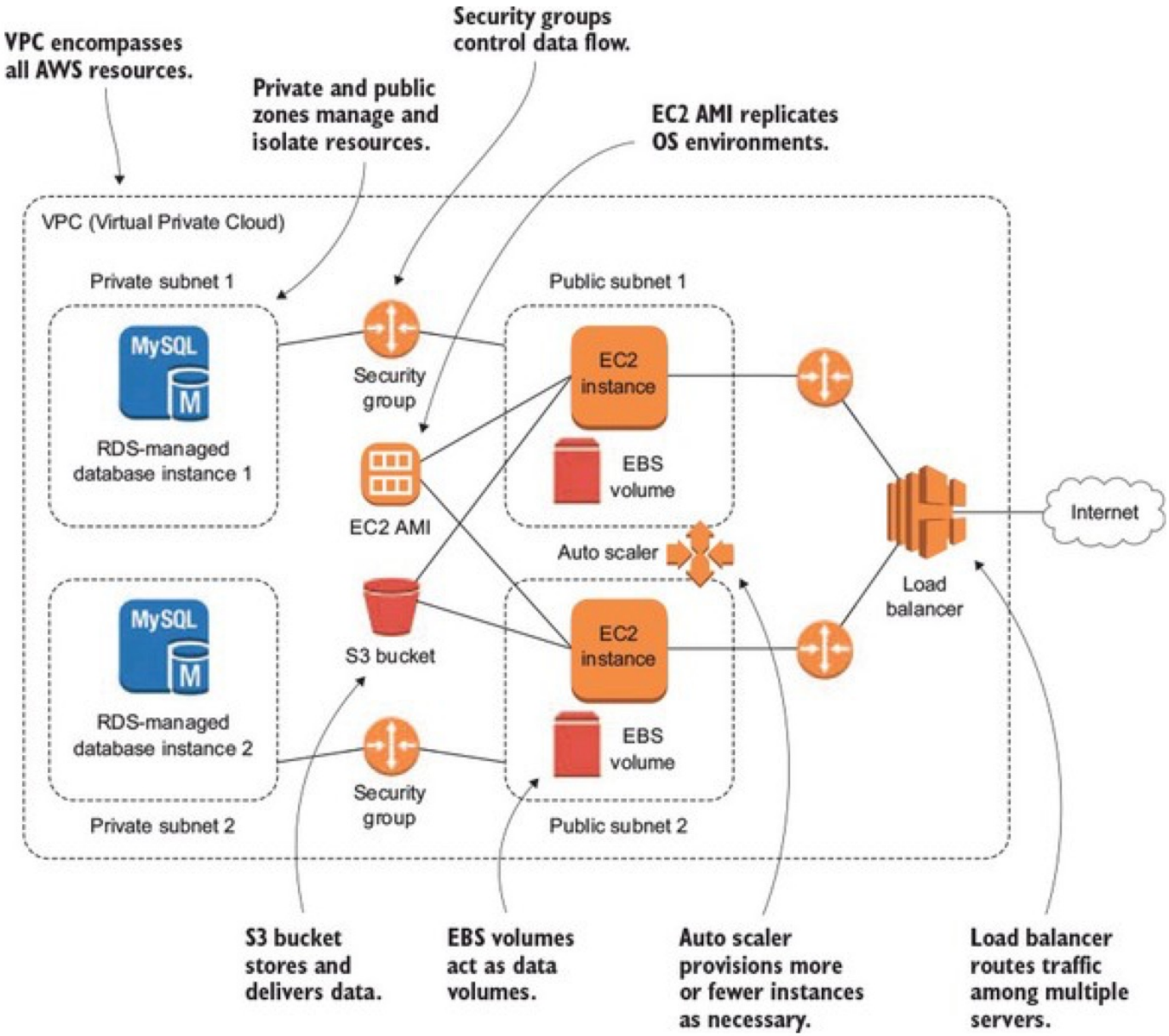
AWS Service Level Agreements (SLAs)

https://aws.amazon.com/legal/service-level-agreements/

- For **Compute**, "AWS will use commercially reasonable efforts to make the Included Services each available for each AWS region with a Monthly Uptime Percentage of at least **99.99%,** in each case during any monthly billing cycle (the "Service Commitment"). In the event any of the Included Services do not meet the Service Commitment, you will be eligible to receive a Service Credit as described below."
  - AWS gives service commitments for 'Included Services": EC2, EBS, ECS (Container Services) and Fargate (Serverless Container Service)
  - Less than 4.5 minutes downtime per month
- For an **individual EC2 instance**, the service commitment is **90%** hourly uptime.
- For higher level services, availability is often **99.9%**
  - Less than 45 minutes downtime per month

Expanded Configuration

Availability
Scaling
Security



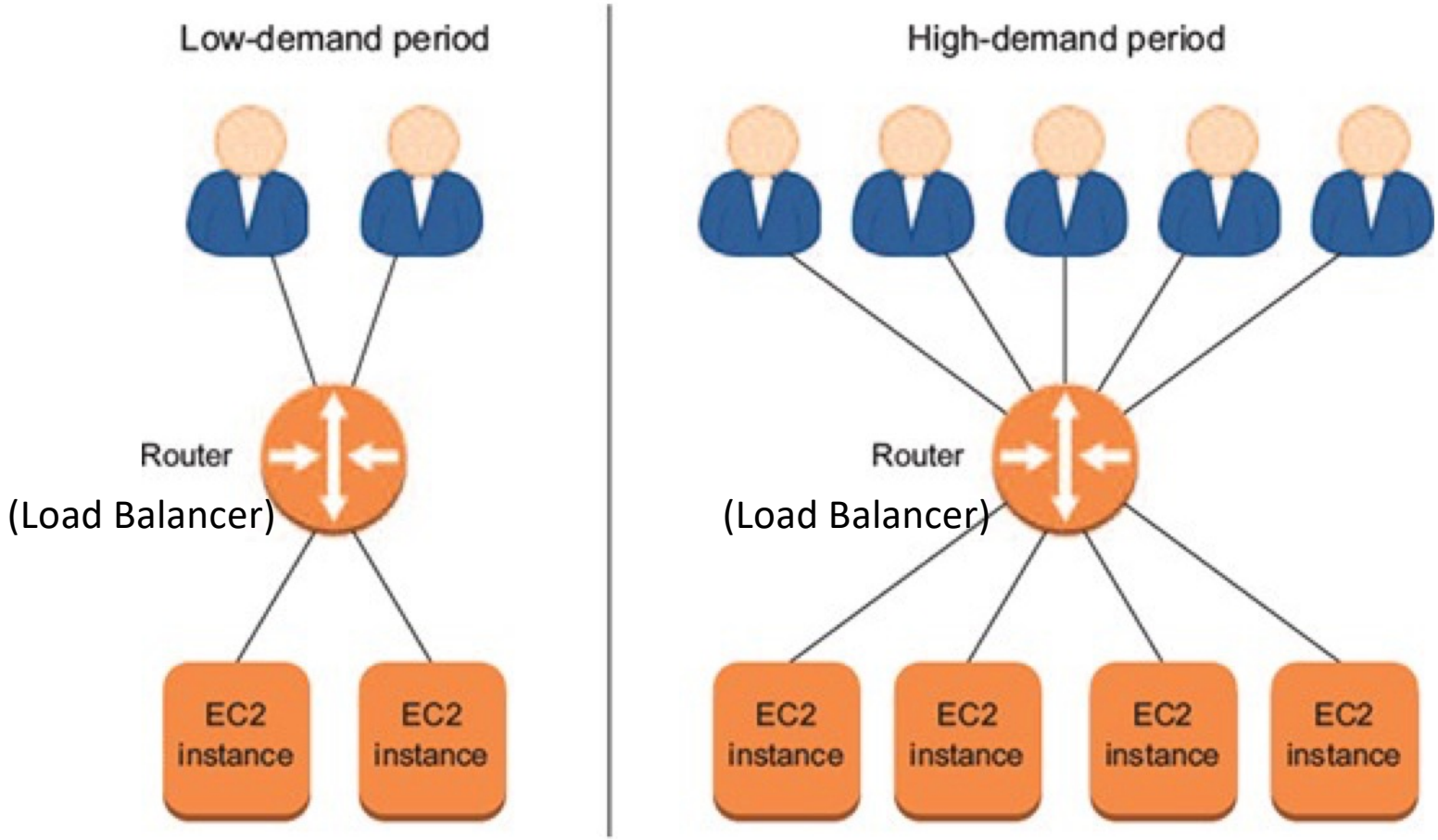
Source: David Clinton, Learn AWS…

Availability and Scaling

- **Load Balancing**
  - Distributing load between resources
- **Auto Scaling**
  - Automatic provisioning and deprovisioning (elasticity)
- Networking Tools
  - Isolating resources
- Content Delivery Networks
  - Scaling across the internet

Elasticity

Dynamically adding and removing resources
Horizontal Scaling



Auto-Scaling

Source: David Clinton, Learn AWS…

Recap - NIST

This is what the Cloud is all about:

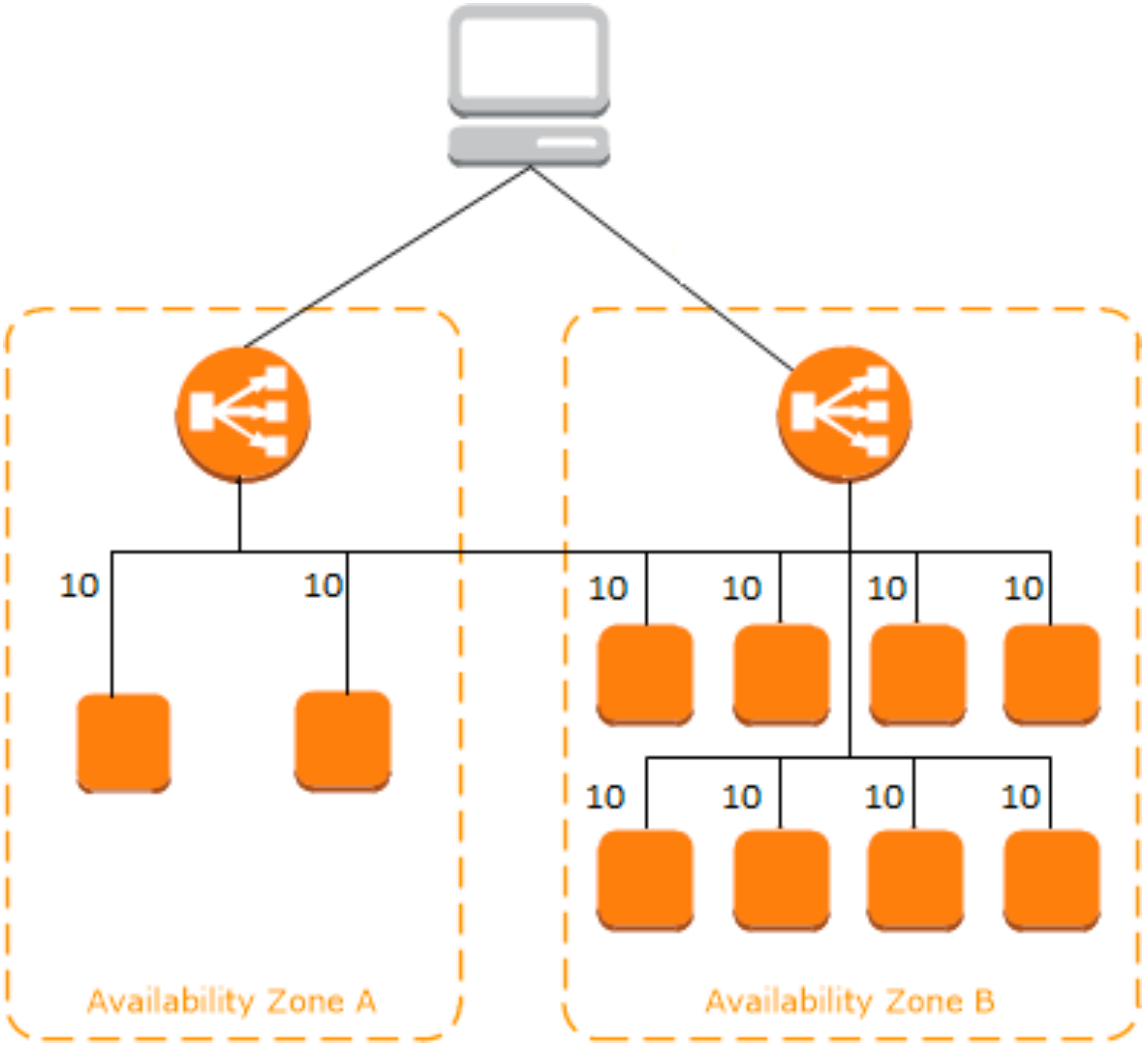
- On-demand self-service
- Broad network access
- Resource pooling
- Rapid elasticity
- Measured service

Load Balancing
Automating Availability and Scalability

- Load Balancers listen for incoming work and automatically distribute it to different targets (groups of virtual machines, containers or other services)
  - They can apply policies based on target state (like utilization)
- AWS Elastic Load Balancing routes traffic within a VPC and comes in four flavors
  - **Application Load Balancing** operates on Layer 7 (individual request level). Best for HTTP and HTTPS traffic
  - **Network Load Balancing** works at Layer 4 (connection level) and is best for TCP, UDP, TLS traffic. Highly performant
  - Gateway Load Balancing works with fleets of third party appliances

Cross-Zone Load Balancing

- The router (node) in each zone can distribute to both zone, so all nodes receive equal traffic
- **Default for Application Load Balancing**
- Disabled for Network Load Balancing



Source: AWS

https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/how-elastic-load-balancing-works.html

Routing Algorithms

- Application Load Balancer
  - Evaluates listener rules in priority order
  - Selects from action group for selected rule in round robin fashion
- Network Load Balancer
  - Selects a target from the target group for the default rule using a flow hash algorithm. It bases the algorithm on:
    - The protocol
    - The source IP address and source port
    - The destination IP address and destination port
    - The TCP sequence number
  - Routes each individual TCP connection to a single target for the life of the connection.

Creating an Application Load Balancer for HTTP

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/application-load-balancer-getting-started.html

- Start at least 2 (webserving) instances in **2 availability zones**
  - Clone an instance if you need to create a second one
  - Change the index.html files to provide different 'greetings'
- On the EC2 Dashboard, select '**Target Groups**' and create one
  - Register targets (i.e. running instances) to the target group
  - Make sure you click the button to register
- Go to the EC2 Dashboard, select 'Load Balancers'
- On the next panel, choose '**Application Load Balancer** for HTTP'
- Configure the load balancer
  - Name the load balancer and select 'Internet-facing'
  - A default listener is set up
  - Select availability zones (see above, at least 2 for high availability)
- **Configure security group to enable HTTP traffic on port 80**
- Configure routing – select target group
- Note: Load Balancers are charged by hour and traffic
- When status is 'active', type the load balancer DNS name into browser URL bar

Load Balancer Rules

- Select the listener you want to add a rule to
- Choose action ‘Edit listener’
- On the listener page, choose ‘Add rule’
- Set **condition** (e.g. path)
- Select **target group** to route to if condition is fulfilled (or other action)
- Set **priority** (in which order the rule will be evaluated) and **create**
  - More specific rules first
- Can be used to route specific request from a website to specific servers, e.g. a path **/wp-login.php** leads to a target group with working wordpress servers

Module 3 - ctd
Availability and Scaling



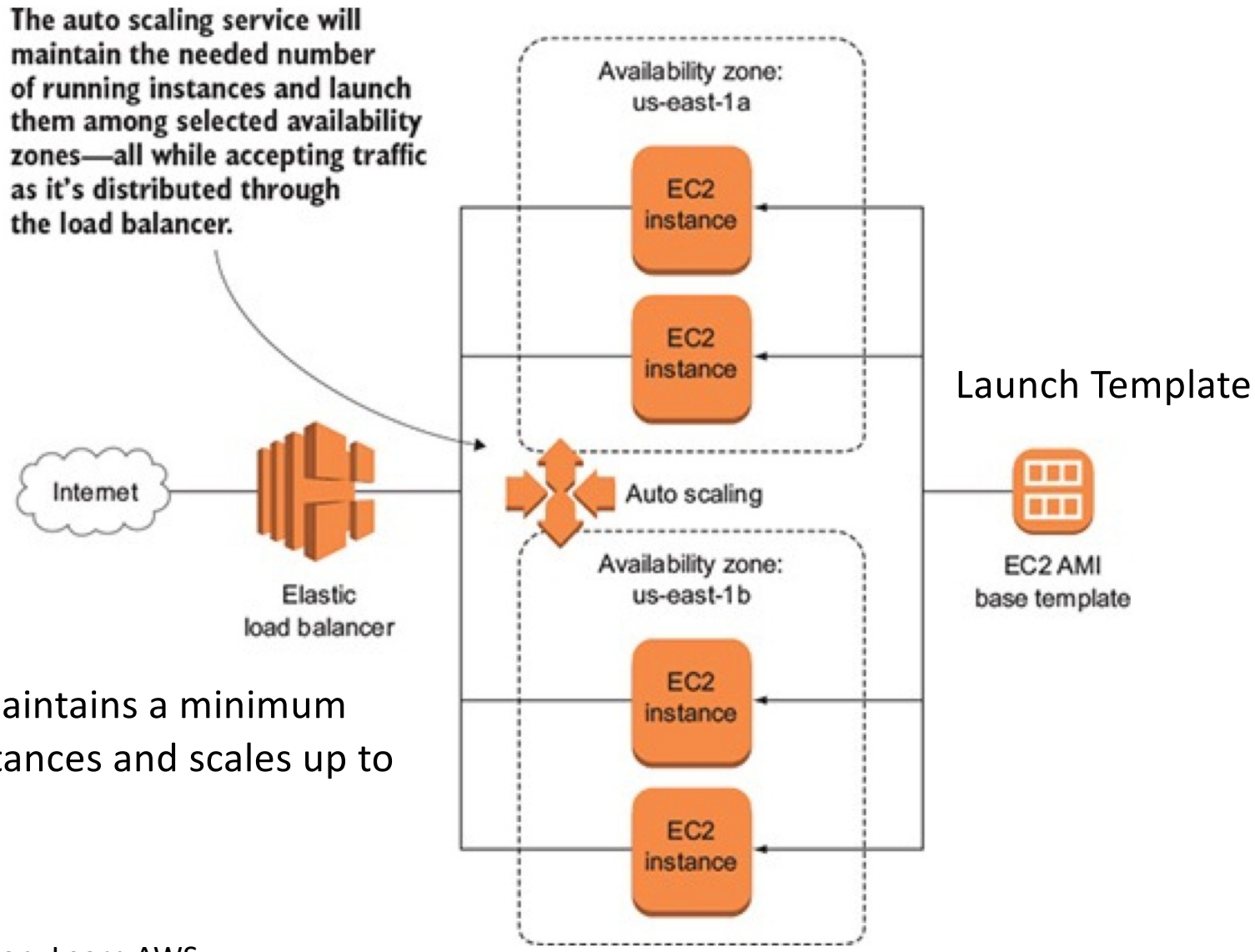
Load Balancer Recap

- Load Balancers distribute workload between multiple targets, thus increasing availability and performance
- Different target groups can contain different types of compute resources
  - EC2 instances, containers, Lambda functions
- Rules allow routing between different target groups
  - Variety of conditions
- Application load balancers are best suited for HTTP traffic

Amazon EC2 Auto Scaling

- Automatically add or remove EC2 instances based on rules
- Often sits ‘behind’ a load balancer
- Fleet Management
  - Monitor health of running instances
  - Replace impaired instances automatically
  - Balance capacity across availability zones
- Scheduled Scaling
- Dynamic Scaling (based on metrics and rules)
- Predictive Scaling (in advance)
- https://aws.amazon.com/ec2/autoscaling/

Auto Scaling

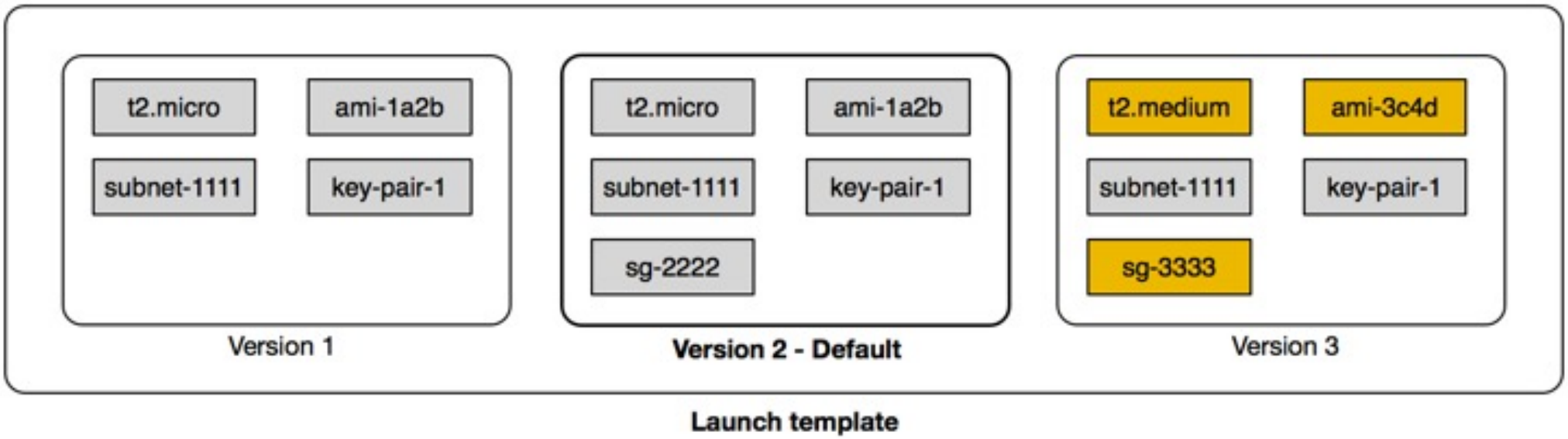


Auto scaling maintains a minimum number of instances and scales up to a maximum.

Source: David Clinton, Learn AWS…

Launch Templates

- Launch templates allow to save an instance **configuration** and reuse it later to 'clone like instances' (automatically)
  - AMIs, Instance Types, VPCs,
  - Autoscaling
    https://console.aws.amazon.com/ec2/home?region=us-east-1#LaunchTemplates:
- 'Launch configurations' are very similar with a different management interface, no longer recommended



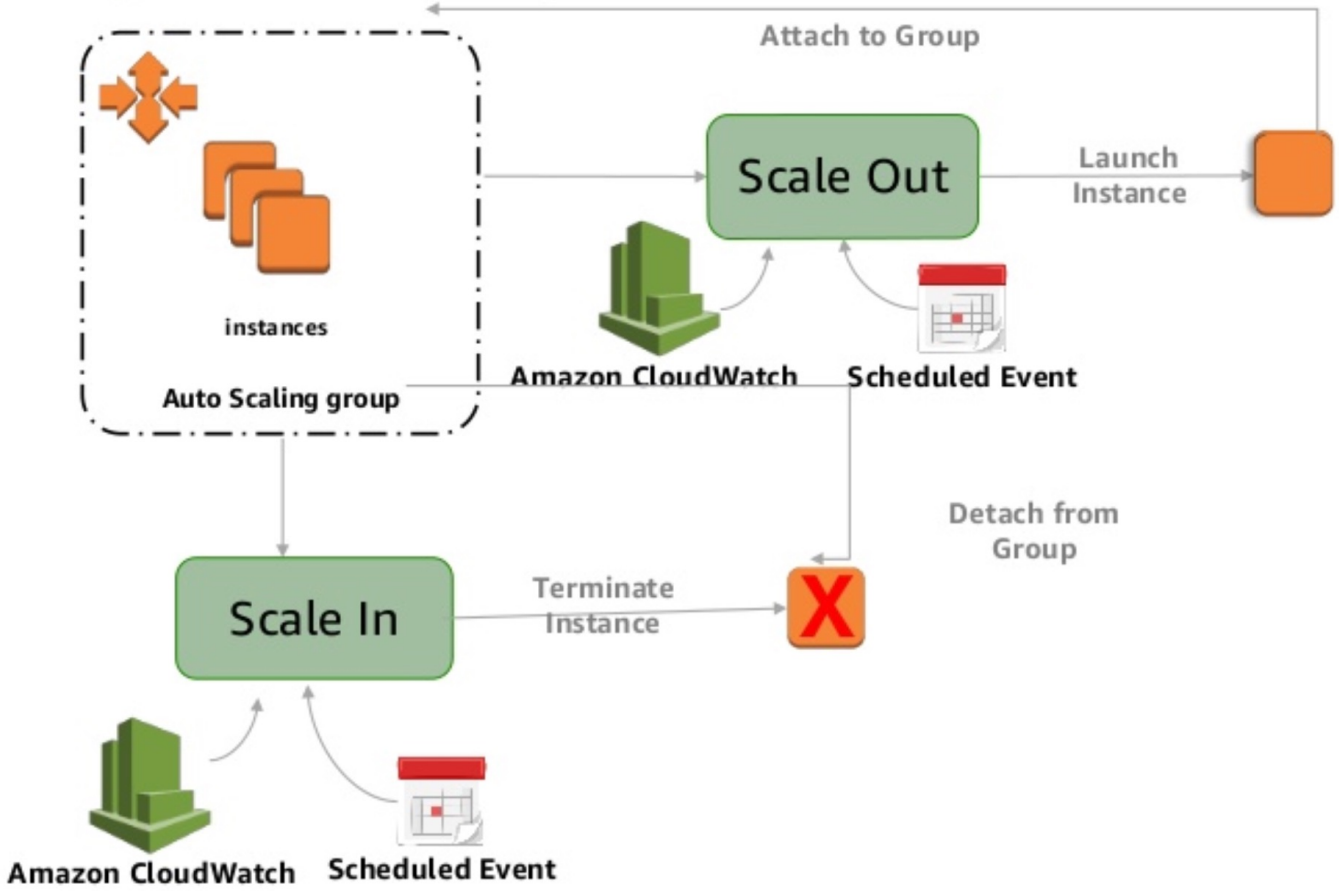
Creating a Launch Template for Auto Scaling

- Benefits of Launch Templates
  - Enable auto scaling
  - Simplify permissions
  - Ensure best practices (governance)
- Select ‘Launch Template’ from EC2 Dashboard
- Click create button and
  - Select AMI
  - Select instance type
  - Select key pair, VPC and security group
  - Option to request spot instances (advanced details)
- Once created, you can launch an instance or create an Auto Scaling Group
- Note: Launch Configurations are similar, but older

Creating an Auto Scaling Group

- Configure group details in addition to elements taken from template
- Attach to a load balancer (target group)
- Configure scaling policies
  - Max and min size
  - Metric (CPU, Network)
- Configure notifications
  - Send notifications to an SNS topic and a recipient
- Configure tags
- Tutorial for autoscaling with load balancing: https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-register-lbs-with-asg.html
- Note: Delete auto scaling group after test (or reduce number of desired instances to 0)

Auto Scaling Basic Lifecycle



© 2019, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Source: Awsome Day 2019 Detroit Slideshare

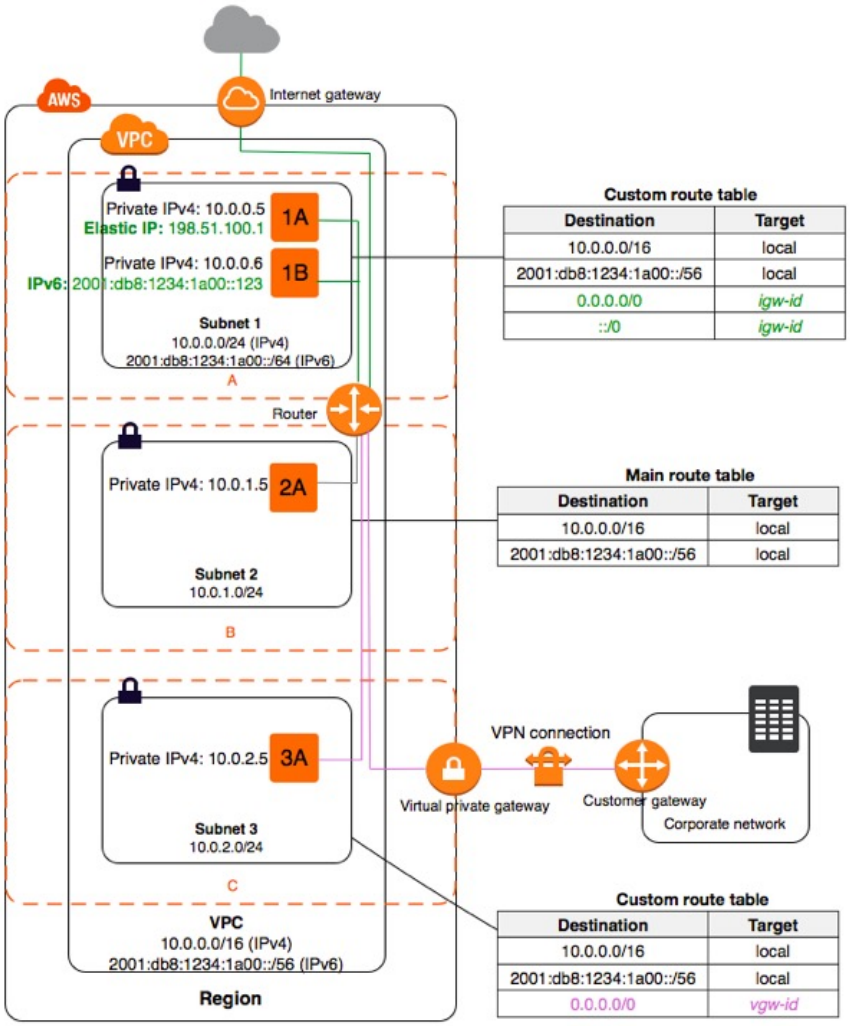
Module 4: Additional Material



VPC with Subnets Revisited

VPC span multiple AZs in a region

Having subnets for multiple AZs can enhance availability



Public traffic goes through an Internet Gateway

Only local traffic

Access to a VPN

Source: AWS Documentation

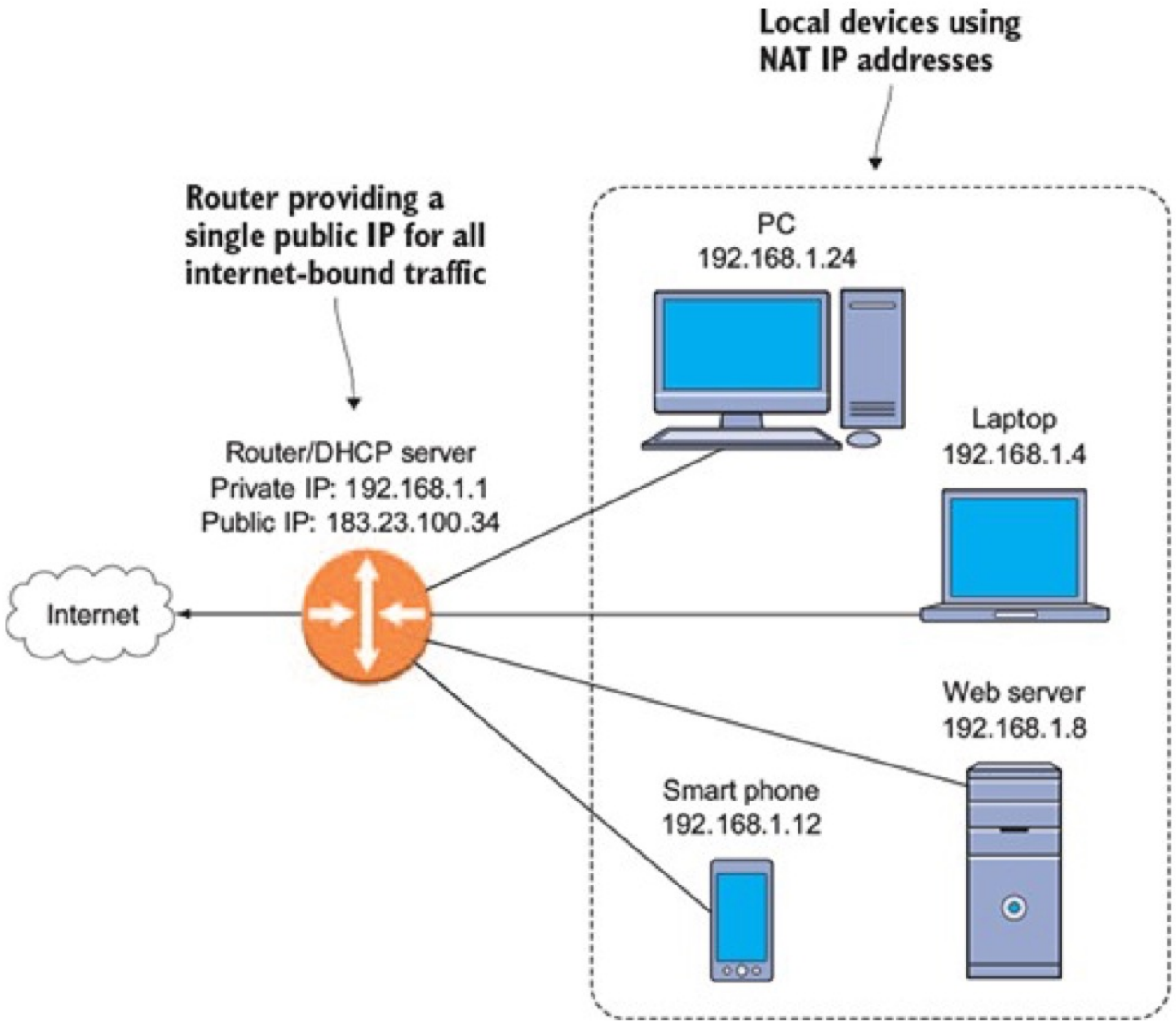
More about VPCs

- A logically isolated section of the AWS cloud that controls access to all resources created in to it
- AWS creates a default VPC for each account
- VPC dashboard reached from main console (Networking and Content Delivery Services)
  - Virtual Private Cloud
    - My VPCs
    - Routing Tables
    - Internet Gateways
    - Elastic IP Adresses
    - (Public and Private) Subnets
  - Security
    - Network Access Control Lists (ACL)
    - Security Groups (at an instance level)
- Note: There is a ‘EC2 Classic’ flat network shared among multiple users for early generation (t1) instances

Creating VPCs

- Manual Creation through dashboard (under My VPCs)
  - Need to choose CIDR Block to allow for subnets (10.0.0.0/16)
  - Note: Internet gateway not default, has to be associated for public subnet
- 'Launch VPC Wizard' button (under VPC Dashboard) allows selection of templates
  - VPC with single public subnet
  - VPC with public and private subnets
  - VPC with public and private subnets and hardware VPN access
  - VPC with private subnet only and hardware VPN access
- Note: Wizard creates NAT gateway not covered by free tier as default (hourly charge, restrictions for Educate)
- Subnets within a VPC can be in different availability zones, to increase availability, replicate instances across different subnets

Network Address Translation (NAT)



Source: David Clinton, Learn AWS…

Content Delivery Networks (CDN)

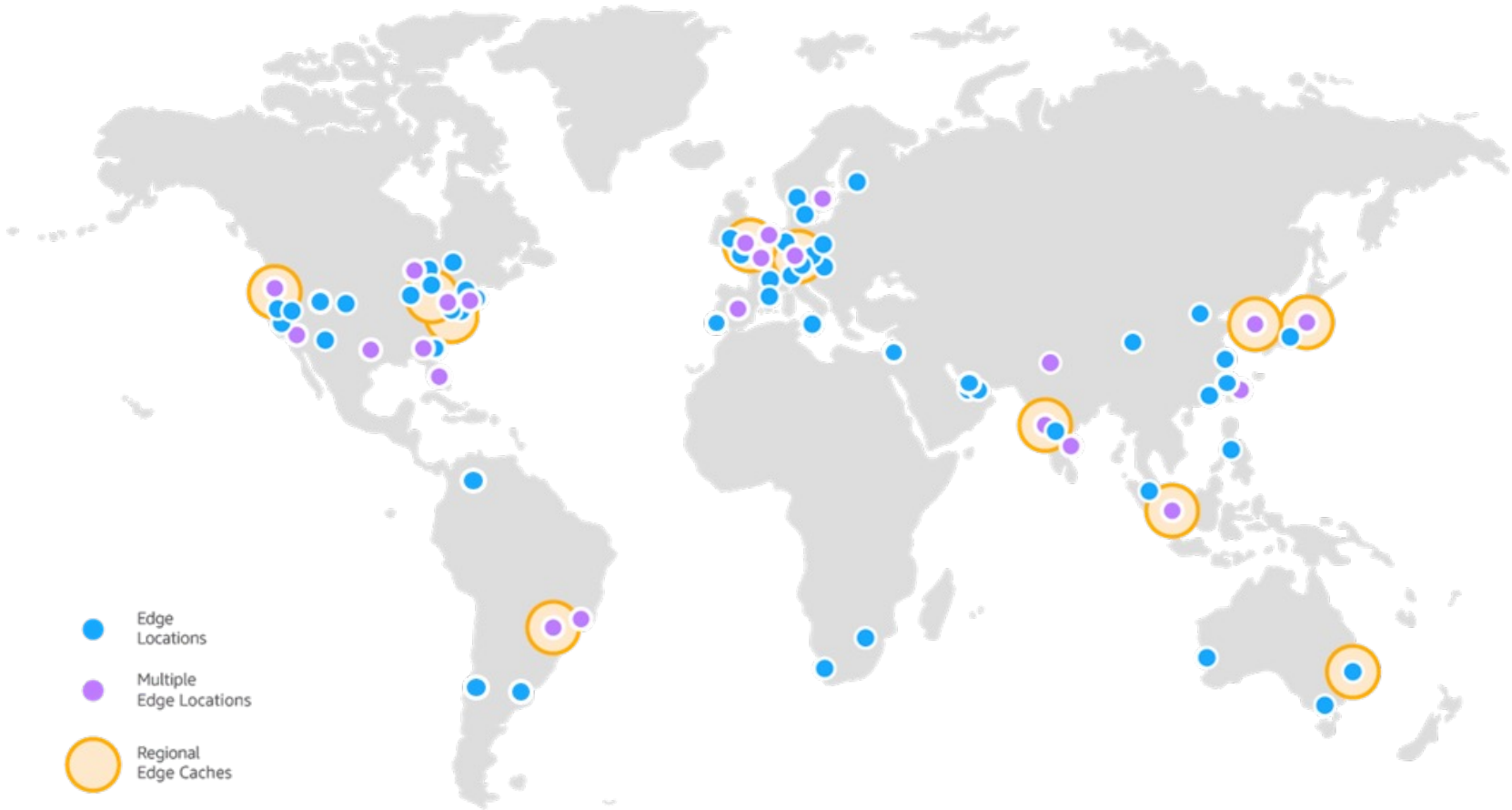
- Speeding up delivery of static and dynamic web content by caching it at ‘edge locations’ (i.e. closer to the user)
- When a user requests content, it is delivered from the location with lowest latency
  - If it is not already at an edge site, the CDN retrieves it from the ‘origin server’
- Amazon CloudFront
  - Many commercial alternatives, like Akamai or CloudFlare
  - CloudFront has more than 100 edge locations
  - https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html

Delivering Content from the Edge

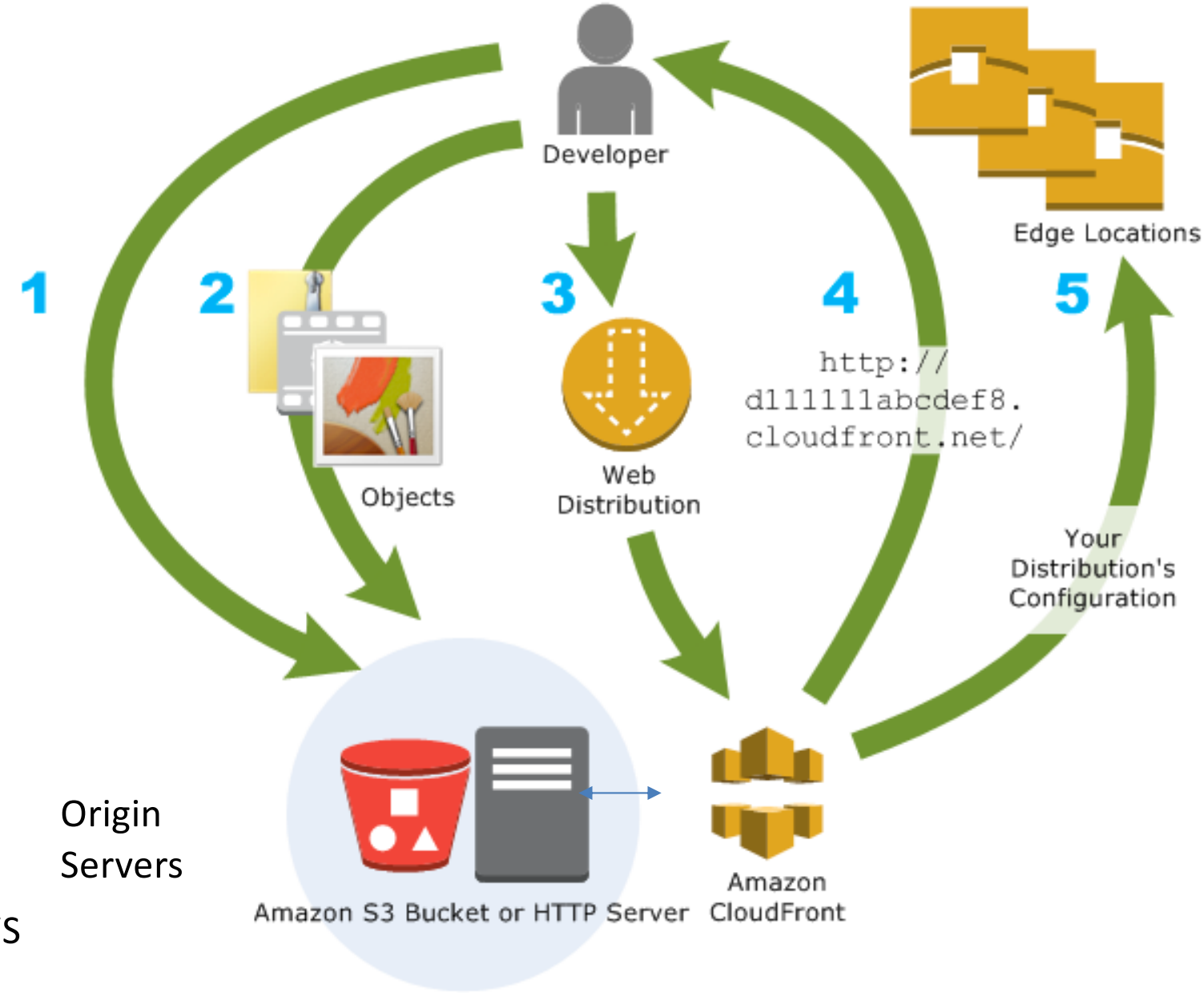


Source: David Clinton, Learn AWS…

Amazon CloudFront Infrastructure



Setting up CloudFront to Deliver from the Edge

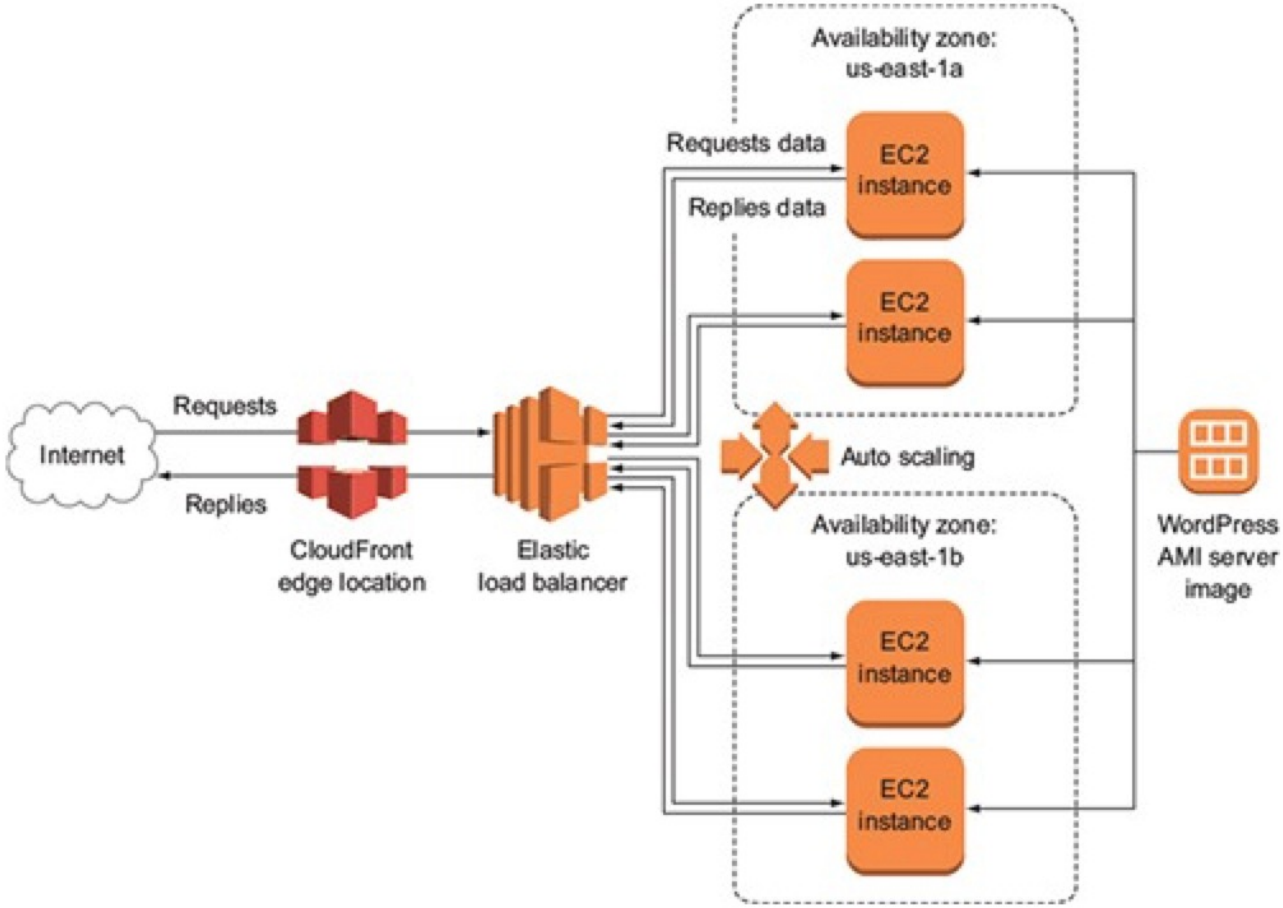


Source: AWS

Setting up CloudFront to Deliver from the Edge

1. Specify *origin servers*, like an Amazon S3 bucket or your own HTTP server, from which CloudFront gets your files which will then be distributed from CloudFront edge locations all over the world.
2. Upload your files to your origin servers.
3. Create a CloudFront *distribution*, which tells CloudFront which origin servers to get your files from when users request the files through your web site or application.
4. CloudFront assigns a domain name to your new distribution.
5. CloudFront sends your distribution's configuration (but not your content) to all of its *edge locations* or *points of presence* (POPs)

CloudFront integrated into a High Availability Setup



Source: David Clinton, Learn AWS…

Building a CloudFront Distribution

1. Select ‘CloudFront’ from main console
2. Click the blue ‘Create Distribution’ button
3. Select delivery method
   - Web for HTTP/HTTPS (S3 bucket or Web Server or Load Balancer)
   - RTMP for Adobe Flash Media (needs S3 bucket)
4. Set distribution parameters
   - Origin settings (name, permissions)
   - Cache Behavior
     - TTL = Time-to-Live, i.e. time to wait before CloudFront looks for an update
   - Distribution settings
     - Price class (regions for edge locations)
     - Certificates
5. Create distribution (click blue button), wait for it to be enabled, then access content through CloudFront domain name, e.g.

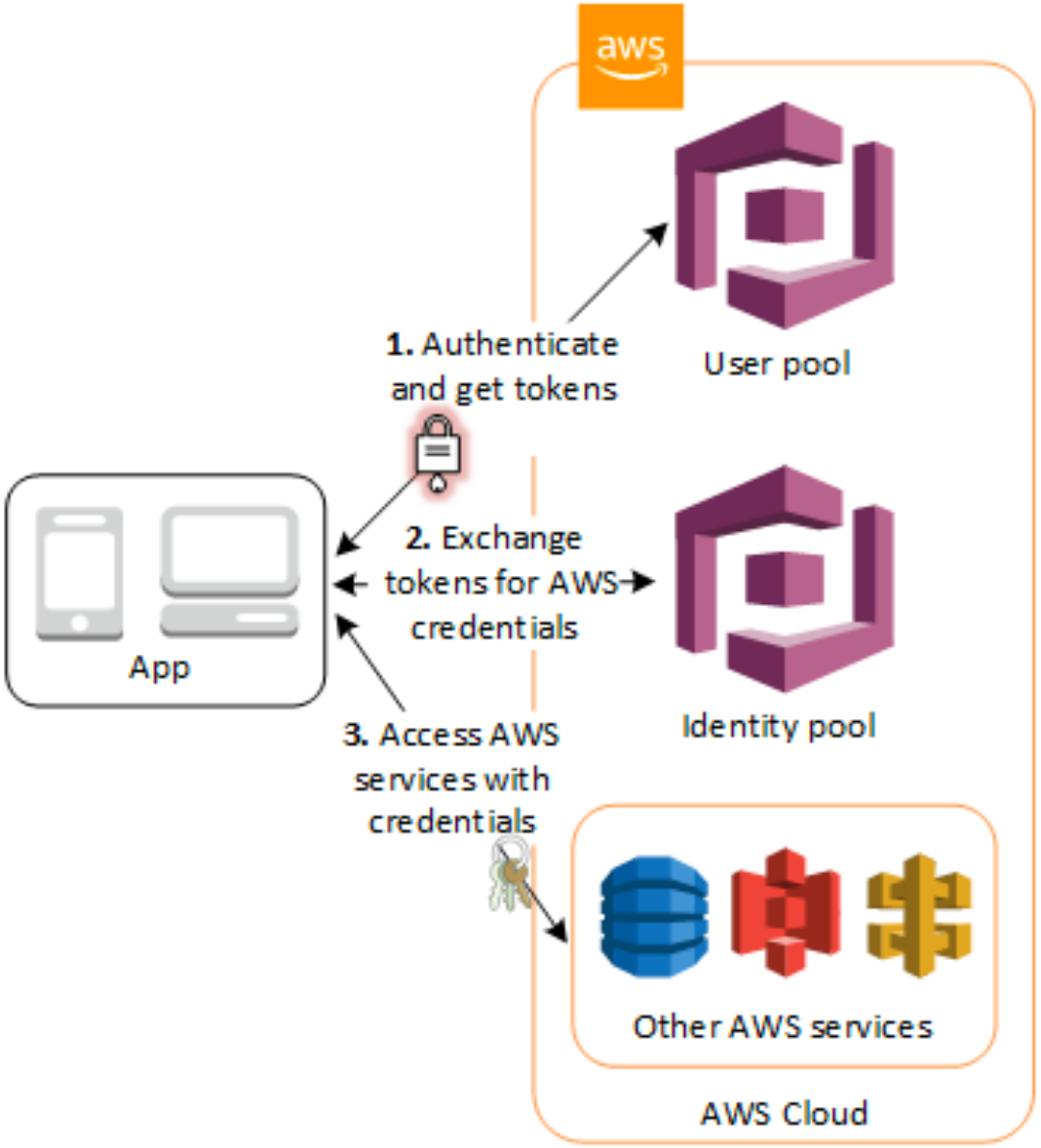
   d1kamfgwnn74ds.cloudfront.net/index.html

Domain Name System (DNS)

- Makes your service reachable through a domain name that gets translated to the IP address
- AWS Service: Route 53
  - Domain Registration
    - Costs between $10 and $15 per year
  - DNS Management (Hosted Zones)
  - Traffic Management (Redirection)
  - Availability Management

Amazon Cognito

Authentication, Authorization and User Management for Mobile and Web



Homework

- Do the exercises demonstrated during the lecture
  - Configure a load balancer and test it
  - Add rules to the listener
  - Create an autoscaling group, vary the number of instances desired
  - Shut everything down at the end!

Module 5
Automation



Automation

Deployment and Lifecycle Management

- **Amazon CloudFormation**
  - Set up and manage '**stacks**' defined in YAML or JSON
  - Infrastructure-as-Code
  - **Severless Application Model (SAM)** is an extension
- **Amazon Elastic Beanstalk**
  - Deploy and scale web application
- Amazon Lightsail
  - Simple blueprints (starter environments)
  - Doesn't work with Learner Labs
- Amazon OpsWorks
  - Deployment automation with Chef or Puppet

CloudFormation

- Implements a philosophy of ‘infrastructure as code’
- Allows to model infrastructure and application resource ‘stacks’ through templates using YAML/JSON or programming languages like Python
- Has a graphical Infrastructure Composer, integrates with Chef
- Allows reuse, modification and version control of existing templates
  - Partner Quickstart templates
- Automatic provisioning (and deprovisioning) of the resources in a stack



https://aws.amazon.com/cloudformation/

Working with CloudFormation

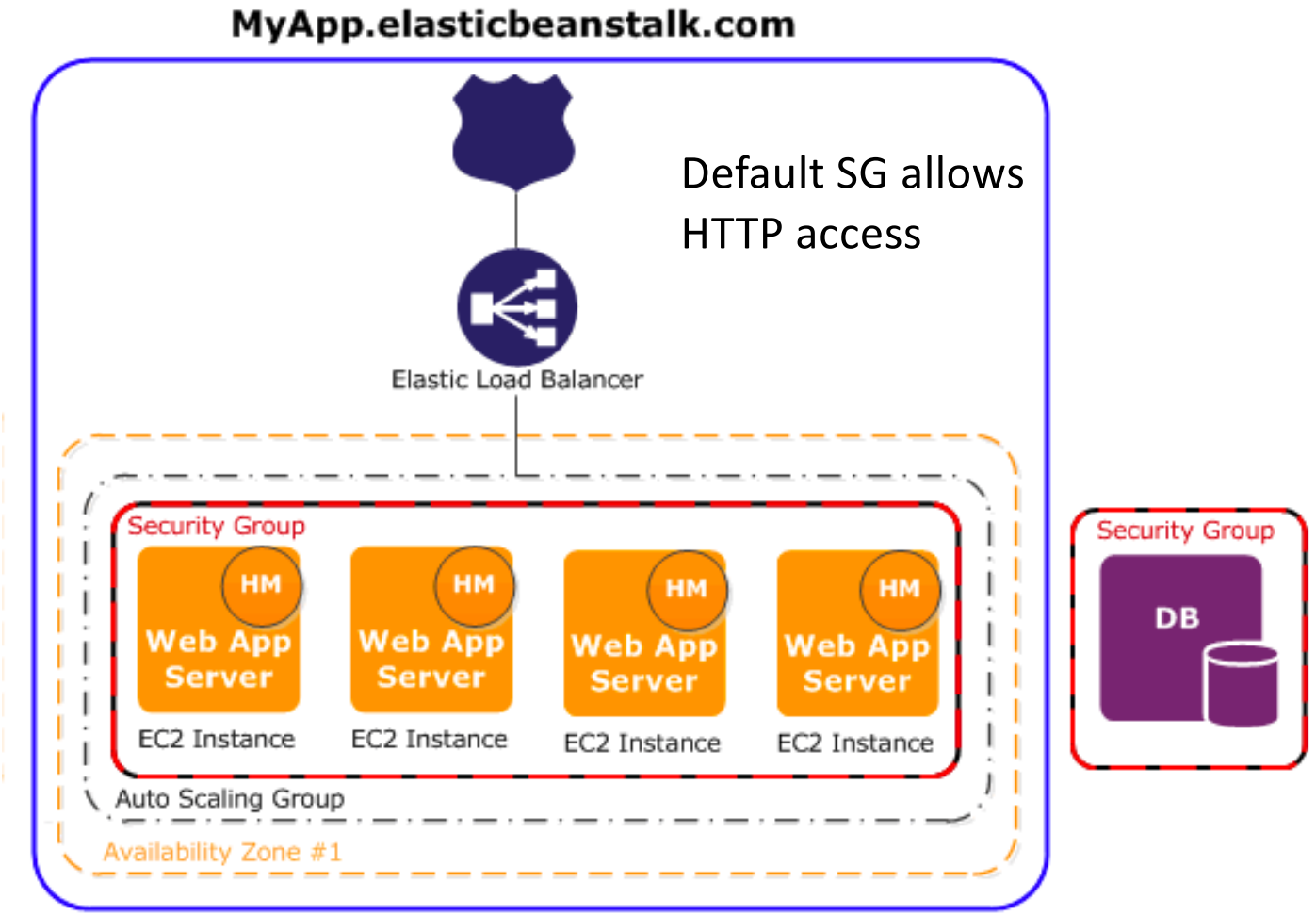
- https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/GettingStarted.html
- Create a template from scratch or select a pre-built template
  - https://aws.amazon.com/cloudformation/resources/templates/
- Make sure you have required elements (like **keys**)
- Create the stack
  - Enter the template URL (e.g. a file in an S3 bucket) or upload file
  - Enter a name
  - Where prompted, enter your choices
  - Make sure you choose LabRole when asked for role
- After clicking 'create' monitor the progress (events)
- Look at template in composer
- After completion (and test), delete the stack to avoid charges

Elastic Beanstalk

Elastic Beanstalk (EB)

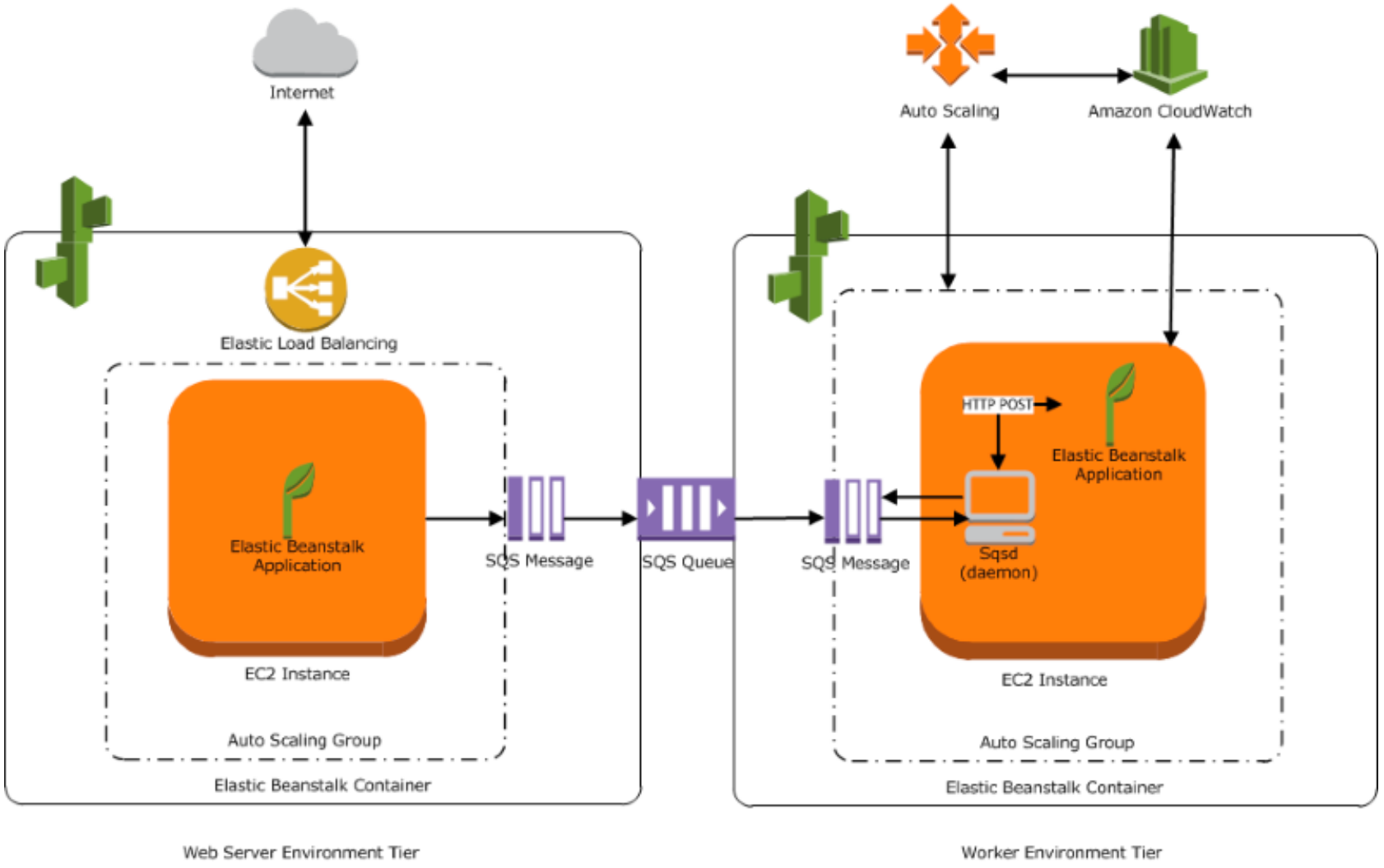
- AWS Elastic Beanstalk is a service for deploying and scaling web applications and other services
  - Supports **'Platforms'** like Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker
  - Familiar servers such as Apache, Nginx, Passenger, and IIS
  - **Web server environment and worker environment**
- You create an application source bundle
  - Zip or WAR file
- Then you select the environment, upload your code
  - Elastic Beanstalk creates a CloudFormation stack
  - Elastic Beanstalk automatically sets up load balancing, auto-scaling, health-monitoring
  - Choose LabRole etc. when asked
  - https://console.aws.amazon.com/elasticbeanstalk/home?ad=c&cp=bn&p=elbk&region=us-east-1#/welcome
- Elastic Beanstalk can provide managed updates of the 'platform'
- No extra charge for Beanstalk, but you pay for resources used

Web Server Environment



HM = Host Manager

Web Server and Worker Environment



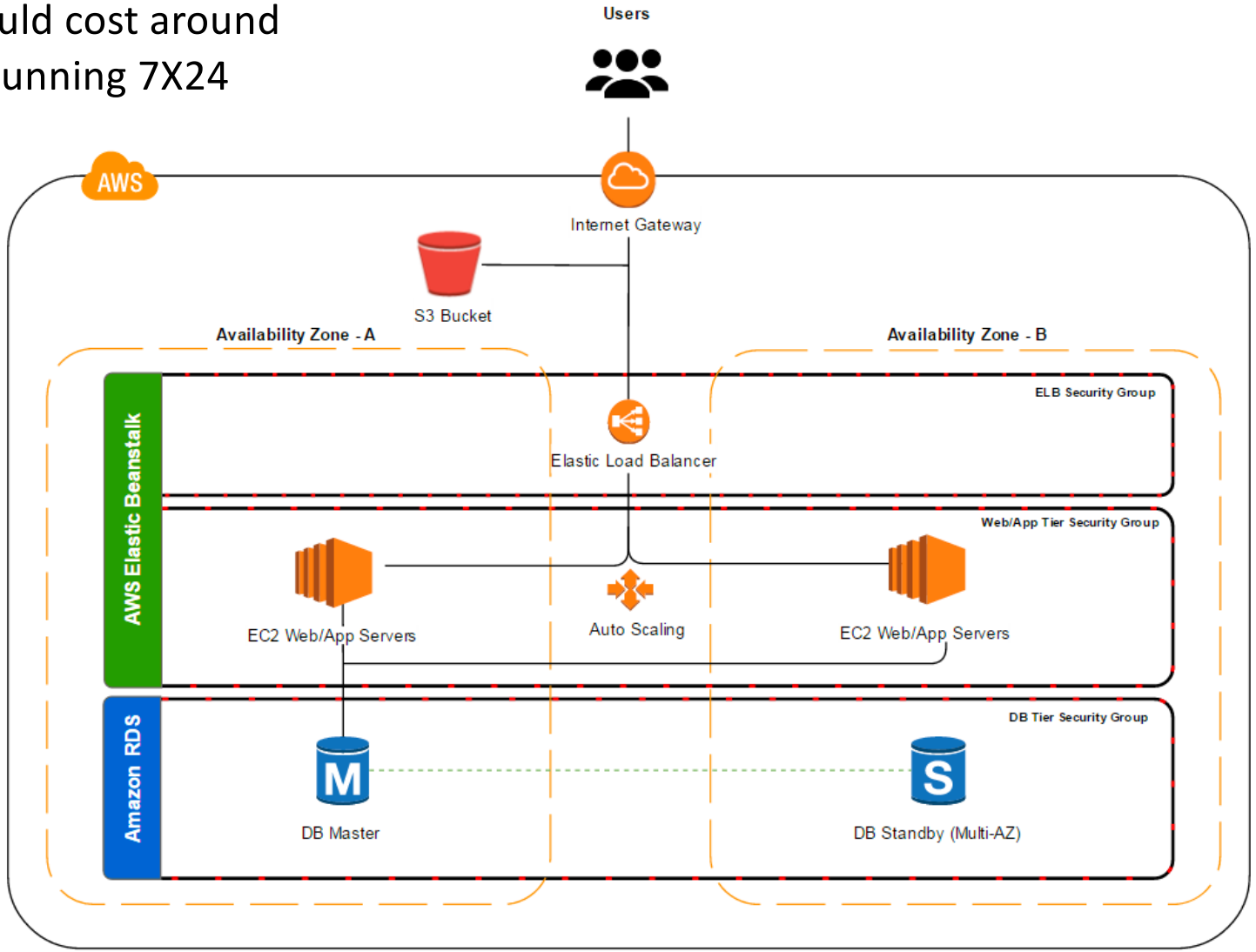
Getting started with a simple web app

https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/GettingStarted.html

- The blue ‘Get started’ button on the EB console brings you to the web app wizard
- Name the app
- Select ‘sample application’ or upload code
- Create application
- Elastic Beanstalk creates an environment with the following resources and launches the app (using CloudFormation automation under the covers)
  - EC2 security group
  - S3 bucket
  - Amazon CloudWatch alarms
  - **An AWS CloudFormation stack**
  - A domain name
- When the creation is complete (after about 5 minutes), the environment dashboard comes up
- Access the app through the url provided by EB

WordPress Production Environment

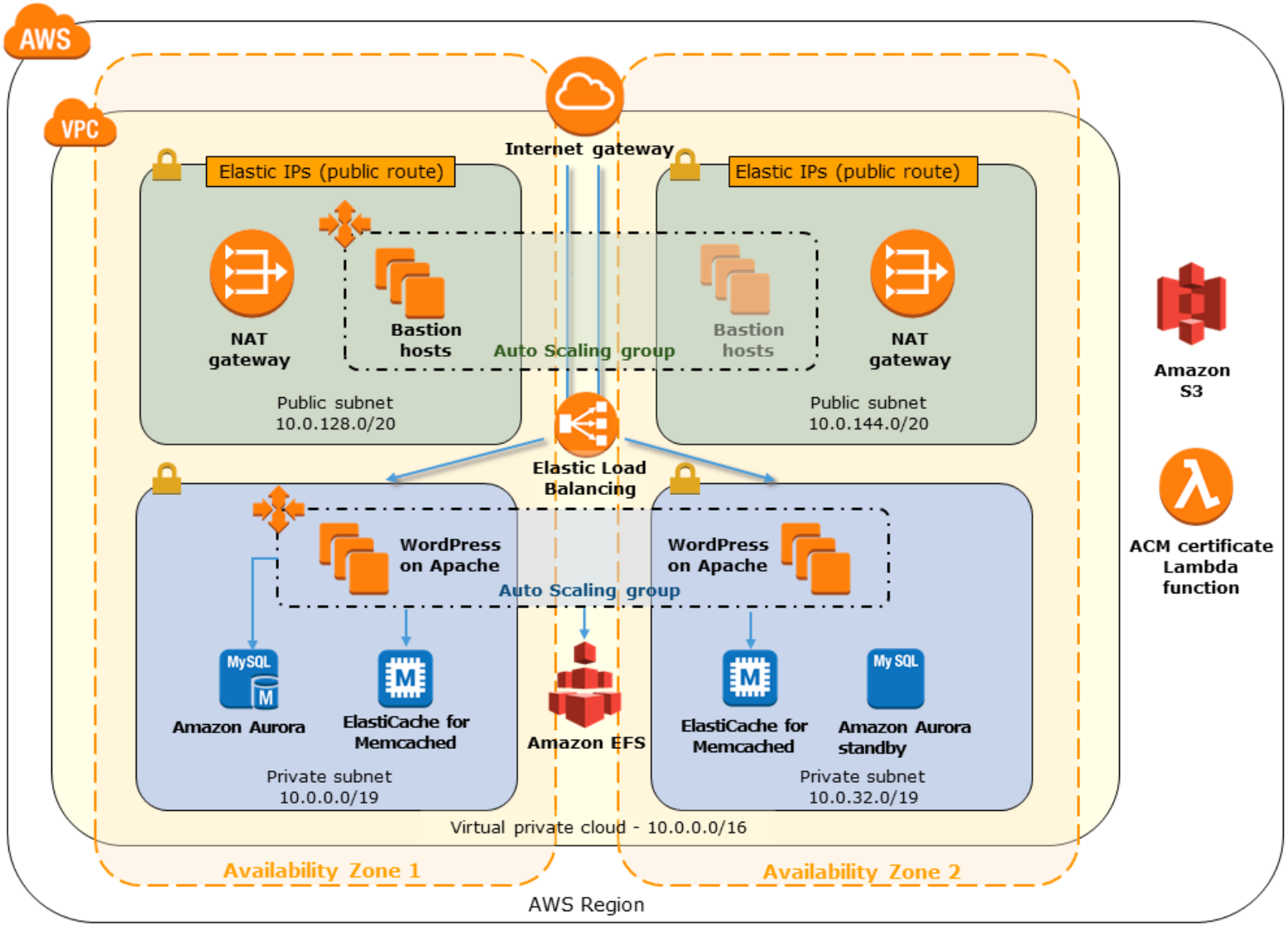
This setup would cost around
$450/month running 7X24



Source:https://aws.amazon.com/getting-started/projects/build-wordpress-website/?c=4

WordPress High Availability by Bitnami

https://aws.amazon.com/quickstart/architecture/wordpress-high-availability-bitnami/



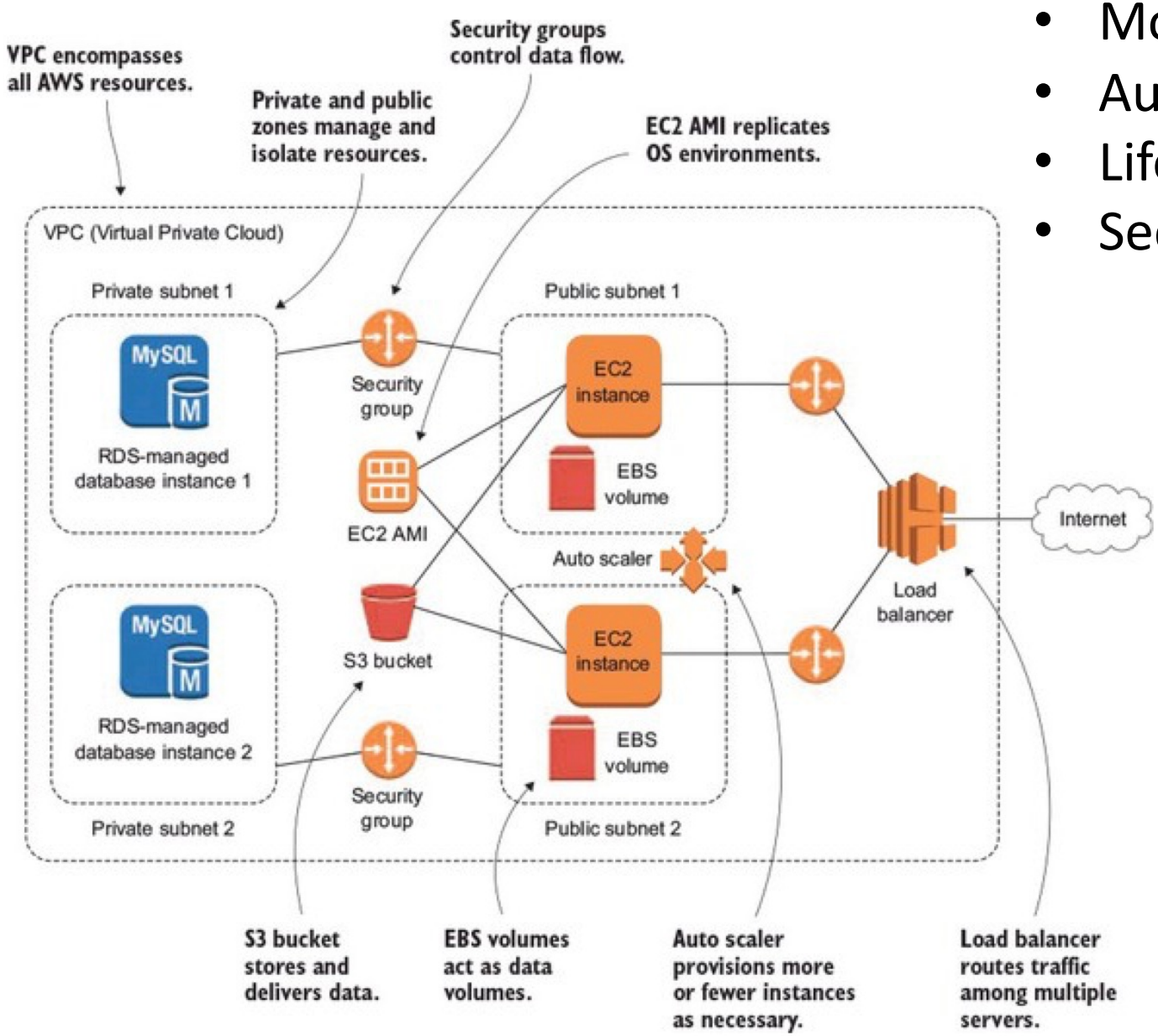
Source code on GitHub: https://github.com/aws-quickstart/quickstart-bitnami-wordpress

Homework

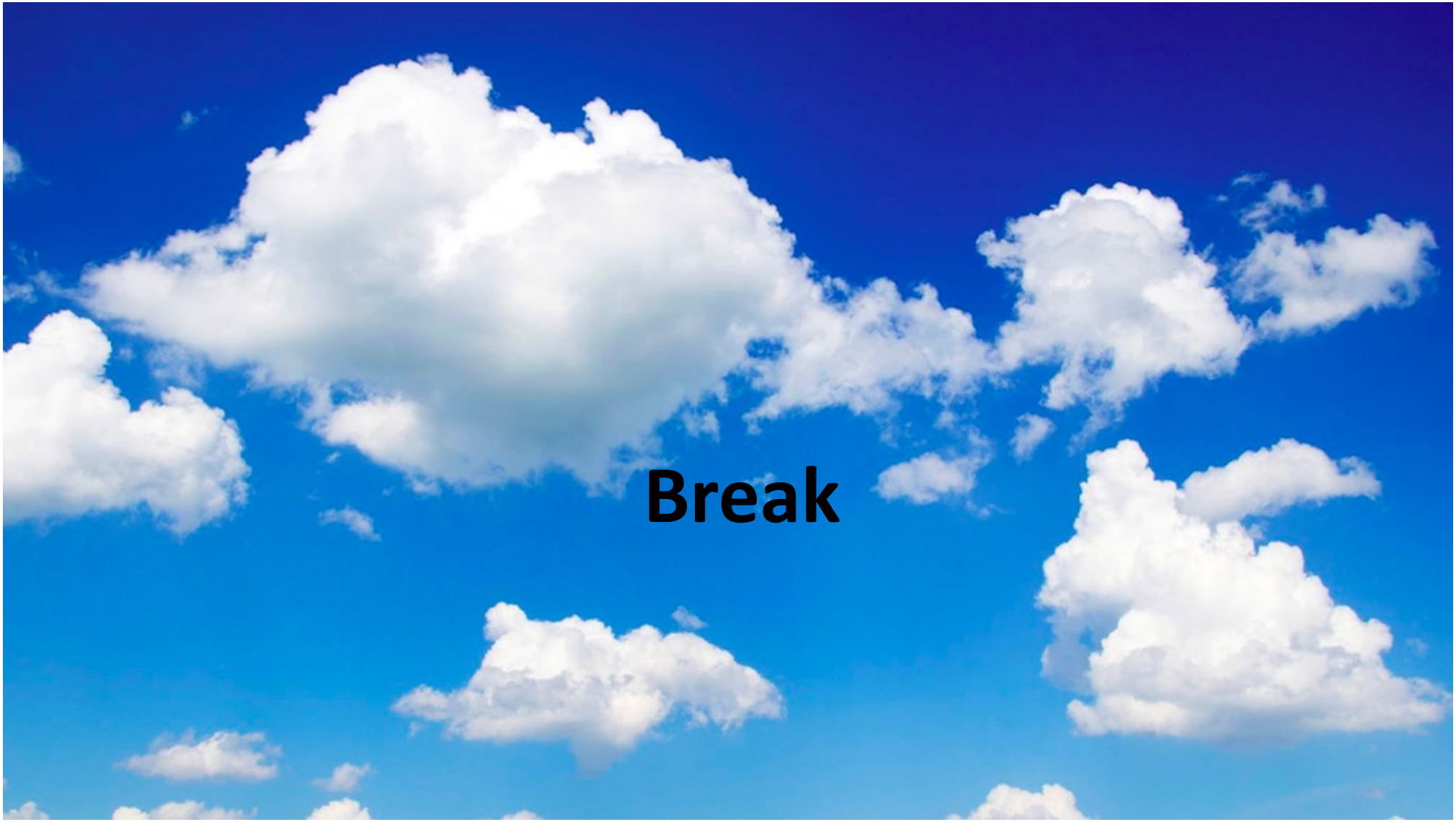
- Complete the exercises demoed during class
  - Build a sample web application with Elastic BeanStalk
  - Check out CloudFormation Stacks & Infrastructure Composer
  - Build a WordPress site with CloudFormation

An EC2 Production Environment

- Monitoring
- Automation
- Lifecycle
- Security



Source: David Clinton, Learn AWS…


Break

Amazon Lightsail

- https://lightsail.aws.amazon.com/ls/webapp/home/instances
- Preconfigured starter environments (**blueprints**) starting with an instance
  - Instances, managed databases, SSD-based storage, backups (snapshots), data transfer, domain DNS management, static IPs, and load balancers, now also containers
  - Fixed monthly price
  - **Not supported by AWS Academy Learner Labs**
- The following tutorial provides a shortcut through our WordPress exercise
  - https://lightsail.aws.amazon.com/ls/docs/en_us/articles/amazon-lightsail-tutorial-launching-and-configuring-wordpress

Lightsail Tutorial

Setting up WordPress

- Step 1: Sign up for AWS
- Step 2: Create a WordPress instance in Lightsail
- Step 3: Connect to your instance via SSH and get the password for your WordPress website
- Step 4: Sign in to the administration dashboard of your WordPress website
- Step 5: Create a Lightsail static IP address and attach it to your WordPress instance (Note: this is an elastic IP address)
- Step 6: Create a Lightsail DNS zone and map a domain to your WordPress instance
- Step 7: Complete the next steps

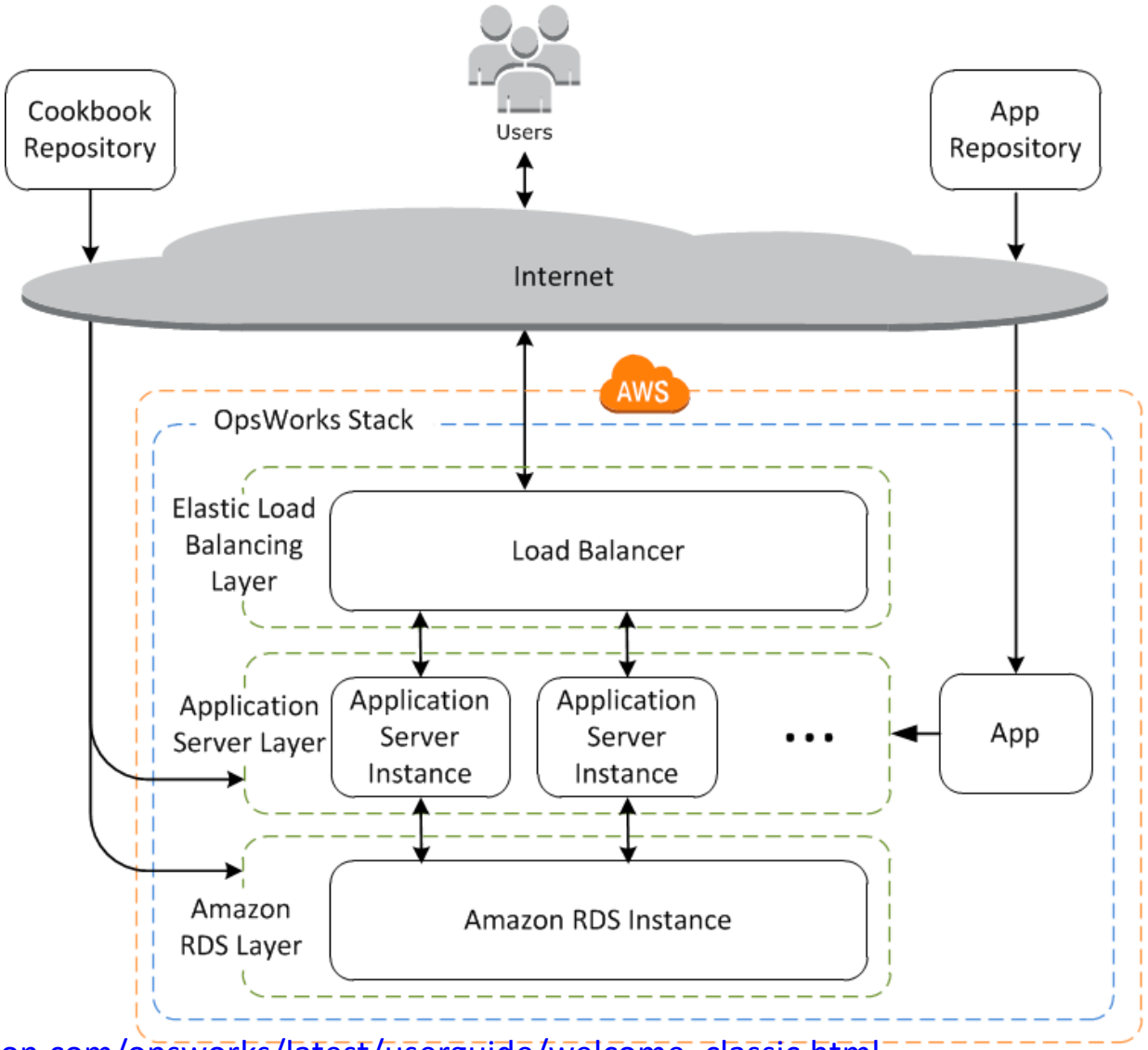
OpsWorks

- OpsWorks offers 3 (agent-based) configuration management solutions working with Chef or Puppet
  - OpsWorks Stacks
    - Uses Chef in local mode
  - OpsWorks for Chef Automate
  - OpsWorks for Puppet Enterprise
- 'Configuration as Code'
- Both Chef and Puppet have large communities and many reusable assets
- For a sample stack, go to OpsWorks console

  https://console.aws.amazon.com/opsworks/home?region=us-east-1#/dashboard!=origin:global

OpsWorks Stacks for Application Server

Chef



https://docs.aws.amazon.com/opsworks/latest/userguide/welcome_classic.html

Chef

- Open source configuration management tool with a domain specific language based on Ruby with a large community
- Uses culinary language
  - A **cookbook** describes a configuration scenario and all its prerequisites. They can be created from **templates.**
  - It contains **recipes** that describe resources and the order in which they are applied
  - A **resource** is a statement of configuration policy describing a target state and the steps which are needed. Resource types are template, service or package.
  - A node is any device under management by Chef
  - A **run-list is** an ordered list of recipes and roles required to bring a node into the desired state, a **role** defines processes belonging to a single job function
  - **Knife** is a command-line tool that provides an interface between a local Chef repository and a Chef server

Sample Chef Recipe

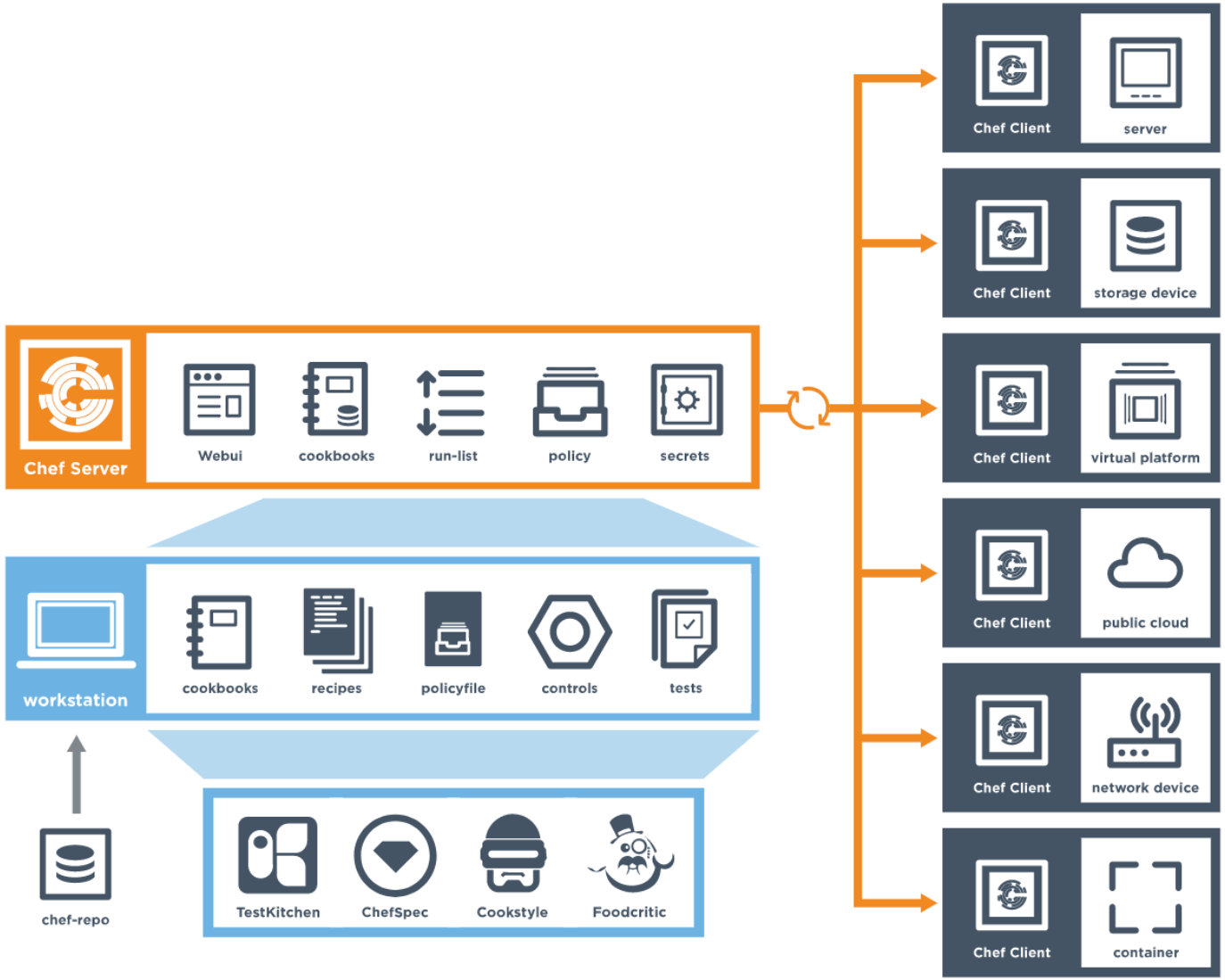
This simple recipe runs ‘apt’ and then installs ‘vim’

```
execute "apt-get update" do
  command "apt-get update"
end

apt_package "vim" do
 action :install
end
```

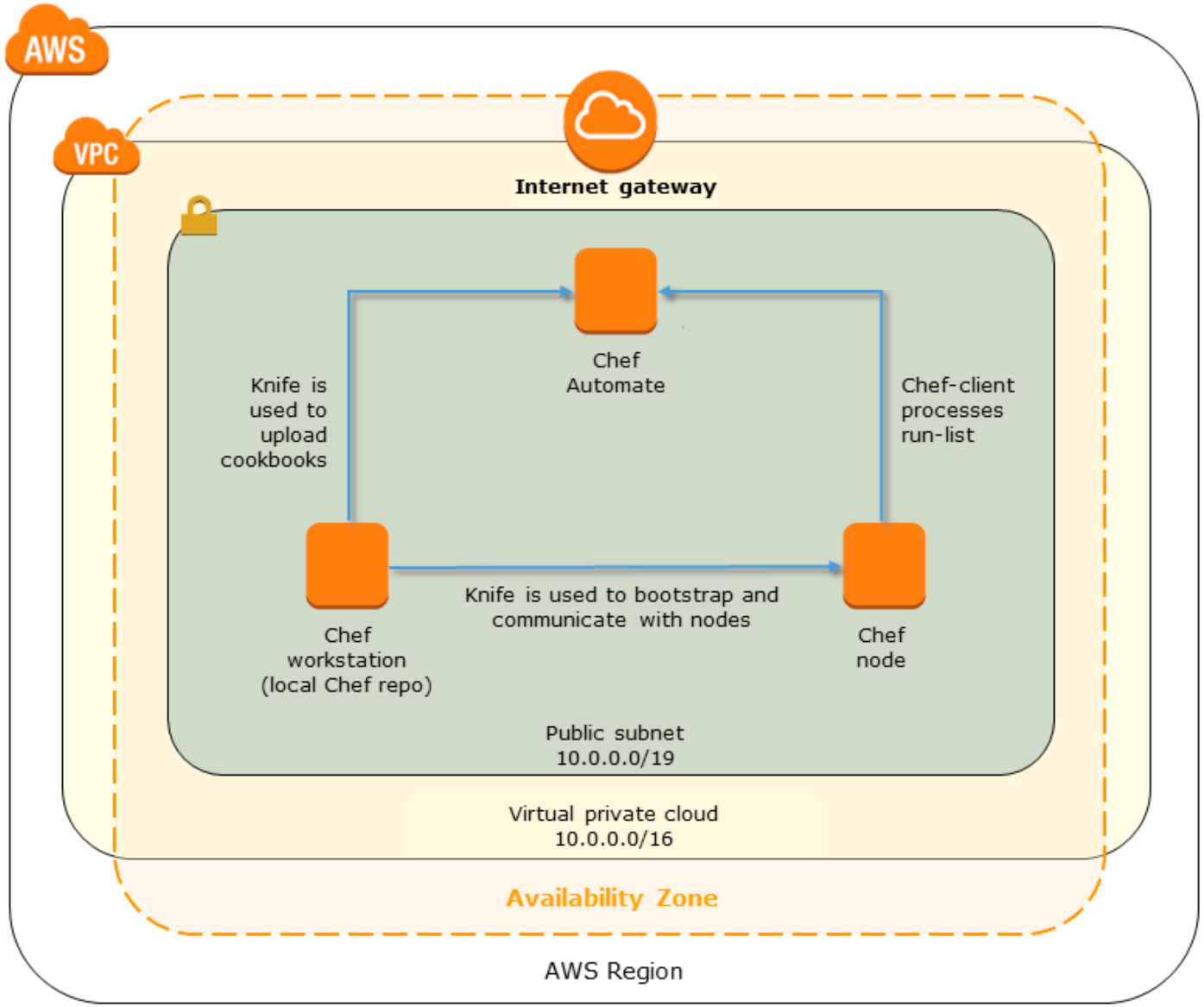
https://www.digitalocean.com/community/tutorials/configuration-management-101-writing-chef-recipes

Chef Structure



https://docs.chef.io/chef_overview.html

Chef Automate on AWS



https://aws.amazon.com/quickstart/architecture/chef-automate/

Homework

- Complete the exercises demoed during class
  - Build a sample WebApp with Elastic BeanStalk
  - Check out CloudFormation Stacks & Infrastructure Composer
  - Build a Webserver with CloudFormation
  - Optionally, build a single instance WordPress with CF

Appendix



Appendix

Accessing Cloud Resources

- AWS Console after login to the account
- AWS Command Line Interface (CLI)
  - Needs configuration of credentials
- SSH Connection to a virtual machine (EC2 instance)
  - Needs configuration of instance and the Virtual Private Cloud it belongs to
  - Needs a key
  - Instance connection uses ssh under the covers
- Through another AWS service
  - Needs permissions granted through IAM roles

Useful Commands

- Limit access to a private key (Linux/MacOS)

  chmod 400 keyname.pem
- SSH to a remote server

  ssh -i keyname.pem ubuntu @xx.xxx.x.xx
- Install the OpenSSH Server package

  apt install openssh-server
- Start the OpenSSH server

  sudo systemctl start ssh
- sudo systemctl start …

Useful Commands (2)

- Create a database in MySQL
  - mysql> CREATE DATABASE wordpressdb;
- Extract an archive using tar
  - tar xzf latest.tar.gz
- Make a working copy of the sample config file
  - cp wp-config-sample.php wp-config.php
- Delete a complete line of text (in nano)
  - Ctrl-K
- Copy the current directory contents to the web root
  - sudo cp -r * /var/www/html

JSON

- Java Script Object Notation
- Data Exchange Format (like XML), works with many languages
- **Key-value pairs**, can be arranged in objects and arrays and be nested
- **JSON Schema** is a vocabulary that allows you to annotate and validate JSON documents

```
var jane = {
    "firstName" : ”Jane",
    "lastName" : ”Smith",
    "age" :  "25"
};
```

```
{"students":[
        {"name":"John", "age":"23", "city":”New York"},
        {"name":"Steve", "age":"28", "city":”London"},
        {"name":"Peter", "age":"32", "city":”Fairfield"},
        {"name":”Jane", "age":”25", "city":”Frankfurt"}
]}
```

- Access e.g. with jane.firstName or students[0].age

boto3: Using Python with AWS

- https://boto3.amazonaws.com/v1/documentation/api/latest/guide/quickstart.html
- To use AWS commands in Python, install boto3

  $pip install boto3 (or pip3…. For Python3)

- To use boto3, you must have installed the CLI and configured it with your credentials
- Features:
  - Low-level APIs mapping HTTP calls to APIs
  - Resource APIs with resource objects
  - Dynamically generated classes driven by JSON models describing AWS APIs
  - 'Waiters' polling for predefined status changes

boto3 Examples

- EC2

```
import boto3

ec2 = boto3.client('ec2')
response = ec2.describe_instances()
print(response)
```

- S3

```
import boto3
# Retrieve the list of existing buckets
s3 = boto3.client('s3')
response = s3.list_buckets()
# Output the bucket names
print('Existing buckets:')
for bucket in response['Buckets']:
        print(bucket["Name"])
```

Running commands at launch of an EC2 instance

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/user-data.html

- You can pass user data to an instance launch to to perform configuration tasks and run scripts
- Either pass it into a user data field when creating an instance, or use the console after it has been created
  - Stop the instance. When stopped, choose 'View/change user data' from Instance Settings
- The script below installs and configures a web server on Amazon Linux. Note user shell scripts need to start with #!

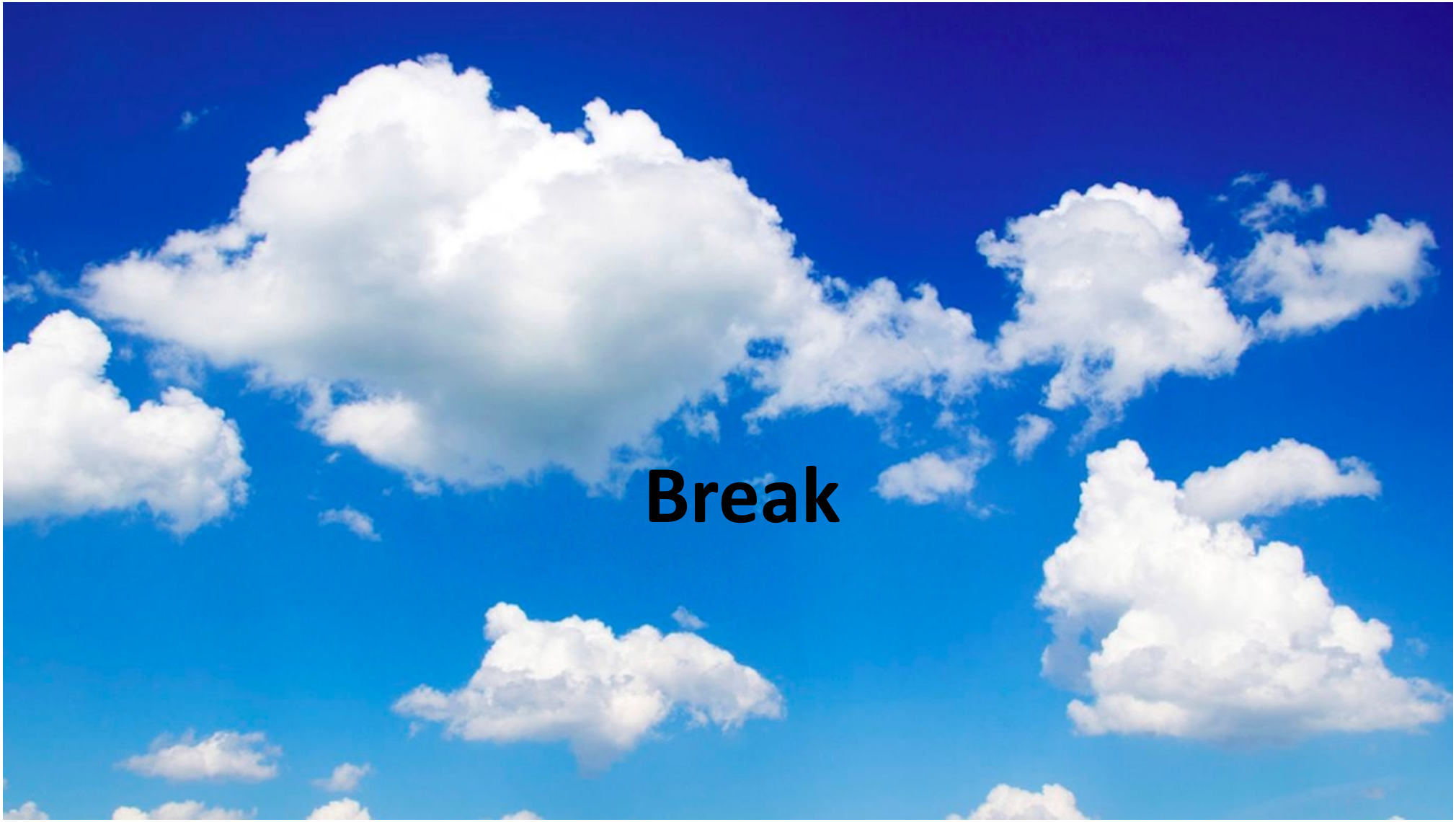
```
#!/bin/bash
yum update -y
amazon-linux-extras install -y lamp-mariadb10.2-php7.2 php7.2
yum install -y httpd mariadb-server
systemctl start httpd
systemctl enable httpd
usermod -a -G apache ec2-user
chown -R ec2-user:apache /var/www
chmod 2775 /var/www
find /var/www -type d -exec chmod 2775 {} \;
find /var/www -type f -exec chmod 0664 {} \;
echo "<?php phpinfo(); ?>" > /var/www/html/phpinfo.php
```

Connecting to a Linux Instance

- https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AccessingInstancesLinux.html
- If you have problems with SSH on Windows, use PuTTY to connect to user_name@public_dns_name.
  - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/putty.html
- Transferring a file with scp
  - scp -i /path/my-key-pair.pem /path/SampleFile.txt ubuntu@ec2-198-51-100-1.compute-1.amazonaws.com:~
  - For Amazon Linux replace ubuntu with ec2-user
  - You may have to install the scp client on your Linux machine

Windows 10

- Windows 10 Home supports
  - AWS CLI
  - Python
- Windows 10 Professional is required for
  - Containers/Docker
- Enable the Windows Subsystem for Linux and install a free Linux distribution
  - Does not support Docker
- Install VirtualBox and create a Linux Virtual Machine
  - Supports full Linux environments


Break